

Technical Design Whitepaper

PartsModel

The Owned Part Tree: Leaf Parts, One Mutation Authority, Gated Composite Mass Properties

How a rocket is represented, edited, persisted, displayed, and flown — from a pure-leaf Part through the node tree that owns it to the integrator that consumes it.

Status	As built — implemented and gate-verified on branch <code>PartsModel</code>
Scope	<code>core/model</code> (<code>PartsModel</code> , <code>parts</code> , <code>DesignSerializer</code>), <code>cli</code> , <code>gui tree view</code> , <code>visualizer</code>
Target	QtRocket simulation core (C++23)
Audience	Maintainers and future contributors of the simulation core

Abstract

A rocket in QtRocket is a tree of physical components, and nearly everything the program does — editing a design in the CLI, painting the GUI tree view, rendering the 3D visualizer, integrating a flight — is a walk over that tree. This whitepaper documents the architecture that carries all of those roles: a strict three-way decomposition into **Part**, a pure leaf that knows one component’s mass, geometry profile, and aerodynamic contribution and nothing else; **PartNode**, the owned tree node that binds a leaf to its placement link, its children, and the cached composite quantities of its sub-tree; and **PartsModel**, the single mutation authority that owns the root, indexes every node by stable id, enforces the single-motor invariant, and narrates every edit through a typed aboutTo/did event stream.

The load-bearing commitments are few and strict. Ownership is linear (`unique_ptr` end-to-end), so attaching a part is surrendering it: double-attachment, ownership cycles, and retained mutable aliases are unrepresentable rather than checked. Reads hand out `const` references; the only post-attach writes are routed verbs on **PartsModel** that perform the edit and then invalidate exactly the caches the edit dirtied. Composite mass, CM, and inertia rebuild behind a mass-delta gate keyed on a live mass sum — cheap enough to run every integrator stage, exact enough to freeze bit-for-bit after burnout — while geometry resolves once per structural edit and never during a burn. Structural errors are typed values, a self-intersecting design is a hard located failure on every read, and “no design” is a representable state, not a placeholder.

The document specifies each class contract with UML (PlantUML) and the control flow with activity and component diagrams (TikZ), walks the two execution paths end to end — a CLI design session from `newdesign` to `savedesign`, and a simulated flight from ignition through the burn to the post-burnout cache freeze — and records the design decisions and their rejected alternatives with the rationale that selected them.

Three non-negotiable rules

1. One authority mutates. Every post-attach edit is a **PartsModel** verb that writes and then dirties the correct cache chain; reads are `const`. **2. Ownership is linear.** Nodes own leaves and children uniquely; the type system, not a runtime check, forbids aliasing. **3. Derived state is gated, never trusted.** Composite quantities rebuild when their inputs moved and freeze when they did not — and a failed geometry solve throws on *every* read.

Contents

Abstract	ii
1 Introduction: one tree, four consumers	1
1.1 A rocket is a tree	1
1.2 Four consumers, one representation	1
1.3 Three roles and a bridge	2
1.4 Where each topic lives	3
2 Design philosophy and invariants	3
2.1 One authority, one owner	5
2.2 Derived state is gated	6
2.3 One id space	7
2.4 Typed errors, and a mutation that cannot silently fail	7
2.5 The empty state is real	8
2.6 Single-writer by contract	8
2.7 Events shaped for the strictest subscriber	9
3 The Part leaf	9
3.1 Identity: id, name, type tag	9
3.2 Mass properties: the per-unit-mass split	11
3.3 The aerodynamic contribution	12
3.4 The geometry profile	12
3.5 Duplication: clone() through a protected copy	13
3.6 The routed-mutation seam	14
3.7 The concrete leaves	14
4 The node tree: PartNode and PartsModel	16
4.1 The node: an owned interior, a const read surface	17
4.2 Building detached trees: make, addChild, clone	18
4.3 The model: identity, lookup, traversal	19
4.4 Structural verbs and typed errors	20
4.5 Routed leaf edits and the mutation hint	20
4.6 The event stream	23
4.7 The install facade	24
5 Composite mass properties and the two cache gates	25
5.1 One live sum, two cached tensors	25
5.2 The mass-delta gate	26
5.3 The placement gate	27
5.4 computeCompositeAt: two passes over one resolve	29
5.5 Aero and area: aggregates on the same resolve	29
5.6 What the flight loop pays	31
5.7 The single-writer contract	31

6	Placement integration	32
6.1	The placement vocabulary	32
6.2	Per-edge geometry: placeChild	35
6.3	The node-tree resolver	35
6.4	Seam and envelope diagnostics	36
6.5	The diagnostics gate	38
7	The motor subsystem	38
7.1	The Motor leaf: everything derives live	40
7.2	A point in the airframe geometry	42
7.3	The verbs and the typed borrow	42
7.4	Const thrust, one runtime mutation	45
7.5	Facade, persistence, and clients	46
8	Design persistence: the .qrd format	46
8.1	File anatomy and the version gate	47
8.2	Parameters: the factory round-trip	47
8.3	Placement: the <link> element, elided by default	49
8.4	The motor: a database reference	49
8.5	Load: build detached, install once	50
8.6	The <sim> block and the override tri-state	50
9	Execution walkthrough: a CLI design session	53
9.1	The line protocol	53
9.2	The session	54
9.3	Building the tree: newdesign and addpart	54
9.4	Inspecting the design: listparts and checkdesign	56
9.5	setmotor: the database meets the tree	58
9.6	Persistence and the no-design contract	59
9.7	The launch gate	59
10	Execution walkthrough: a simulated flight	60
10.1	Ignition	60
10.2	The loop and its stages	61
10.3	One stage, itemized	62
10.4	The accepted step: recording through the gate	63
10.5	Termination and the flight record	65
11	GUI integration: the Qt item model over the event stream	66
11.1	Index identity: internalId carries Part::Id	67
11.2	Structure derived live	67
11.3	Consuming the aboutTo/did stream	68
11.4	What granularity buys	68
11.5	The subscription chain and its lifetime	70
11.6	Edit widgets route through the verbs	70
11.7	Verification: the model contract under test	71

12 Design decisions and rejected alternatives	72
12.1 Ownership and structure	72
12.2 Cache placement and the gate	73
12.3 Routed mutation	74
12.4 The motor	75
12.5 Errors, events, and the empty design	76
12.6 Concurrency: single-writer now, snapshots later	77
13 Verification strategy	78
13.1 Behavioral seams, not white-box hooks	78
13.2 Unit pins on the tree and its gates	80
13.3 Assembled-system pins: round trips and the corpus baseline	81
13.4 Black-box command contracts	82
13.5 The Qt adapter contract	83
13.6 The corpus transcript diff	83
13.7 Toolchain gates	84
A Glossary	85
B Invariants and guarantees	87

1 Introduction: one tree, four consumers

QtRocket is an open-source model-rocket simulator: a Qt6 Widgets GUI and a Qt-free command-line REPL built on a shared C++23 simulation core, with a standalone OpenGL viewer alongside them. In that core, a rocket is exactly one thing — a tree of physical components. A nose cone sits on a body tube; fins and a motor mount attach to the tube; a motor seats in the mount. The tree is not a rendering convenience or a UI affordance: it is the physical description from which every quantity the simulator needs (mass, center of mass, inertia, aerodynamic coefficients, thrust) is derived, and every view the user sees is drawn.

This document describes the architecture that carries that tree: a pure-leaf component class (`Part`), the owned node that composes leaves into assemblies (`PartNode`), the single mutation authority that owns the whole structure (`PartsModel`), and the bridge that presents the finished rocket to the numerics (`RocketModel`). This section establishes the shape of the design at survey depth — what the tree is, who reads it, and which class answers for what — and closes with a map of where each topic is developed in full.

1.1 A rocket is a tree

The leaves of the tree are concrete components, defined under `core/model/parts/`: `ConicalNoseCone`, `BodyTube`, `FinSet`, `HollowSphere`, and `Motor`, a zero-length node that wraps the time-aware `MotorModel` so that burning propellant mass is just another leaf in the composite sum (Section 7). Each leaf knows only itself: its own mass in kg, its per-unit-mass inertia tensor about its own CM, its closed-form silhouette radii, and its Barrowman aerodynamic contribution (Section 3). Units are SI throughout, and every part shares one local frame convention — $+z$ = forward, the part occupying $z \in [-L, 0]$ (Section 2).

The edges of the tree carry *placement intent*, not coordinates: a `StationLink` (`core/model/parts/Placement.h`) names a fractional station on the parent, a fractional station on the child, a seat kind, and a gap in metres — “this rim seats against that rim,” “this tube nests in that bore.” A single resolver turns intent into absolute geometry once per structural edit, and an envelope sweep passes an overlap verdict on the result (Section 6). Because stations are fractions of live length, resizing a part moves every seam that references it; nothing downstream stores a stale coordinate.

Finally, the empty tree is a real state. `PartsModel::hasDesign()` is false until a design is installed, and every consumer is written against that possibility; there is no placeholder part standing in for “nothing yet” (Section 4).

1.2 Four consumers, one representation

Four independent subsystems consume the tree, and none of them holds a private copy of it:

1. **The CLI editor** (`cli/Repl.cpp`) builds and inspects designs interactively: `newdesign` installs a root, `addpart` resolves a parent by id, name, or the literal `root` and calls the `attach` verb (`cli/Repl.cpp:952`), `listparts` walks the tree in attachment order printing composite mass and CG, `checkdesign` reports the envelope sweep’s overlap verdict, and `savedesign/loaddesign` round-trip the design through the versioned XML serializer (Sections 8 and 9).
2. **The GUI tree view** (`gui/RocketTreeView.h`) adapts the live tree to Qt: `RocketPartModel` is a `QAbstractItemModel` whose rows are the tree’s children spans, and which consumes the typed `aboutTo/did` event stream so that an attach or detach becomes a surgical row insert or remove rather than a model reset (Section 11). Edits made elsewhere in the GUI route through the same verbs — the cannonball preset writes its mass via `setPartMass` (`gui/CannonballTab.cpp:130`).

3. **The 3D visualizer** (`visualizer/RocketMesh.cpp`) meshes each part at its resolved pose. It reads the root node's cached `resolvedPlacements()` (`visualizer/RocketMesh.cpp:433`) — the same poses the composite mass pass integrates over — and reads `placementDiagnostics()` to paint overlap offenders in an error color with a located message.
4. **The flight integrator** (`core/sim/Propagator.cpp`) never sees parts at all. It drives the `Propagatable` interface, evaluating `getForces(...)` / `getMass(t)` at every solver stage (`core/sim/Propagator.cpp:41`) and recording composite mass properties once per recorded step. `RocketModel` answers every one of those calls from the root node's composite queries, as [Listing 1](#) shows ([Section 10](#)).

```
void RocketModel::writeMassProperties(double t, StateData& st)
{
    // Via the gated accessors, so after burnout this reads the frozen cache. The three are mutually
    // consistent at this t.
    st.mass    = parts_.root()->compositeMass(t);
    st.cg      = parts_.root()->compositeCm(t);
    st.inertia = parts_.root()->compositeI(t);
}
```

Listing 1. The model side of the sim bridge delegates every mass property to the root node's cached composite queries (`core/model/RocketModel.cpp:49`). The integrator flies whatever the tree says — there is no second bookkeeping of mass anywhere in the sim path.

Why one representation matters

The pose the visualizer draws a coupler at and the CM the integrator flies are derived from the *same* cached resolve on the *same* node; the mass column in the GUI and the CLI's `listparts` read the same leaf values through the same `const` accessors. Consumers cannot disagree about what the rocket is, because no consumer owns a copy that could drift. Where representations must genuinely differ in shape — Qt model indexes, GL vertex buffers — they are throwaway projections rebuilt from the tree, never parallel sources of truth.

Rejected — per-consumer scene copies

The conventional alternative gives each front end its own mirror: the viewer keeps a scene graph, the GUI keeps a shadow item tree, and change-propagation code keeps them synchronized. Every such mirror is a place for the rocket to silently become two rockets — a missed notification is not an error anyone sees, it is a viewer that draws a design the integrator is not flying. The design here spends its effort on making the one tree cheap to read (cached composites, a cached resolve, $O(1)$ id lookup) instead of on synchronizing copies, so the failure mode does not exist.

1.3 Three roles and a bridge

The tree is carried by three classes with deliberately unequal jobs, plus one bridge above them. [Figure 1](#) places them among their consumers.

Part (`core/model/parts/Part.h:34`) is a pure leaf: one component's mass, geometry profile, and aero contribution, with no children, no parent, and no knowledge that trees exist. Its mutating setters are private to the `PartsModel` seam, so a leaf cannot be edited behind the caches' back. [Section 3](#) gives its full contract.

PartNode (`core/model/PartsModel.h:42`) is one node of the owned tree: it owns its `Part`, stores the incoming `StationLink`, owns its children, and caches the derived quantities of its sub-tree — composite mass, CM, and inertia behind a mass-delta gate, and the resolved placements plus overlap verdict behind a placement gate. Any node is a valid sub-assembly root, so the same queries that describe the whole rocket describe a stage or a pod. [Section 4](#) covers the structure, [Section 5](#) the two gates, and [Section 6](#) the resolve itself.

PartsModel (`core/model/PartsModel.h:133`) is the single owner and the only mutation authority. It holds the root, an $O(1)$ id index so stale references fail safe, and the motor slot with its at-most-one invariant ([Section 7](#)). Every structural edit is a typed verb — `attach`, `attachSubtree`, `detach`, `installRoot` — and the fallible ones report `std::expected` errors rather than throwing on misuse. Every post-attach edit below the structural level is a routed verb — `setPartMass` for mass, `setLink` to re-seat a seam, `mutatePart` for arbitrary leaf edits under a declared hint — that writes and then dirties exactly the cache chain the edit touched. Around every mutation it fires a typed `aboutTo/did` event pair, which is precisely the shape Qt’s row-operation contract requires ([Section 11](#)).

RocketModel (`core/model/RocketModel.h:27`) sits above as the rocket-level facade: it owns the `PartsModel`, implements `Propagatable` for the sim ([Section 10](#)), carries the drag configuration, and wraps design installation in `installDesign` so install-time policy (such as resetting a manual reference-area override) lives in one place.

One representation, many readers, one writer

Reads are `const` everywhere — `children()`, `find()`, the composite queries, the resolved placements. The only write paths are `PartsModel` verbs. Consequently every consumer observes every edit, no edit can skip cache invalidation, and adding a fifth consumer is a matter of reading the same tree, not of joining a synchronization protocol.

1.4 Where each topic lives

The rest of the document develops each role in the order the dependencies stack. [Section 2](#) states the design rules the whole structure obeys — one mutation authority, linear ownership, gated derived state — and fixes the geometric conventions. [Section 3](#) specifies the `Part` leaf and its concrete types; [Section 4](#) the `PartNode` tree and `PartsModel`’s verbs, identity model, and typed errors. [Section 5](#) details the two cache gates and why the ODE-hot path never re-resolves geometry; [Section 6](#) the station-link placement model, the per-edge pose math, and the envelope sweep. [Section 7](#) covers the motor slot and its single-motor invariant; [Section 8](#) the versioned `.qrd` design format. Two walkthroughs then trace the system end to end: a CLI editing session in [Section 9](#) and a full simulated flight in [Section 10](#). [Section 11](#) describes the Qt adapter built on the event stream, [Section 12](#) records the load-bearing trade-offs and rejected alternatives in one place, and [Section 13](#) documents the test architecture that pins all of it down.

2 Design philosophy and invariants

The part tree is the most widely shared piece of state in QtRocket: the ODE loop reads its mass properties per integrator stage ([Section 10](#)), the serializer walks it ([Section 8](#)), the CLI addresses into it ([Section 9](#)), and the Qt tree view mirrors it row-for-row ([Section 11](#)). A structure with that many consumers cannot rely on discipline; it needs invariants that hold *by construction*. This section states the eight commitments that shape every type and function in the design. Each one is chosen to move a class of defect from “a bug someone must remember not to write” to “a state the program

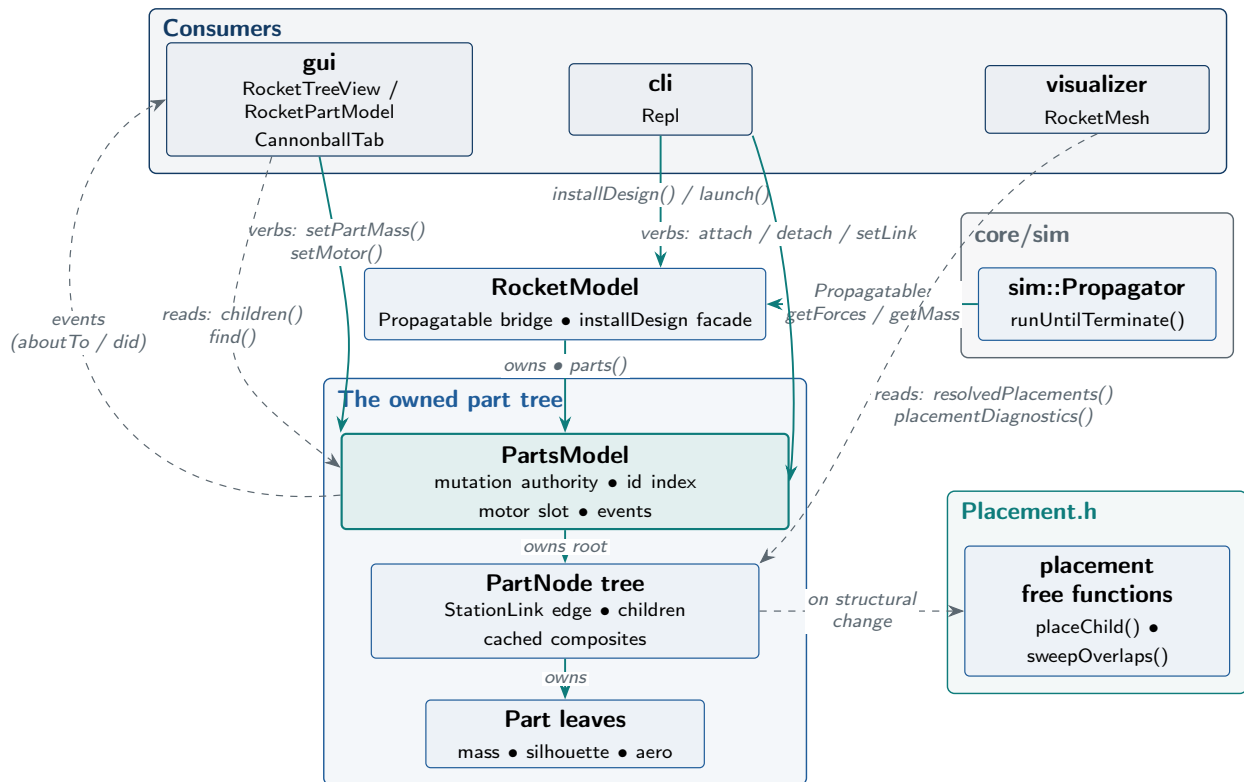


Figure 1. One tree, four consumers. The front ends and the integrator (top and right) reach the owned part tree only through `RocketModel`'s facade and `PartsModel`'s verbs and `const` reads; the typed event stream (dashed, left) flows back out to the GUI adapter. `PartNode` calls the pure placement functions of `core/model/parts/Placement.h` only when a structural edit dirties its geometry cache.

cannot represent” — or, where full unrepresentability is impossible, to a hard, located failure instead of a silently wrong number.

The commitments are stated here as rules, before any type is introduced, because their value is precisely that they constrain everything downstream: the leaf contract (Section 3), the tree and its verbs (Section 4), the cache gating (Section 5), the motor slot (Section 7), and the GUI adapter (Section 11) are all forced to be mutually consistent with them.

The eight invariants

- 1. One mutation authority.** Reads hand out `const Part&`; every post-attach write is a `PartsModel` verb that writes and then invalidates the correct cache chain.
- 2. Linear ownership.** Nodes own their `Part` and their children by `std::unique_ptr`; double-attach, cycles, and retained aliases are uncompileable.
- 3. Derived state is gated.** Caches rebuild iff their inputs moved, and a gate is stamped only after a successful build.
- 4. One id space.** `Part::Id` — process-unique, fresh on clone, never serialized — is the only identity that survives across edits.
- 5. Typed errors.** Structural verbs return `std::expected`; a failed mutation is a value the caller can switch on, never a silent no-op.
- 6. “No design” is representable.** The root is nullable and `hasDesign()` reports it; there is no placeholder part.
- 7. Single-writer by contract.** The `const` accessors memoize through mutable caches; all access comes from one thread at a time, and the contract is documented where it can be broken.
- 8. Events are aboutTo/did pairs.** Every mutation fires one notification before and one after the tree changes, carrying the row coordinates Qt’s `beginInsertRows/endInsertRows` contract requires.

2.1 One authority, one owner

The first two invariants are one idea seen from two sides: *who may write* and *who may hold*. `PartsModel` is the single mutation authority (`core/model/PartsModel.h:133`): reads hand out `const Part&`, and the only post-attach write paths are its routed verbs — `setPartMass`, `setPartInertia`, `setLink`, `mutatePart`, the structural `attach/detach` family, and the motor verbs (Section 4). Each verb performs the plain write and then dirties exactly the cache chains that write can invalidate. The leaf cooperates by construction: `Part`’s setters are *private*, exposed only through a single friend seam.

```
private:
    friend class model::PartsModel; // the sole post-attach mutation seam

    /// Plain writes: PartsModel's routed verbs own cache invalidation.
    void setMass(double m) { mass = m; }
    void setI(const Matrix3& I) { inertiaTensor = I; }
```

Listing 2. The mutation seam on the leaf (`core/model/parts/Part.h:133`). The setters carry no invalidation duty and no virtual dispatch; routing and invalidation are entirely the authority’s job.

Invalidation cannot be skipped, by construction

Cache invalidation is the classic “must remember” duty: one forgotten dirty-flag write and a composite tensor is silently stale for the rest of the session. Here the compiler carries the duty. Client code cannot reach `setMass` at all — the only path to a leaf write runs through a `PartsModel` verb, and every verb pairs the write with its invalidation (`core/model/PartsModel.cpp:399`). The one deliberately open door, `mutatePart`, requires the caller to declare a `MutateHint` (`core/model/PartsModel.h:139`) so that even the escape hatch states which chains its edit can dirty — a mass-only edit never re-runs the geometry sweep.

Ownership is equally linear. Each `PartNode` owns its `Part` and its children through `std::unique_ptr` (`core/model/PartsModel.h:108`); `PartsModel` owns the root; `RocketModel` owns the model. A part therefore has exactly one owner at every instant of its life, and the transfer points are visible in the signatures: `attach consumes` a `std::unique_ptr<part::Part>`, and `detach returns` the owned sub-tree to the caller as a first-class value (`core/model/PartsModel.h:216`) — the natural undo and clipboard memento. Duplication has exactly one spelling, `clone()`, and the assignment operators are deleted so a subclass can never be sliced into a base `Part` (`core/model/parts/Part.h:56`).

Rejected — shared ownership with runtime membership checks

The alternative is a `std::shared_ptr` forest in which the tree checks, at attach time, that a part is not already attached elsewhere. This demotes every ownership violation from a compile error to a runtime bug class: double-attach becomes a check that must be written (and reached), a cycle becomes a traversal that must terminate, and — worst — a *retained alias* remains perfectly legal: any client that keeps its `shared_ptr` after attaching can write through it forever, past every routed verb, producing precisely the unsanctioned mutation invariant 1 exists to eliminate. Under `unique_ptr` the retained alias is uncompileable rather than unsanctioned. The mass-delta gate of [Section 5](#) remains as a backstop for any residual `const_cast` -class bypass, but the front door is closed by the type system, not by review.

2.2 Derived state is gated

Composite mass properties, resolved placements, and the overlap verdict are all *derived* state: pure functions of the leaves, the links, and (for mass) the query time. Invariant 3 says derived state may be cached only behind a gate that provably tracks its inputs. The design has exactly two gates, one per kind of input motion: the *mass-delta gate* rebuilds the composite CM and tensor when the live mass sum moves (a burning motor) or a mass edit lands, and the *placement gate* re-resolves geometry and re-runs the envelope sweep only when a node or link changes — geometry is invariant under a burn, so the hot ODE path never pays for a re-resolve. [Section 5](#) develops both gates in full.

What belongs to the philosophy is the stamping rule, because it is a correctness property and not an optimization:

Stamps land only after success

`ensureCompositeCache` computes the rebuild *first* and writes the gate stamps (`builtAtMass_`, clearing `massDirty_`) only after `computeCompositeAt` returns (`core/model/PartsModel.cpp:122`). The ordering is load-bearing: a self-intersecting design makes the composite build throw (`core/model/PartsModel.cpp:184`), and because nothing was stamped, the throw recurs on *every* read. A cache that recorded the failed build as done would convert a hard, located failure into a

stale tensor served indefinitely — the worst outcome available to a simulator, a plausible wrong number.

2.3 One id space

Every consumer that must refer to a part across edits — GUI selection, CLI commands, event payloads, the model’s own index — uses the same handle: `Part::Id`, a `std::uint64_t` minted from a process-wide atomic counter at construction (`core/model/parts/Part.cpp:16`). The counter starts at 1 so that 0 is reserved as “none/invalid,” which the event payloads exploit for roots and clearing resets (`core/model/PartsModel.h:147`). Three properties make one id space sufficient:

- **Fresh on clone.** Every constructor mints a new id, including the protected copy constructor that backs `clone()` (`core/model/parts/Part.cpp:37`). Two parts never share an id, however identical their names and mass properties.
- **Never serialized.** Ids are meaningful only within a single run (`core/model/parts/Part.h:113`); a loaded design mints fresh ids. Persisting them would import cross-file collision handling into every load path for no benefit — nothing outside the process ever needs the handle.
- **Fails safe when stale.** `PartsModel::find` is an $O(1)$ id-keyed lookup that returns `nullptr` for an absent id (`core/model/PartsModel.h:167`). A consumer holding the id of a detached part gets a testable miss, where a retained pointer would dangle.

Names are labels; ids are identity

Part names are human-facing and deliberately non-unique; `findByName` exists for console convenience and is documented as pre-order first-match (`core/model/PartsModel.h:179`). No correctness property anywhere in the system hangs on a name. There is also no second handle type — no node pointer or row index offered as an identity — because every additional handle kind is one more thing a consumer can hold stale. One id space means one staleness story, and it is fail-safe.

2.4 Typed errors, and a mutation that cannot silently fail

Structural verbs return `std::expected: attach` and `attachSubtree` yield the new id or an `AttachError` (`NullPart`, `NoSuchParent`, `DuplicateMotor`); `detach` yields the owned sub-tree or a `DetachError` (`NoSuchId`, `IsRoot`) (`core/model/PartsModel.h:136`). The leaf-edit verbs, whose only failure mode is an unknown id, return `bool`. In every case failure is a value at the call site: the CLI switches on the error code and prints the precise cause (`cli/Repl.cpp:957`), and a test can assert the exact rejection rather than probing for side effects.

Rejected — logged void no-ops

The tempting minimal interface — `void attach(...)` that logs and returns on failure — makes failure invisible at the call site. A caller that must know whether the attach happened is reduced to diffing the tree’s size before and after, and even then the *cause* is gone: was the parent id wrong, the part null, the motor slot occupied? The log knows; the program does not. Errors-as-values costs one `std::expected` in the signature and buys call sites that are impossible to write incorrectly by accident: for `detach`, the success value is the detached sub-tree, so a caller cannot take the value while ignoring the possibility of failure — they arrive in the same object.

The `DuplicateMotor` rejection illustrates why the errors are typed rather than boolean: the single-motor invariant is enforced at the doors through which a part can join an owned tree — `attach` and `attachSubtree`, which scans the entire incoming sub-tree (`core/model/PartsModel.cpp:319`) — and the caller is told *that* the motor slot is the objection, not merely that “something” failed ([Section 7](#)).

2.5 The empty state is real

A freshly started session, or one whose design was cleared, has no rocket. The design represents that state honestly: the root is nullable (`core/model/PartsModel.h:259`), `hasDesign()` reports it (`core/model/PartsModel.h:161`), `size()` is 0, and every query fails safe — `find` misses, the pre-order walk visits nothing. Consumers respond in their own idiom; the CLI’s contract is an explicit `ERR . . . : no design` on commands that need one ([Section 9](#)).

Rejected — a mandatory placeholder part

The alternative is to guarantee a non-null root by installing a placeholder part in an empty model. This trades an honest null for a dishonest object: the placeholder has *some* mass, *some* geometry, and a simulation will happily consume both — flying a rocket that does not exist and returning plausible numbers for it. Every consumer must then ask “is this the placeholder?” before trusting the tree, which is exactly the null check again, now spelled as a magic-value comparison with strictly worse failure modes when someone forgets it. `hasDesign()` is the same predicate with a compiler-visible spelling and no physics attached.

2.6 Single-writer by contract

The composite accessors are `const` but not concurrently callable, and the design says so out loud rather than letting the signatures imply otherwise. `compositeCm`, `compositeI`, `placementDiagnostics`, and `resolvedPlacements` all memoize through mutable caches (`core/model/PartsModel.h:114`), and the motor’s log-once burnout latch writes inside `const thrust(t)` (`core/model/MotorModel.h:342`). Concurrent access — even all-`const` — is therefore a data race outright. The contract is single-writer: all access from one thread at a time, stated in the class documentation and again at the cache fields (`core/model/PartsModel.h:39`), so that any future threading work reopens the question deliberately instead of trusting `const`.

The race is deeper than torn reads

Even a hypothetically race-free interleaving would be wrong: the composite CM and the composite tensor are only meaningful as a *pair* from the same rebuild — the tensor is expressed about that CM. Two threads interleaving through the gate could pair a CM from one rebuild with a tensor from another: two individually valid values forming a physically meaningless answer. Any concurrency story must therefore preserve multi-value consistency, not merely atomicity of individual reads.

The recorded direction, should a real concurrency requirement appear, is *snapshotting*: each thread flies its own `PartNode::clone()` and only the handoff synchronizes. [Section 12.6](#) develops that plan and the rejected locking alternative.

2.7 Events shaped for the strictest subscriber

Every mutation — structural, link, leaf edit, motor swap, and whole-tree reset — fires a change notification as an *aboutTo/did* pair: the same `Event` value delivered once with `before == true` immediately before the tree changes and once with `before == false` immediately after (`core/model/PartsModel.h:143`). The payload is `{kind, id, parentId, row}` — the affected node, its (ex-)parent, and its row within that parent.

That shape is dictated by the most demanding subscriber the model must serve. Qt’s `QAbstractItemModel` contract requires `beginInsertRows`/`beginRemoveRows` to be called *before* the underlying data changes, with parent-and-row coordinates, and the matching `end` call after. Because the event stream fires exactly that way, the GUI adapter is a one-to-one mapping with no bookkeeping of its own: `aboutTo` maps to `begin*`, `did` maps to `end*`, and the payload supplies the coordinates verbatim (`gui/RocketTreeView.cpp:29`; [Section 11](#)). Subscribers with weaker needs — a view that repaints wholesale — simply ignore the `before` leg. The registration is a single callback slot rather than a multicast list (`core/model/PartsModel.h:248`): the model layer stays Qt-free, one subscriber exists, and the callback fires synchronously on the mutating thread, which the single-writer contract already pins.

None of these invariants is free-standing: the leaf’s private seam ([Section 3](#)) exists to serve invariant 1, the verbs of [Section 4](#) are shaped by invariants 1, 2, and 5, the gates of [Section 5](#) are invariant 3 made concrete, and the trade-off discussion of [Section 12](#) weighs what each rule costs. The test suite exercises them directly — rejected attaches, stale-id misses, the empty state, and the after-success stamping rule are all asserted behaviors, not aspirations ([Section 13](#)).

3 The Part leaf

A `Part` (`core/model/parts/Part.h`) answers exactly four questions about one physical component: who it is (a stable id, a human-facing name, a type tag), what it weighs and how that mass is distributed (own mass, a per-unit-mass inertia tensor, a CM offset), what silhouette it presents along the axis (length and closed-form radius profiles), and what it contributes to the Barrowman stack (a normalized `AeroComponent`). It answers nothing else. There is no parent pointer, no child list, no placement, no cache, and no composite quantity anywhere on the class — the tree structure and everything derived from it belong to `PartNode` and `PartsModel` ([Section 4](#)), and the derived state is cached there behind the two gates of [Section 5](#). The leaf is the passive, physical vocabulary; the tree is the grammar.

This division is enforced at the type level rather than by convention. A part is a unique-ownership value: assignment is deleted, the copy constructor is protected, and duplication exists only as the virtual `clone()`. Post-attach mutation exists only as two private setters whose sole friend is `PartsModel`, so an edit that skips cache invalidation is not merely discouraged — it does not compile. [Figure 2](#) shows the full contract and the five concrete leaves; the rest of this section walks it top to bottom.

3.1 Identity: id, name, type tag

Every part carries a `Part::Id` (`std::uint64_t`), drawn from a process-wide atomic counter at construction (`core/model/parts/Part.cpp:17`). The counter starts at 1, reserving 0 as the “none/invalid” value that other layers use for a root’s parent or a cleared event field; it increments with `std::memory_order_relaxed` because uniqueness, not ordering, is the contract. Every constructor assigns a fresh id — including the protected copy constructor (`core/model/parts/Part.cpp:35`) — so a clone is a new part, never an alias of its original. Ids are deliberately not serialized: they are

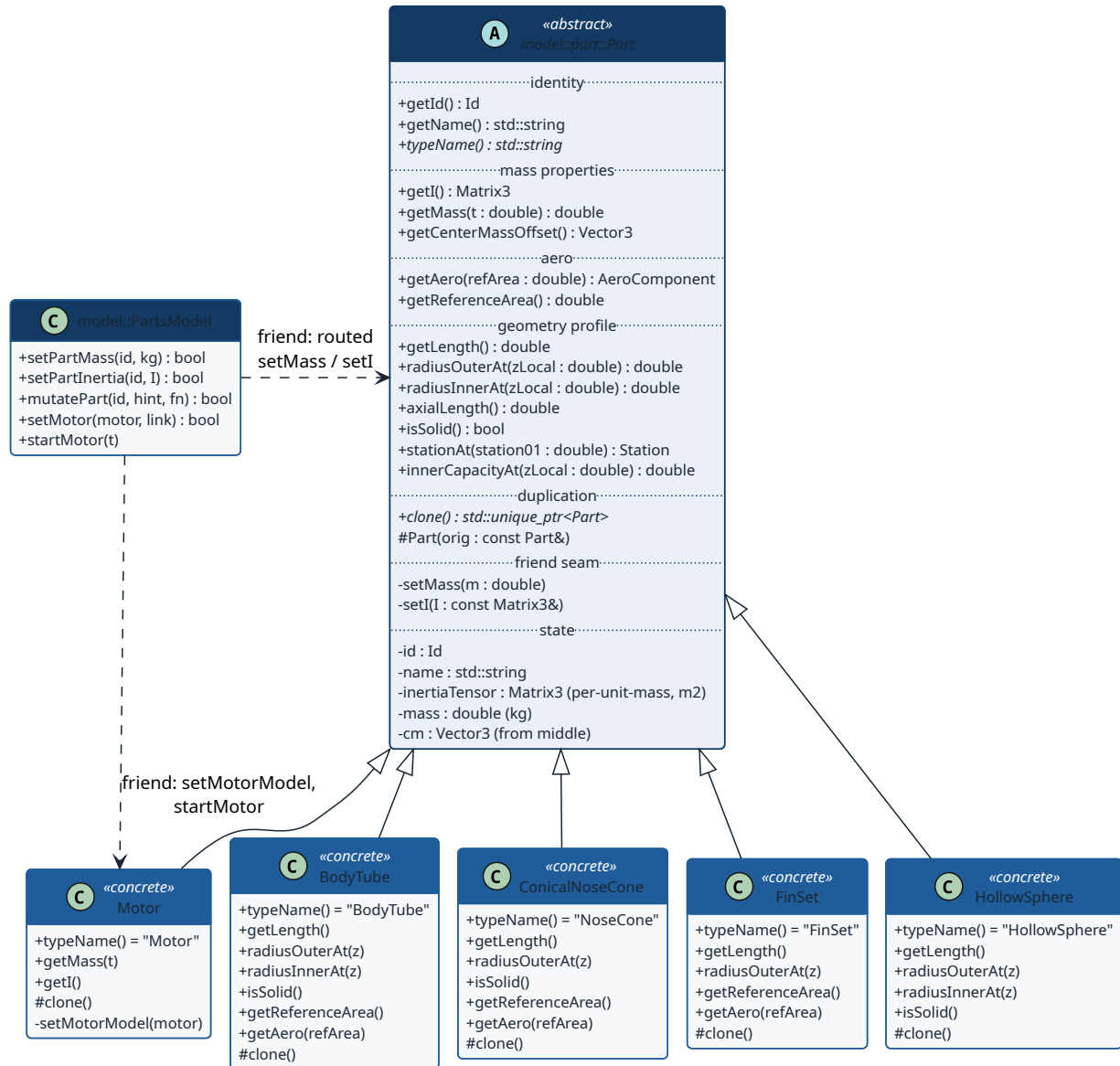


Figure 2. The **Part** leaf contract (`core/model/parts/Part.h`) and its five concrete types. Concrete boxes list only the members each type overrides; **PartsModel** appears only in its leaf-facing role, as the friend behind the routed **setMass/setI** seam (and, for **Motor**, the motor verbs).

meaningful only within a single run, and a design file round-trip (Section 8) yields a structurally identical tree with new ids.

The `name` is a human-facing label with no uniqueness requirement; `PartsModel::findByName` is a pre-order convenience, but identity is always the id. The type tag `typeName()` is the one pure identity virtual: a stable string ("NoseCone", "BodyTube", "FinSet", "HollowSphere", "Motor") that doubles as the part-factory key (`core/model/parts/PartFactory.h`), the design-file `<part type=...>` attribute, and the CLI `listparts` label. One vocabulary serves the factory, the file format, and the front-ends, so the three cannot drift apart.

3.2 Mass properties: the per-unit-mass split

The leaf stores its own mass in kilograms and its inertia tensor *per unit mass* — a purely geometric tensor in m^2 , about the part's own CM (`core/model/parts/Part.h:60`). The full mass-weighted tensors in kg m^2 exist only as composites, assembled by the node walk of Section 4, which weights each part's geometric tensor by its live mass and shifts it to the composite CM with the parallel-axis theorem.

```

/// Per-unit-mass (geometric) inertia tensor about this part's CM (m^2).
virtual Matrix3 getI() const { return inertiaTensor; }

/// CM relative to the component middle. Zero for symmetric parts, non-zero for a
/// cone (CM at L/4 or L/3 from the base). Consumed by the composite walk as
/// ~-L/2 + getCenterMassOffset().z().
Vector3 getCenterMassOffset() const { return cm; }

/// This part's own mass at simulation time @p t (kg). Lets overrides model
/// time-varying mass (e.g. a motor).
virtual double getMass(double t [[maybe_unused]]) const
{
    return mass;
}

```

Listing 3. The mass-property surface of the leaf (`core/model/parts/Part.h:59–70`): a geometric tensor, a middle-relative CM offset, and a time-parameterized mass.

The split (Listing 3) exists because mass is the only time-varying physical input in the model. The hook is `getMass(t)`: the base implementation ignores `t` and returns the stored mass, and `Motor` overrides it to read its wrapped `MotorModel` live — loaded weight before ignition, falling to the casing mass over the burn (Section 7). Since the composite walk re-weights resolved geometry by `getMass(t)` on every rebuild, this one virtual is what makes every composite quantity — mass, CM, inertia — a function of time.

Rejected — mass-weighted leaf tensors

Storing kg m^2 at the leaf would bind a snapshot of the mass into the tensor. Every mass edit would then require a paired tensor rescale — two writes maintaining one invariant — and a burning motor would need its stored tensor rewritten every timestep. Factoring the mass out entirely makes the two orthogonal: mass moves, geometry does not, and the tensor written at construction stays valid for the part's whole life. The price is one scalar multiply per part per composite rebuild, paid inside a walk that is already touching every part.

Units at the leaf boundary

`Part::getI()` is per-unit-mass, m^2 , about the part's own CM. Composite tensors (`PartNode::compositeI`) are mass-weighted, kg m^2 , about the composite CM. The weighting is applied exactly once — `c.mass * c.part->getI()` inside the node walk — so a leaf tensor is never double-weighted and a composite is never un-weighted.

The CM offset is deliberately expressed relative to the *component middle*, not the fore plane. The composite walk locates each part's CM at $-L/2 + \text{getCenterMassOffset().z}()$ below its resolved fore-plane origin (`core/model/PartsModel.cpp:199`); middle-relative storage makes the zero vector the correct value for every symmetric part (tube, sphere), so only genuinely asymmetric shapes record an offset, computed once in their constructors. A fore-plane-relative convention would instead force every symmetric part to store $-L/2$ and keep it synchronized with its length — exactly the derived-from-geometry data this design refuses to store. The offset is non-virtual and set once at construction; its radial components are carried but unconsumed until 6-DOF force application. Placement itself never reads the CM: attachment is geometric (`StationLink`, [Section 6](#)), and the CM participates only in the mass walk.

3.3 The aerodynamic contribution

`getAero(refArea)` (`core/model/parts/Part.h:75`) returns the part's Barrowman contribution as an `AeroComponent` — C_{N_α} , the weighted moment `cnAlphaXcp`, and a drag term — normalized to the caller-supplied shared reference area so that contributions from different parts are directly additive. The default is aerodynamically inert (`return {};`, $C_{N_\alpha} = 0$), and the value is a pure function of the part's geometry: computed on demand, never stored.

The own-CM aero datum

Each part reports its center of pressure as x_{cp} measured from *its own CM* along $+z$. A leaf cannot know its station in the rocket — that is placement, which lives in the tree — but its own CM is the one landmark both sides can locate: the part derives it from its geometry, and the composite walk knows exactly where that CM lands (`core/model/PartsModel.cpp:237`). The walk adds $C_{N_\alpha} \cdot z_{cm}$ to each contribution, re-expressing every part on the sub-tree-root datum shared with `compositeCm` — which is what makes cp – cg the static margin with no frame conversion.

The component stores the product `cnAlphaXcp` rather than a raw x_{cp} , so composing a CP is plain field addition and a zero-lift body drops out of the weighted average with no special case — a `BodyTube` contributes $C_{N_\alpha} = 0$ and simply vanishes from the sum (`core/model/Aero.h`).

`getReferenceArea()` is the second, decoupled half of the aero story: each part reports its own frontal disc (πr^2 , or 0 for a part with no frontal disc), and the tree's `maxFrontalReferenceArea` takes the maximum over the sub-tree — the shared Barrowman reference area that is then fed back into every `getAero` call. A part thus *reports* an area once and *receives* the winning area as a parameter; no leaf ever assumes it defines the rocket's reference.

3.4 The geometry profile

Every part describes itself in the same local frame: it occupies $z \in [-L, 0]$ with its fore plane at the origin and $+z$ = forward (`core/model/parts/Part.h:79`). `getLength()` defaults to 0, which is the contract for a geometrically inert node — a `Motor` contributes mass but consumes no stack length — and every concrete geometry type overrides it. The profile proper is two closed-form virtuals,

`radiusOuterAt(zLocal)` and `radiusInnerAt(zLocal)`: a tube returns constants, a cone a linear taper, a sphere a circular arc. They are callable independently of any station bookkeeping because the envelope sweep of [Section 6](#) samples them across whole spans, not just at seams. `axialLength()` defaults to `getLength()` and exists as the override point for a part whose envelope span differs from its nominal length; no current part needs it, but the seam keeps “envelope” from silently aliasing “length” in the sweep.

```
Station Part::stationAt(double station01) const
{
    // Clamp the fraction (degenerate guard) and map it into the +z = forward local
    // frame: station 1 is the fore plane (z = 0, the origin), station 0 the aft
    // plane (z = -length).
    const double s = std::clamp(station01, 0.0, 1.0);
    const double z = (s - 1.0) * getLength();
    return Station{z, radiusOuterAt(z), radiusInnerAt(z)};
}

double Part::innerCapacityAt(double zLocal) const
{
    // Solid-host rule: a solid part is bounded by its outer skin (the poke-through
    // test); a bored part by its inner wall. Branching here keeps the overlap sweep
    // free of solidity special cases.
    return isSolid() ? radiusOuterAt(zLocal) : radiusInnerAt(zLocal);
}
```

Listing 4. Fraction-to-landmark mapping and the solid-host rule (`core/model/parts/Part.cpp:44–58`).

`stationAt(station01)` ([Listing 4](#)) turns a dimensionless fraction — 0 the aft plane, 1 the fore plane — into a resolved `Station` landmark: the axial coordinate plus the outer and inner radii read from the profile there. The fraction is clamped to $[0, 1]$, the base mapping is $z = (s - 1)L$, and because it is built entirely from `getLength()` and the radius virtuals, no symmetric part needs an override; a `Station` is derived on every read and never stored, so editing a length can never strand a stale landmark.

`isSolid()` is inferred, not stored: the base asks whether the part has a bore at its fore plane (`radiusInnerAt(0.0) <= 0.0`). Two classes of part must override the inference. A `HollowSphere`’s cavity vanishes at the poles, so the fore-plane probe would misreport a shell as solid; it answers from its inner radius directly. A shell `ConicalNoseCone` does not model an inner profile at all, so its solidity is *authored* — the constructor’s `solid` flag is the answer. Getting this right matters because `isSolid()` selects the edge in `innerCapacityAt`: the radius a host offers an inserted child is its bore if it has one, and its outer skin (the poke-through bound) if it does not. Centralizing that branch in the base class keeps the envelope sweep entirely free of per-type solidity cases — an offender of outer radius r fits at a station iff $r \leq \text{innerCapacityAt} + \text{tolerance}$, whatever the host is.

3.5 Duplication: `clone()` through a protected copy

A part is duplicated one way: `clone()`, a pure virtual returning `std::unique_ptr<Part>` (`core/model/parts/Part.h:125`). Each concrete type implements it as `new BodyTube(*this)` against its defaulted copy constructor, which chains to `Part`’s *protected* copy constructor — same mass properties, fresh id. The access choreography is the point: because the base copy constructor is protected and both assignment operators are deleted, external code cannot copy-construct a part at all, so slicing a

`BodyTube` down to a base `Part` is unrepresentable rather than a lurking bug. The concrete types keep their own copy constructors and `clone()` overrides protected too, so even a caller holding the concrete type goes through the virtual. `PartNode::clone` builds on this to deep-copy whole sub-trees — every leaf cloned with fresh ids, links copied verbatim (Section 4).

3.6 The routed-mutation seam

After a part is attached to the tree, exactly two things about it may change: its mass and its tensor. Both writes are private, and their sole friend is `PartsModel` (Listing 5). The public paths are the routed verbs `PartsModel::setPartMass` and `setPartInertia` (`core/model/PartsModel.cpp:399`), which perform the plain write and then dirty the mass-cache chain up the tree — write and invalidation fused in the only code that can do either.

```
protected:
    /// Copy for concrete clone() implementations only: same mass properties,
    /// fresh id. Protected so external code can't copy or slice.
    Part(const Part&);

private:
    friend class model::PartsModel; // the sole post-attach mutation seam

    /// Plain writes: PartsModel's routed verbs own cache invalidation.
    void setMass(double m) { mass = m; }
    void setI(const Matrix3& I) { inertiaTensor = I; }
```

Listing 5. The copy and mutation seams of the leaf (`core/model/parts/Part.h:127–137`).

Rejected — protected setters

Protected access would admit every subclass to the seam, and a subclass can re-publish what it inherits: one public `void setDensity(double)` wrapper that calls a protected `setMass` internally, and the invalidation contract is silently gone — the caches of Section 5 would keep serving mass properties for a part that no longer has them. Private + `friend` names the one class that may write, and that class is the one that owns cache invalidation, so the seam cannot be widened by inheritance. `Motor` repeats the pattern one level down: its `setMotorModel` is itself private with `PartsModel` as friend, so even a motor swap is a routed verb (Section 7).

3.7 The concrete leaves

Five concrete types cover the current vocabulary; Table 1 maps each against the virtuals it overrides. The umbrella header `core/model/parts/Parts.h` registers them for use sites, and `makePart/params` (`core/model/parts/PartFactory.h`) construct and reflect the geometric four from `typeName()` plus named parameters (Sections 8 and 9).

BodyTube (`core/model/parts/BodyTube.h`) is a uniform-density hollow cylinder: $V = \pi(r_o^2 - r_i^2)L$, tensor from `InertiaTensors::Tube`, CM at the geometric center (zero offset). Its profile is two constants — outer skin and bore — and $r_i = 0$ degrades it to a solid rod, which its `isSolid()` reports directly. Aerodynamically it is the canonical zero-lift body: a constant-diameter cylinder produces no

Virtual	BodyTube	NoseCone	FinSet	Sphere	Motor
typeName() (pure)	•	•	•	•	•
clone() (pure)	•	•	•	•	•
getLength()	•	•	•	•	—
radiusOuterAt()	•	•	•	•	—
radiusInnerAt()	•	—	—	•	—
isSolid()	•	•	—	•	—
getReferenceArea()	•	•	•	—	—
getAero()	•	•	•	—	—
getMass(t)	—	—	—	—	•
getI()	—	—	—	—	•
axialLength()	—	—	—	—	—
stationAt()	—	—	—	—	—

Table 1. Virtual overrides by concrete part. No type overrides `axialLength()` or `stationAt()`: every current envelope equals its nominal length, and the base fraction-to-landmark mapping serves all profiles. `Motor` overrides no geometry at all — it is the mass-only, zero-length leaf.

normal force, so `getAero` returns the inert component and the tube drops out of the composite CP average; its wetted area is exposed for an eventual skin-friction drag term.

ConicalNoseCone (`core/model/parts/ConicalNoseCone.h`) is a straight cone, uniform solid by default or a thin open-base lateral shell. It is the working example of the CM-offset machinery: the centroid sits $L/4$ (solid) or $L/3$ (shell) forward of the base, so the constructor bakes $\bar{h} - L/2$ — that is, $-L/4$ or $-L/6$ from the middle — into the base class via `coneCmOffset`, on top of any user-supplied offset. Its `getAero` carries the matching datum conversion (Listing 6): the Barrowman cone CP sits $\frac{2}{3}L$ aft of the tip regardless of solidity, which lands at $L/3 - \bar{h}$ from the CM — $+L/12$ for a solid cone, exactly 0 for a shell, whose CM and CP coincide. The outer profile is a linear taper with a guard so a degenerate zero-length cone reads as a flat disc rather than evaluating $0/0$; solidity is authored by the constructor flag, as Section 3.4 requires.

FinSet (`core/model/parts/FinSet.h`) models $N \geq 3$ identical trapezoidal flat-plate fins as *one* analytic leaf rather than N child parts. The tree shares a single body-frame orientation and never rotates child tensors, so an azimuthally-arrayed component cannot be represented as children at different clock angles; instead the N -fin rotate-and-sum is baked into the set’s own per-unit-mass tensor (`InertiaTensors::TrapezoidalFinSet`), which is transversely isotropic for $N \geq 3$. $N < 3$ is truly anisotropic and unsupported — the constructor warns and applies the $N \geq 3$ approximation rather than throwing. Its axial extent is the root chord (`getLength() == rootChord`), and its silhouette is deliberately the mounting disc: `radiusOuterAt` returns `bodyRadius`, with the fin-tip radius $r_b + s$ exposed separately as extent (`getMaxRadius`). The same reasoning fixes its reference area: `getReferenceArea()` reports the body disc πr_b^2 , *not* a span-inflated disc, because that value competes in the tree’s max-frontal-disc search — fins are lifting surfaces, not frontal area, and letting a fin tip win the search would corrupt the normalization of every coefficient in the rocket. `getAero` is the full Barrowman fin-set term — interference factor $1 + r_b/(s + r_b)$, referenced to the body cross-section and rescaled to the shared area — with the CP converted from the root leading edge to the set’s own mass centroid, and a degenerate guard that reports an inert component for $r_b = 0$ (fins on the axis make the $(s/d)^2$ term singular, and with no body there is no interference to model).

```

// CM relative to the component middle, in the +z = forward frame (tip at +L/2,
// base at -L/2). The centroid is hbar forward of the base, i.e. hbar - L/2 from
// the middle: -L/4 (solid), -L/6 (shell).
Vector3 ConicalNoseCone::coneCmOffset(double L, bool solid)
{
    const double hbar = solid ? (L / 4.0) : (L / 3.0);
    return Vector3{0.0, 0.0, hbar - L / 2.0};
}

model::AeroComponent ConicalNoseCone::getAero(double refArea) const
{
    // Barrowman cone: CNalpha = 2 at the base area (pi R^2), rescaled to refArea.
    // The cone CP is 2/3 L aft of the tip (shape-independent). Report x_cp from the
    // cone's own CM (the composite datum) in the +z = forward frame:
    // x_cp(from CM) = -L/6 - (hbar - L/2) = L/3 - hbar (= +L/12 solid, 0 shell).
    const double cnAlpha = 2.0 * (std::numbers::pi * baseRadius * baseRadius) / refArea;
    const double hbar = solid ? (length / 4.0) : (length / 3.0);
    const double xcpFromCm = length / 3.0 - hbar;
    return {cnAlpha, cnAlpha * xcpFromCm, 0.0};
}

```

Listing 6. The cone’s centroid offset and its own-CM aero datum conversion (core/model/parts/ConicalNoseCone.cpp:49–68, lightly elided).

HollowSphere (core/model/parts/HollowSphere.h) is a thick-walled shell: $V = \frac{4}{3}\pi(r_o^3 - r_i^3)$, per-unit-mass tensor $\frac{2}{5}(r_o^5 - r_i^5)/(r_o^3 - r_i^3) \mathbf{1}$, axial extent the full diameter (pole to pole), centered at $z = -r_o$. It is the one part with genuinely curved closed forms — outer silhouette $\sqrt{r_o^2 - (z + r_o)^2}$ and an inner cavity profile that vanishes outside the band $|z + r_o| \leq r_i$ — and it is the motivating case for the `isSolid()` override: its bore is zero at the fore plane, so only the direct $r_i > 0$ answer classifies a shell correctly.

Motor (core/model/parts/Motor.h) is surveyed here and specified in [Section 7](#). It wraps a `MotorModel` by value and overrides exactly the two mass virtuals: `getMass(t)` reads the wrapped motor live, and `getI()` derives a solid-cylinder per-unit-mass tensor from the catalog diameter and length on every call (degrading to a zero-tensor point mass when the catalog omits geometry). It overrides no geometry virtual at all — length 0, no silhouette, no aero — so it participates in the composite as pure time-varying mass at its placed station, and its runtime mutations (in-place swap, ignition) go through `PartsModel`’s motor verbs via its own private-plus-friend seam.

Taken together, the leaves hold every fact that is local to one component and nothing that is not. What a part weighs *now*, where those masses sit relative to each other, and whether the assembly self-intersects are questions about the tree — and the tree is where this document goes next ([Section 4](#)).

4 The node tree: PartNode and PartsModel

Where the leaf of [Section 3](#) is the physical vocabulary, the tree is the grammar. Two classes in `core/model/PartsModel.h` carry all of it. `PartNode` is one interior node of the rocket: it owns exactly one `Part`, the `StationLink` that seats it on its parent ([Section 6](#)), and the child nodes beneath it — so any node is a complete sub-assembly, and the composite queries of [Section 5](#) run identically on a full rocket or a booster stack. `PartsModel` is the single owner of that tree and its only mutation authority:

reads hand out `const Part&`, and every post-attach change — structural or leaf-level — is a named verb on the model that performs the write, maintains the id index and the motor borrow, dirties the correct cache chain, and brackets itself in change events. Figure 3 shows the full ownership picture, with `RocketModel` above as the value owner and install facade.

This section covers the tree’s structure and mutation surface: the node’s owned interior and read surface, how detached trees are built and cloned, lookup and traversal, the structural verbs and their typed errors, the routed leaf edits with their invalidation hint, and the event stream that lets a Qt front-end mirror the tree without the model knowing Qt exists. The caches that hang off each node are specified in Section 5; the motor slot’s semantics in Section 7.

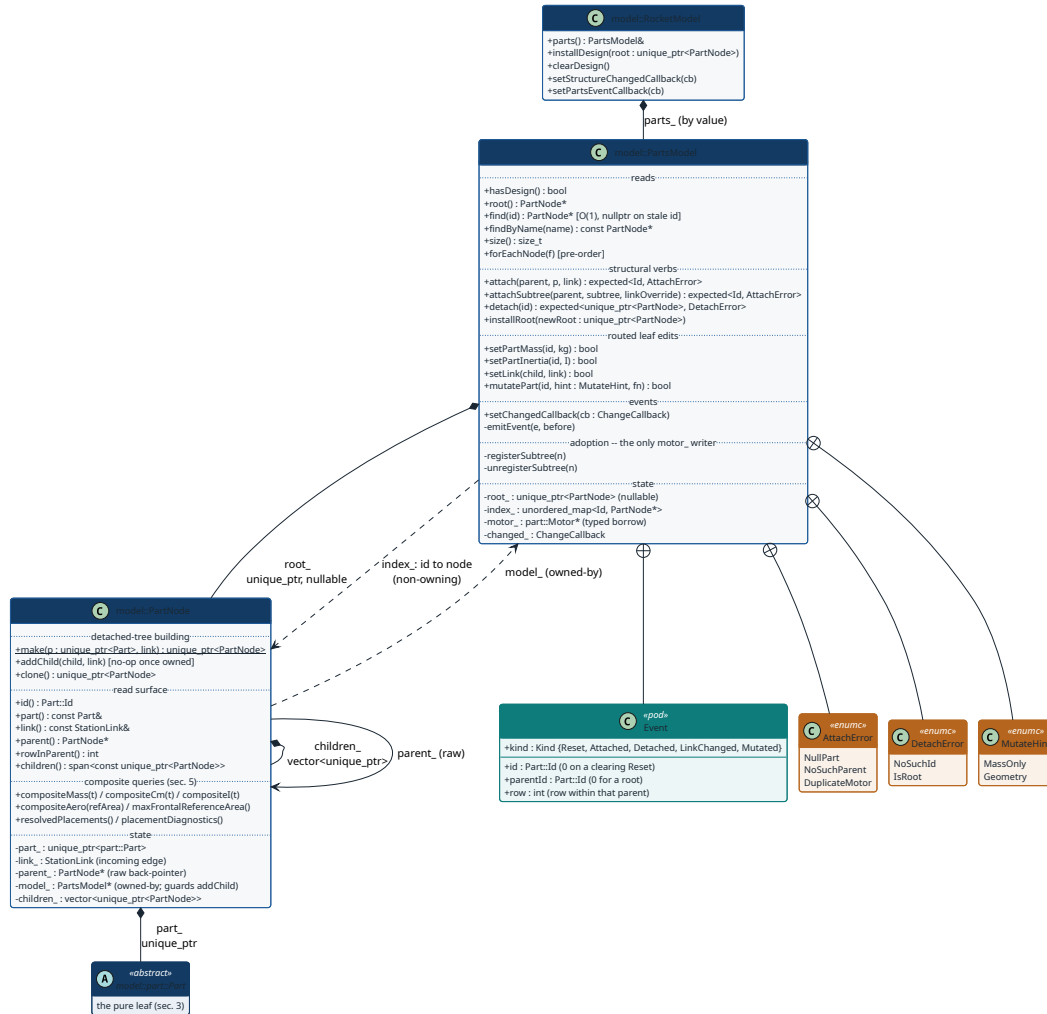


Figure 3. The owned part tree (`core/model/PartsModel.h`). `RocketModel` holds `PartsModel` by value; `PartsModel` owns the root node through `unique_ptr` and indexes every node by part id; each `PartNode` owns its leaf and its children the same way. `Event` and the error enums are nested types of `PartsModel`.

4.1 The node: an owned interior, a const read surface

A `PartNode` holds five members (`core/model/PartsModel.h:108–112`). Three express ownership and shape: `part_`, a `std::unique_ptr<part::Part>` — the node is the owner of its leaf; `children_`,

a `std::vector<std::unique_ptr<PartNode>>` in attachment order; and `link_`, the `StationLink` stored on the *child* as its incoming edge — the placement intent relative to whatever parent the node currently has, a default link on a root. Storing the edge on the child rather than in a parent-side table means a detached sub-tree carries its internal seating with it: every edge except the one that was cut travels inside the value.

The remaining two are raw back-pointers, and both are deliberate non-owners. `parent_` points up the tree; it is safe as a bare pointer because ownership already guarantees the parent outlives the child — destroying a parent destroys its `children_` first. `model_` points at the owning `PartsModel` while the node is attached to one and is null while detached; it exists purely as a guard bit for `addChild` (Section 4.2). Because nodes point back at their model, `PartsModel` is non-copyable and non-movable (`core/model/PartsModel.h:156`) — a moved-from model would leave every node’s back-pointer dangling, so the type simply forbids it.

The read surface is small and entirely `const`: `id()` forwards to the part’s own id, so there is exactly one identity namespace — a node is named by the part it owns, and no separate node-id can ever disagree with it; `part()` returns `const Part&`, so no reader can mutate a leaf behind the model’s back; `link()`, `parent()`, and `children()` (a `std::span` view, exposing the sequence but not the vector) complete the topology; and `rowInParent()` reports the node’s index among its siblings, 0 for a root (`core/model/PartsModel.cpp:55`). That last accessor earns its place by shape: (parent, row) is precisely the coordinate system of `QAbstractItemModel`, so the Qt adapter of Section 11 reads its geometry straight off the node instead of maintaining a parallel map.

4.2 Building detached trees: `make`, `addChild`, `clone`

Nodes begin life detached. The only way to create one is the static factory `PartNode::make(p, link)` (`core/model/PartsModel.h:47`), which wraps a leaf and its intended incoming edge into a fresh node — null part in, null node out. A detached tree is then assembled bottom-up with `addChild`, which wires the child’s `parent_` pointer, stores the edge, and moves the child in (Listing 7). This is the loader’s idiom: `DesignSerializer` builds the whole tree from the file — `make` per part, `addChild` per edge (`core/model/DesignSerializer.cpp:163`) — and only then hands the finished root to the model in one step.

The guard in the middle of Listing 7 is the boundary of the whole design: `addChild` is a logged no-op the moment either node is owned by a model.

Owned trees mutate only through verbs

Once a node belongs to a `PartsModel`, a structural edit implies bookkeeping the node cannot do alone: the id index must gain or lose entries, the motor borrow may need re-resolving, cache chains must be dirtied *from the right node upward*, and the Qt-facing event pair must fire with correct pre-state. `addChild` on an owned node would silently skip all four, so it refuses — the `model_` back-pointer exists to make “am I owned?” answerable at the one place it matters. Detached trees have none of those obligations, which is exactly why free assembly is legal there.

`clone()` (`core/model/PartsModel.cpp:92`) deep-copies a sub-tree by recursion on this same machinery: every leaf duplicated through the virtual `Part::clone` — so each copy carries a fresh id, per the leaf’s identity contract (Section 3) — and every `StationLink` copied verbatim, so the copy is seated exactly as the original was. The result is a detached root, ready for `attachSubtree`: clone-then-attach is copy-paste.


```

void PartNode::addChild(std::unique_ptr<PartNode> child, part::StationLink link)
{
    if(!child)
    {
        utils::Logger::getInstance()->error("PartNode::addChild: ignoring null child");
        return;
    }
    if(model_ != nullptr || child->model_ != nullptr)
    {
        utils::Logger::getInstance()->error(
            "PartNode::addChild: owned trees mutate only through PartsModel verbs");
        return;
    }
    child->parent_ = this;
    child->link_ = link;
    children_.push_back(std::move(child));
    markMassDirty();
    markPlacementDirty();
}

```

Listing 7. Detached-tree assembly and its ownership guard (`core/model/PartsModel.cpp:72–90`).

4.3 The model: identity, lookup, traversal

`PartsModel` holds a *nullable* root (`std::unique_ptr<PartNode> root_`) and reports it through `hasDesign()` (`core/model/PartsModel.h:161`). The null state is a real state, not a placeholder: the GUI opens with no design loaded, the CLI runs before any `loaddesign`, and both must render, save-refuse, and report sensibly. Representing “no design” as an empty-but-present dummy root would force every consumer to distinguish a real single-part rocket from the sentinel; a null root makes the distinction structural, and consumers gate on `hasDesign()` before asking composite questions (the CLI’s design commands and the geometry-derived reference area both check it explicitly, [Sections 9](#) and [10](#)).

Lookup is by id, in constant time: `index_` maps `Part::Id` $\rightarrow$ `PartNode*` for every owned node, and `find(id)` returns the node or `nullptr` (`core/model/PartsModel.h:167`). The index is maintained exclusively by the adoption pair `registerSubtree/unregisterSubtree` ([Section 4.4](#)), so it can never disagree with the ownership tree.

Stale ids fail safe

Front-ends hold ids across time: a GUI selection survives a delete, a CLI script names parts that a previous command may have removed, an undo stack refers to nodes that no longer exist. Every verb therefore resolves its id through `find` and returns a typed error or `false` on a miss — and because ids are drawn from a process-wide counter and never reused within a run ([Section 3](#)), a stale id can never alias a *different* part later. The worst outcome of a dangling identity is a refused verb with a diagnosable error, never a write to the wrong node and never undefined behavior. This is the payoff of making the id, not a pointer or a name, the currency of the mutation API.

`findByName` (`core/model/PartsModel.cpp:257`) is the deliberate second-class citizen: a pre-order first-match walk, provided because humans type names at the CLI, but names carry no uniqueness requirement — two “`BodyTube`”s are legal — so the method is a convenience for interactive lookup

and nothing in the model keys on it. `size()` is the index's size, $O(1)$ and zero with no design. Traversal is `forEachNode` (`core/model/PartsModel.h:185`), a template over `f(const PartNode&, int depth)` that visits parent-before-child in attachment order — the same deterministic order the placement resolver emits (Section 6), so a listing printed from the walk and a diagnostic reported from the resolver name parts in the same sequence.

4.4 Structural verbs and typed errors

Four verbs change the shape of an owned tree, and each returns its failure modes in the type: `attach` and `attachSubtree` return `std::expected<Part::Id, AttachError>`, `detach` returns `std::expected<std::unique_ptr<PartNode>, DetachError>`, and `installRoot` is total. The enums are small and exact — `AttachError` is `{NullPart, NoSuchParent, DuplicateMotor}`, `DetachError` is `{NoSuchId, IsRoot}` — so a front-end can map each case to a specific message instead of parsing a boolean (Section 9). Figure 4 traces the attach path; Listing 8 is its body.

Three details of Listing 8 carry most of the design. First, `attach` is literally `attachSubtree` over a single-node tree (`core/model/PartsModel.cpp:298`) — one adoption path, not two. Second, the single-motor invariant is checked *here*, at the only door a second motor could enter through: when a motor is already installed, the incoming sub-tree is scanned and refused with `DuplicateMotor` if it contains one (Section 7 covers why at most one). Third, `linkOverride` exists for re-seating: a sub-tree built by the serializer arrives with its stored links and should keep them, but a pasted or moved sub-tree is being seated on a *new* parent, and its old incoming edge is intent about a parent it no longer has. The override replaces exactly that one edge — the cut edge — and leaves the sub-tree's interior seating untouched.

`detach(id)` is the inverse (`core/model/PartsModel.cpp:352`), and its return type is the point: the removed sub-tree comes back as a live `std::unique_ptr<PartNode>` — unregistered, parent pointer cleared, internal links intact. A first-class detached value is the building block for every higher-level edit: detach-then-drop is delete (the CLI's `removepart`, `cli/Repl.cpp:1056`), detach-then-attachSubtree is move, hold-and-reattach is one undo frame, and clone-then-attach is paste. None of those features needs model support beyond the verb pair. The root is the one node `detach` refuses (`IsRoot`): it is the datum every resolved pose is expressed against, and removing it is not an edit to the tree but a replacement of the design — which is what `installRoot` is for. `installRoot` atomically swaps the whole tree (null clears), unregistering the old root and registering the new (`core/model/PartsModel.cpp:380`).

All four verbs converge on the private adoption pair. `registerSubtree/unregisterSubtree` (`core/model/PartsModel.cpp:270`) walk the sub-tree once and maintain the three pieces of derived ownership state together: the id index gains or loses every node, each node's `model_` back-pointer is set or cleared, and the typed motor borrow (`motor_`) is re-resolved if the walk crosses a `Motor`. That pair is the *only* code that touches `motor_` — `attach`, `detach`, and a clearing `installRoot` all route through it, so a cleared design can never leave the borrow dangling (Section 7).

4.5 Routed leaf edits and the mutation hint

Post-attach leaf edits reach the tree through four routed verbs, the public face of the private-plus-friend seam on `Part` (Section 3). `setPartMass` and `setPartInertia` (`core/model/PartsModel.cpp:399`) perform the plain write and dirty the mass chain from the edited node upward — and only the mass chain. `setLink` (`core/model/PartsModel.cpp:431`) re-seats an existing edge (station, gap, or seat-kind edits; refused on a root, which has no incoming edge) and dirties *both* chains from the parent: a seat

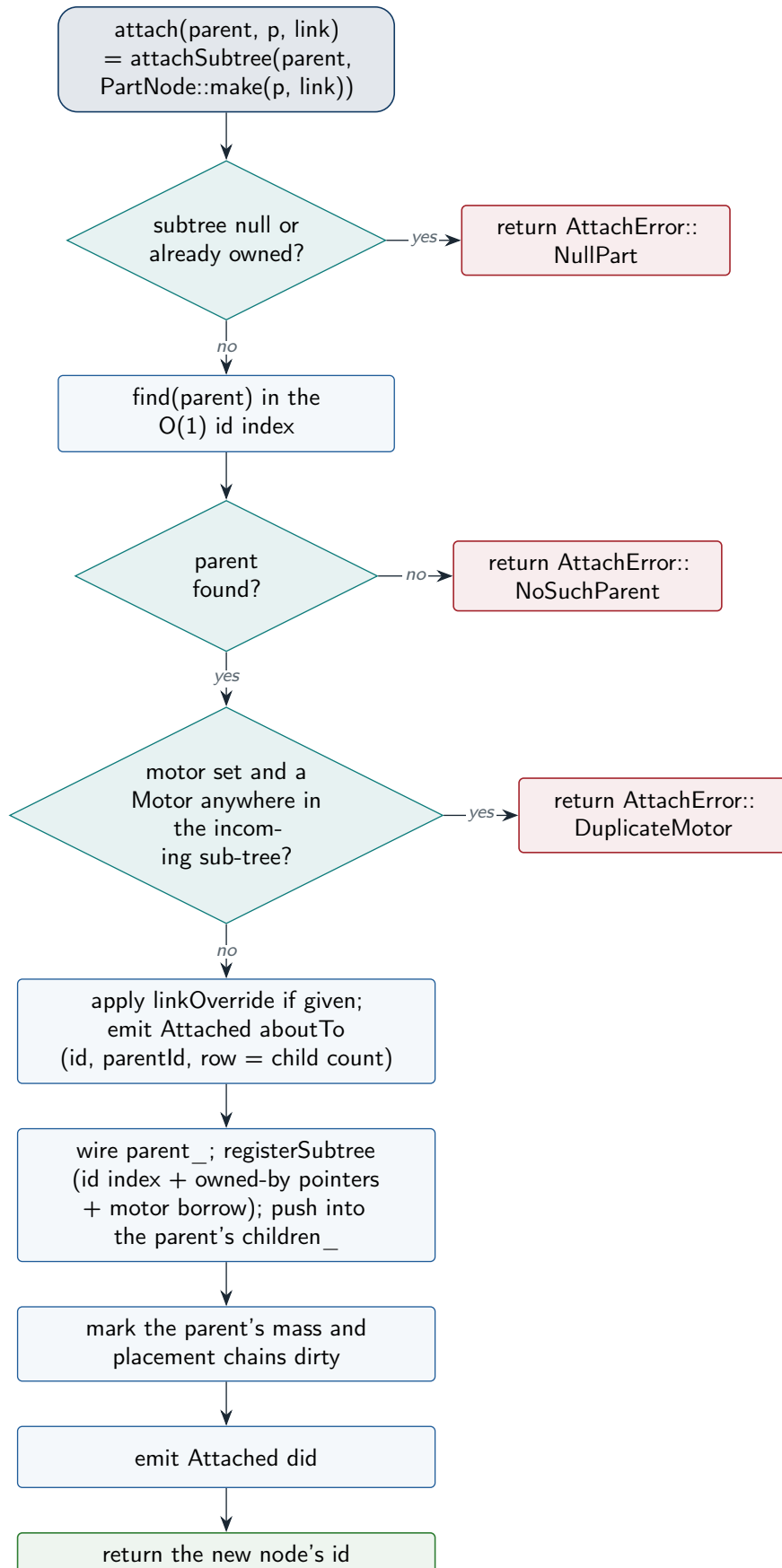


Figure 4. The attach path (`core/model/PartsModel.cpp`). `attach` wraps the bare part and delegates to `attachSubtree`; every failure exits before any state changes, and the event pair brackets exactly the mutating region.

```

std::expected<part::Part::Id, PartsModel::AttachError>
PartsModel::attachSubtree(part::Part::Id parent, std::unique_ptr<PartNode> subtree,
                          std::optional<part::StationLink> linkOverride)
{
    if(!subtree || subtree->model_ != nullptr)
    {
        return std::unexpected(AttachError::NullPart);
    }
    PartNode* parentNode = find(parent);
    if(parentNode == nullptr)
    {
        return std::unexpected(AttachError::NoSuchParent);
    }
    // Single-motor invariant, enforced where it can be broken: reject a second Motor
    // anywhere in the incoming sub-tree.
    if(motor_ != nullptr)
    {
        // ... pre-order scan of the incoming sub-tree ...
        if(hasMotor)
        {
            return std::unexpected(AttachError::DuplicateMotor);
        }
    }

    if(linkOverride)
    {
        subtree->link_ = *linkOverride;
    }
    const part::Part::Id childId = subtree->id();
    const int row = static_cast<int>(parentNode->children_.size());
    const Event e{Event::Attached, childId, parentNode->id(), row};

    emitEvent(e, true);
    subtree->parent_ = parentNode;
    registerSubtree(subtree.get());
    parentNode->children_.push_back(std::move(subtree));
    parentNode->markMassDirty();
    parentNode->markPlacementDirty();
    emitEvent(e, false);
    return childId;
}

```

Listing 8. `attachSubtree`, lightly elided (`core/model/PartsModel.cpp:304–350`): guards first, then the event-bracketed adoption.

edit moves resolved poses, and an equal-mass geometry change is exactly what the mass-delta gate of [Section 5](#) cannot see, so relying on the mass gate alone would serve a stale CM.

`mutatePart` ([Listing 9](#)) is the escape hatch for edits with no typed verb: an arbitrary functor applied to the leaf under a declared `MutateHint`.

```
bool PartsModel::mutatePart(part::Part::Id id, MutateHint hint,
                           const std::function<void(part::Part&)>& fn)
{
    PartNode* n = find(id);
    if(n == nullptr || !fn)
    {
        return false;
    }
    const Event e{Event::Mutated, id, n->parent_ ? n->parent_->id() : part::Part::Id{0},
                  n->rowInParent()};
    emitEvent(e, true);
    fn(*n->part_);
    n->markMassDirty();
    if(hint == MutateHint::Geometry)
    {
        n->markPlacementDirty();
    }
    emitEvent(e, false);
    return true;
}
```

Listing 9. The hinted escape hatch (`core/model/PartsModel.cpp:449–468`): the write and its invalidation are fused, and the hint chooses how much invalidation.

Declare what an edit can move

The two invalidation chains have very different prices. Dirtying the mass chain schedules an $O(N)$ re-weighting at the next composite read; dirtying the placement chain schedules a full geometry re-resolve plus the $O(N \log N)$ envelope sweep (the sort at `core/model/parts/Placement.cpp:149`). A mass-only edit — a motor swap, a payload trim — moves no pose and cannot create an overlap, so `MassOnly` skips the sweep entirely; a length or radius edit can do both, so `Geometry` dirties both chains. The hint is a declaration the caller must make, not an inference the model attempts: the model cannot see inside the functor, and guessing conservatively on every edit would put the sweep on paths that run per-swap in batch motor studies. Note the asymmetry is one-way — `Geometry` still dirties mass, because reshaping a part moves its CM even at constant mass.

4.6 The event stream

Every mutating verb in the previous two subsections brackets itself in a pair of notifications: one `Event` value fired with `before == true` immediately *before* the first write, and the identical value fired with `before == false` after the last ([Listing 10](#)). The event carries the affected node’s id, its (ex-)parent’s id, and its row within that parent — captured *pre-mutation*, so a `Detached` event names the row the node still occupies and an `Attached` event names the row the node is about to land in.

The shape is not incidental: it is the exact contract of Qt’s model/view row operations, which demand `beginInsertRows(parent, row, row)` before the underlying insert and `endInsertRows()` after, with removal mirrored. The GUI adapter is therefore a `switch` that forwards each pair

```

// One change notification, fired as aboutTo/did pairs (before == true, then false)
// around every mutation -- the shape Qt's begin*/end* row-op contract needs.
struct Event
{
    enum Kind : std::uint8_t { Reset, Attached, Detached, LinkChanged, Mutated };
    Kind kind{Reset};
    part::Part::Id id{0};          ///< the affected node (0 on a clearing Reset)
    part::Part::Id parentId{0};    ///< its (ex-)parent; 0 for a root
    int row{0};                   ///< its row within that parent
};
using ChangeCallback = std::function<void(const Event&, bool before)>;

```

Listing 10. The change notification (`core/model/PartsModel.h:143–151`).

verbatim (`gui/RocketTreeView.cpp:32`); [Section 11](#) covers it. The model keeps a single callback slot (`setChangedCallback`; the latest registration wins, `{}` clears), and `RocketModel` is that single subscriber: its constructor installs a bridge (`core/model/RocketModel.cpp:16`) that forwards the typed stream to whichever granular consumer registered via `setPartsEventCallback` and folds each completed mutation (`before == false`) into the coarse “something changed” callback that simpler consumers prefer. One producer, one bridge, two subscription shapes.

Rejected — Qt signals in the model

The natural Qt idiom — make `PartsModel` a `QObject` and emit signals — would drag `QtCore` into `qtrocket_core`, and with it into the CLI and every test binary, breaking the layering rule that only `gui/` may depend on Qt. A `std::function` callback carrying `aboutTo/did` pairs delivers everything the Qt adapter needs while costing the headless consumers nothing: the CLI simply never registers one. Multicast is deliberately absent — `RocketModel` is by construction the one subscriber, and it already fans out — so the model avoids owning listener-list bookkeeping it would never exercise.

4.7 The install facade

Whole-design replacement has one public door: `RocketModel::installDesign` (`core/model/RocketModel.h:87`), with `clearDesign()` as sugar for installing null. It delegates the tree swap to `PartsModel::installRoot` and then performs the one install side effect that belongs to `RocketModel` rather than the tree: it clears the manual reference-area override (`core/model/RocketModel.cpp:118`). The override is drag-model state, not tree state — a provenance flag marking the stored frontal area as user-entered ([Section 10](#)) — and it describes the airframe it was entered for. A freshly installed design must not present it as authored intent for the new airframe; resetting the flag strips that provenance, so a subsequent save records the area as unauthored and a loaded file’s own `<sim>` block decides whether an override is re-established ([Section 8](#)). Both loaders honor the facade: the serializer installs a loaded design through it (`core/model/DesignSerializer.cpp:262`), and the CLI’s design commands build a detached root and hand it over the same way (`cli/Repl.cpp:880`).

The tree, then, is a closed system: one owner, one id currency, verbs that fuse writes with their bookkeeping, and an event stream shaped for the one consumer that needs granularity. What those verbs actually invalidate — and what the flight loop pays when it reads the composites back — is the subject of [Section 5](#).

5 Composite mass properties and the two cache gates

Every physical question the simulator asks of the rocket — what does it weigh, where is its center of mass, what is its inertia tensor — is a *composite* question: a mass-weighted aggregation over the part tree of [Section 4](#), evaluated at a simulation time t because the motor’s mass falls while it burns. These aggregations are answered by `PartNode` (`core/model/PartsModel.h`), and any node answers them for its own sub-tree, so a booster stack and a full rocket use the same code path. The design problem is that the same queries sit on two wildly different clocks. The integrator asks thousands of times per flight, at times chosen by an adaptive step controller that revisits and discards trial times freely. The designer asks after each structural edit, at human cadence. A single “recompute when dirty” flag cannot serve both, because during a flight nothing ever marks the tree dirty — time itself is the thing that changes.

The resolution is two independent caches per node, each invalidated by exactly the events that can change its output. The *placement cache* (resolved poses plus the overlap verdict of [Section 6](#)) is keyed on structure: it rebuilds only when a part is attached, detached, or re-linked, and never during a flight, because geometry is invariant under mass loss. The *composite cache* (CM and inertia tensor) is keyed on the one physical quantity that does change in flight: the composite mass itself. This section walks through the quantities, the two gates, the two-pass mathematics behind the cached values, the aero and reference-area aggregates that share the same resolve, and what the flight loop actually pays.

5.1 One live sum, two cached tensors

`PartNode` exposes three time-aware composite quantities (`core/model/PartsModel.h:70`):

- `compositeMass(t)` — a deliberately *uncached* recursive sum of `Part::getMass(t)` over the sub-tree (`core/model/PartsModel.cpp:102`). It is both the ODE divisor ($a = F/m$) and the key of the mass-delta gate.
- `compositeCm(t)` — the mass-weighted center of mass in the sub-tree root’s fore-plane frame, cached per node behind the gate.
- `compositeI(t)` — the full mass-weighted inertia tensor (kg m^2) about the composite CM, cached behind the same gate.

The asymmetry is the point. The CM and tensor are worth caching: they require the resolved placements, an $O(N)$ weighted average, and N parallel-axis shifts of 3×3 tensors. The mass sum is N double additions with no tensor work at all — cheaper than the Eigen expression it feeds — so caching it would buy nothing. But the deeper reason it must stay live is structural:

The live sum is the tree’s clock

Nothing notifies the part tree that simulation time advanced. No verb fires, no flag flips; the only place t enters the mass properties is `Part::getMass(t)`. The gate therefore needs one authoritative, freshly-evaluated “mass now” to compare against its stamp. If `compositeMass` were itself cached, the gate would be handed its own stamp back: `mNow` would equal `builtAtMass_` by construction, the comparison could never fail, and the CM and tensor would silently stop tracking the burn. The live sum is not an optimization left on the table — it is the input that makes the cache correct.

5.2 The mass-delta gate

Both cached accessors funnel through `ensureCompositeCache` (Listing 11). The gate condition is two-legged: `massDirty_` is the structural leg, raised by every `PartsModel` verb whose edit can move mass (Section 4); the mass comparison is the flight leg, and it is an *exact* floating-point equality.

```
double PartNode::compositeMass(double t) const
{
    // ...
    double m = part_->getMass(t);
    for(const auto& child : children_)
    {
        m += child->compositeMass(t);
    }
    return m;
}

void PartNode::ensureCompositeCache(double t) const
{
    // ...
    const double mNow = compositeMass(t);
    if(massDirty_ || mNow != builtAtMass_)
    {
        const Composite c = computeCompositeAt(t);
        compositeCm_ = c.cm;
        compositeI_ = c.inertia;
        builtAtMass_ = mNow;
        massDirty_ = false;
    }
}
```

Listing 11. The live sum and the mass-delta gate (`core/model/PartsModel.cpp:102`). Stamps are written only after `computeCompositeAt` succeeds, so a failed placement solve is never absorbed by the cache (Section 5.4).

`builtAtMass_` is initialized to `quiet_NaN()` (`core/model/PartsModel.h:116`). `NaN` compares unequal to everything, including itself, so the very first read builds unconditionally — the sentinel makes the gate self-priming without a separate “ever built” flag.

Exact equality looks fragile next to floating point folklore, but here it is precisely correct, because both sides of the comparison are produced by the *same* deterministic computation:

- **Post-burnout the cache freezes.** After the propellant is spent, `MotorModel::getMass` returns the stored casing mass verbatim for every t (`core/model/MotorModel.cpp:65`), and every other part’s mass ignores t entirely. The recursive sum runs over a fixed tree in fixed child order, adding identical doubles in identical sequence, so it reproduces *bit-for-bit* on every read. The gate compares equal, and the coast and descent phases — the majority of most flights — never rebuild.
- **During a burn the cache tracks.** Between recorded steps the interpolated mass curve moves, the sum differs from the stamp, and the gate rebuilds — once per recorded step, as it must, because the CM and tensor genuinely moved.
- **The key is the physics, not the schedule.** In flight the motor is the only time-varying contributor, so the total mass determines the composite state: equal totals at two different times imply identical CM and tensor, and a rebuild would produce the value already stored.

Concurrent mass *edits* that could confound the total are impossible mid-read — they route through verbs that raise `massDirty_`, the gate’s other leg.

Rejected — tolerance comparison, and time as the key

An epsilon comparison ($|m_{\text{now}} - m_{\text{built}}| < \varepsilon$) would freeze the cache during any burn slow enough that per-step mass loss drops below ε — a silently stale CM on exactly the flights where sustainer motors matter. Exact `==` has no false-freeze mode: the only way the sum repeats is that the mass genuinely did not change.

Keying the cache on time (“rebuilt at $t = 3.117$ ”) fails against the adaptive integrator. RK45 evaluates six Fehlberg stages per attempted step at intermediate node times (`core/sim/RK45Solver.h:86`) and discards the whole attempt when the error estimate is out of tolerance; trial times are visited, abandoned, and never exactly revisited. A monotonic time stamp would be poisoned by a rejected attempt that probed *ahead* of the eventually accepted step. The mass key is indifferent to all of this: a rebuild at a rejected trial mass is simply overwritten by the next read, and after burnout no time query of any kind can move the key.

Figure 5 traces the full read path, including the failure branch discussed in Section 5.4.

5.3 The placement gate

The second cache holds the resolved placements — one `Placed` pose per part of the sub-tree, in deterministic depth-first attachment order — plus the overlap-sweep verdict, both produced by the resolver machinery of Section 6. `ensurePlacementCache` (`core/model/PartsModel.cpp:144`) rebuilds both iff `placementDirty_`, which is raised only by structural and link edits: `attach`, `detach`, `installRoot`, `setLink`, and `mutatePart` under the `Geometry` hint (`core/model/PartsModel.h:139`). A mass or inertia edit — `setPartMass`, `setPartInertia`, a motor swap, or the burn itself — never touches it, because none of those can move a seam. The expensive $O(N \log N)$ envelope sweep therefore runs exactly once per edit, at human cadence, and the flight loop reuses its output thousands of times.

Two gates, two cadences

Each cache is invalidated by exactly the events that can change its output, and by nothing else. The placement cache ticks at *editing* cadence: resolve and sweep once per structural or link edit, never during a flight, because geometry is invariant under mass loss. The composite cache ticks at *simulation* cadence: rebuild once per recorded step while mass is changing, then never again. Burning propellant cannot trigger a geometry solve; dragging a fin cannot be missed by the mass gate.

The last clause needs one supporting rule. A link edit can be mass-neutral — sliding a nose cone forward changes no part’s mass, so the mass gate, keyed on the total, is structurally blind to it. `setLink` therefore dirties *both* chains (`core/model/PartsModel.cpp:443`): the placement chain because poses moved, and the mass chain because the composite CM and tensor depend on those poses. The two gates overlap by design wherever one alone would be blind. Dirty marking itself is an $O(\text{depth})$ walk up the parent pointers (`core/model/PartsModel.h:103`), so an edit anywhere invalidates every enclosing sub-assembly, and sibling sub-trees keep their caches untouched.

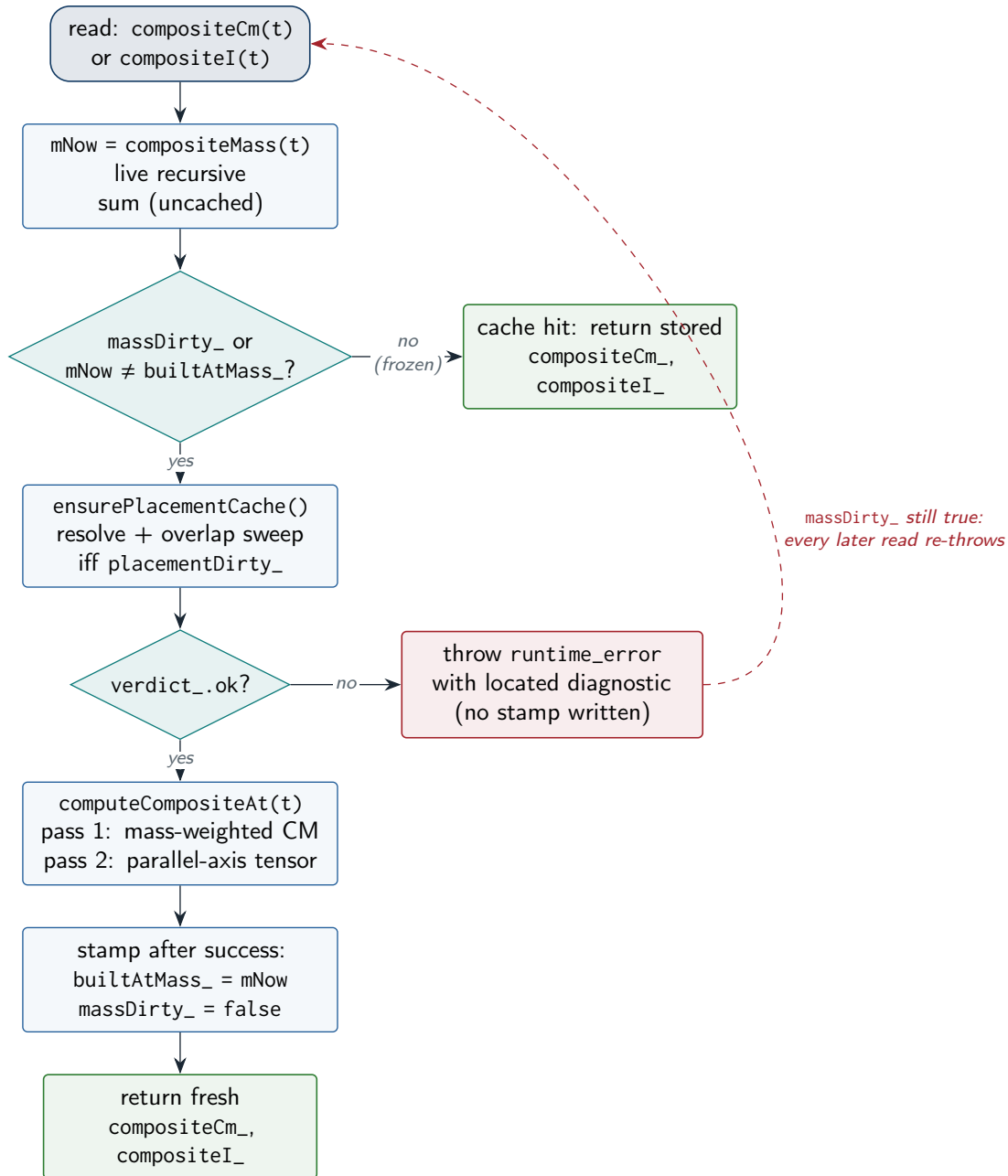


Figure 5. The composite read path. The live mass sum is compared exact-== against the stamp (NaN sentinel on first read); a rebuild runs the placement gate first, refuses a failed overlap verdict by throwing, and stamps the cache only after success.

5.4 computeCompositeAt: two passes over one resolve

A gate-triggered rebuild calls `computeCompositeAt(t)` (`core/model/PartsModel.cpp:171`), which re-weights the *cached* geometry by the *current* masses — the placement gate has already guaranteed the resolved poses are current. The math is two passes over `resolvedPlacements`.

Pass 1 — mass-weighted CM. Each part i stores its CM relative to its component middle (Section 3), so its CM in local coordinates, its position in the sub-tree-root frame, and the composite CM are

$$\mathbf{c}_i^{\text{loc}} = \begin{pmatrix} c_{i,x} \\ c_{i,y} \\ -\frac{L_i}{2} + c_{i,z} \end{pmatrix}, \quad \mathbf{r}_i = \mathbf{o}_i + q_i \mathbf{c}_i^{\text{loc}}, \quad \bar{\mathbf{r}}(t) = \frac{\sum_i m_i(t) \mathbf{r}_i}{\sum_i m_i(t)}, \quad (1)$$

where $\mathbf{o}_i$ and q_i are the part's resolved fore-plane origin and orientation (identity in 3-DOF), and L_i its length — a part occupies $z \in [-L_i, 0]$ in its own frame, $+z$ = forward.

Pass 2 — parallel-axis tensor. Each part's per-unit-mass geometric tensor $\mathbf{I}_i^{\text{geo}}$ (m^2 , about its own CM; Section 3) is weighted to a real tensor and shifted to the composite CM by the parallel-axis theorem:

$$\mathbf{I}(t) = \sum_i m_i(t) \mathbf{I}_i^{\text{geo}} + m_i(t) [(\mathbf{d}_i \cdot \mathbf{d}_i) \mathbf{E} - \mathbf{d}_i \mathbf{d}_i^T], \quad \mathbf{d}_i = \mathbf{r}_i - \bar{\mathbf{r}}(t), \quad (2)$$

with $\mathbf{E}$ the identity. The displacement term is even in $\mathbf{d}_i$, so the sign convention of the offset never matters. Listing 12 is the code, line for line the same shape as the equations.

Before either pass runs, the walk consults the cached overlap verdict, and this is where the design refuses to be quietly wrong:

A self-intersecting design cannot be flown

When the envelope sweep of Section 6 finds interpenetrating parts, `computeCompositeAt` throws `std::runtime_error` carrying the first located diagnostic (`core/model/PartsModel.cpp:184`) rather than integrating overlapping volumes into a plausible-looking but physically meaningless tensor. Because the stamps in Listing 11 land only after a successful build, `massDirty_` stays raised and every subsequent `compositeCm/compositeI` read re-throws — the failure cannot be absorbed by asking twice. The repeat throws are cheap: `placementDirty_` was cleared when the verdict was cached, so no re-sweep runs; the verdict itself is the cached diagnosis. The condition clears only the way it arose — through a structural edit that re-opens the placement gate and produces a clean verdict.

5.5 Aero and area: aggregates on the same resolve

Two further aggregates fold over the same cached resolve. `compositeAero(refArea)` (`core/model/PartsModel.cpp:224`) assembles the whole-rocket Barrowman profile. Each part reports an `AeroComponent` whose CP station $x_{cp,i}$ is measured from *that part's own CM* (`core/model/Aero.h`), stored as the weighted moment $C_{N\alpha,i} x_{cp,i}$ so that a zero-lift body drops out of the average with no special case. The composite walk re-expresses every part onto the shared sub-tree-root datum by adding back the part's CM station:

$$z_{cm,i} = o_{i,z} - \frac{L_i}{2} + c_{i,z}, \quad x_{cp} = \frac{\sum_i C_{N\alpha,i} (x_{cp,i} + z_{cm,i})}{\sum_i C_{N\alpha,i}}. \quad (3)$$

The added $z_{cm,i}$ exactly cancels the per-part CM datum that $x_{cp,i}$ was measured from, so the composite CP depends on external shape alone — and it now lives on the same fore-plane datum as $\bar{\mathbf{r}}$ from

```

Matrix3 parallelAxisTerm(const Vector3& d)
{
    return d.dot(d) * Matrix3::Identity() - d * d.transpose();
}
// ...
// Pass 1: mass-weighted composite CM.
double m = 0.0;
Vector3 weighted = Vector3::Zero();
for(const part::Placed& pl : resolved_)
{
    const double pm = pl.part->getMass(t);
    const Vector3 cmOffset = pl.part->getCenterMassOffset();
    const Vector3 cmLocal(cmOffset.x(), cmOffset.y(),
        -pl.part->getLength() / 2.0 + cmOffset.z());
    const Vector3 cmInRoot = pl.pose.origin + pl.pose.orient * cmLocal;
    m += pm;
    weighted += pm * cmInRoot;
    parts.push_back(Contribution{cmInRoot, pl.part, pm});
}
// Guard the divide: a fully massless sub-tree has no meaningful CM, so leave it at the origin.
Vector3 cm = Vector3::Zero();
if(m > 0.0)
{
    cm = weighted / m;
}

// Pass 2: inertia about the composite CM. Shift each part's own mass-weighted tensor to cm via
// the parallel-axis theorem. 6-DOF adds the R*I*R^T rotation (identity today) here.
Matrix3 I = Matrix3::Zero();
for(const Contribution& c : parts)
{
    const Vector3 d = c.cmInRoot - cm;
    I += c.mass * c.part->getI() + c.mass * parallelAxisTerm(d);
}

```

Listing 12. The two-pass composite walk (`core/model/PartsModel.cpp:171`, elided). The massless-subtree guard keeps a zero-mass assembly — legal while sketching a design — from producing a 0/0 CM; it reads as the origin instead.

Eq. (1). That shared datum is the payoff: the static margin is literally `profile.cp() - cg`, one subtraction with no frame conversion. The profile is re-derived on each call rather than cached; it sits on no flight-loop path, and its only nontrivial input — the resolved placements — is already cached behind the placement gate.

`maxFrontalReferenceArea` (`core/model/PartsModel.cpp:243`) supplies the drag reference area as a *maximum*, not a sum: the largest single frontal disc over the sub-tree, the Barrowman convention. Summing would double-count — a nose cone in front of a body tube of the same caliber presents one disc to the flow, not two — and a fin set deliberately reports only the disc of the body it mounts on (`core/model/parts/FinSet.h:71`), so adding fins never inflates the reference area. `RocketModel` exposes it as `deriveReferenceAreaFromGeometry()` for front ends (`core/model/RocketModel.cpp:44`); the drag term itself always reads the stored `referenceArea` scalar, which `setReferenceArea` marks as a manual override (`core/model/RocketModel.h:69`).

5.6 What the flight loop pays

The gates exist so that the composite machinery costs the integrator almost nothing. Table 2 itemizes the work by event; Fig. 6 shows the resulting cadence over a flight.

Event	Work	Cache traffic
Integrator stage (RK4: 4 per step; RKF45: 6 per attempt, re-run on rejection)	Two live <code>compositeMass</code> sums: the gravity weight inside <code>getForces</code> (<code>core/model/RocketModel.cpp:73</code>) and the F/m divide in the ODE (<code>core/sim/Propagator.cpp:41</code>). No tensor math.	None. The stage path never reads the cached CM or tensor.
Accepted, recorded step	<code>writeMassProperties</code> snapshots mass, CG, and tensor into the state history (<code>core/model/RocketModel.cpp:49</code> , called at <code>core/sim/Propagator.cpp:132</code>): one more live sum plus the two gated reads.	At most one gate rebuild while the motor burns; exactly zero after burnout.
Structural / link edit	Dirty walk to the root, $O(\text{depth})$ flag writes.	Next read: one resolve + envelope sweep, then one composite rebuild.

Table 2. Per-event cost of the composite machinery. The hot path touches only the mass-only recursive sum; the tensor work runs once per recorded step during a burn and not at all afterward.

The RKF45 interaction deserves emphasis because it is where a naive cache design breaks. A rejected attempt evaluates all six stages at trial times and throws the results away; as wired, those stages touch nothing but the live sum, so there is nothing to poison. And even a future consumer that read `compositeI` mid-stage would be safe, because the gate keys on the mass *value*: a rebuild triggered at a discarded trial mass is just a valid cache entry for that mass, corrected by the next read that sees a different one. Correctness is independent of the step controller’s accept/reject sequence — a property a time-keyed cache cannot offer.

5.7 The single-writer contract

All of the above state — both dirty flags, the stamp, the cached CM and tensor, the resolved placements and verdict — is `mutable`, written behind `const` accessors. A `const PartNode&` is therefore

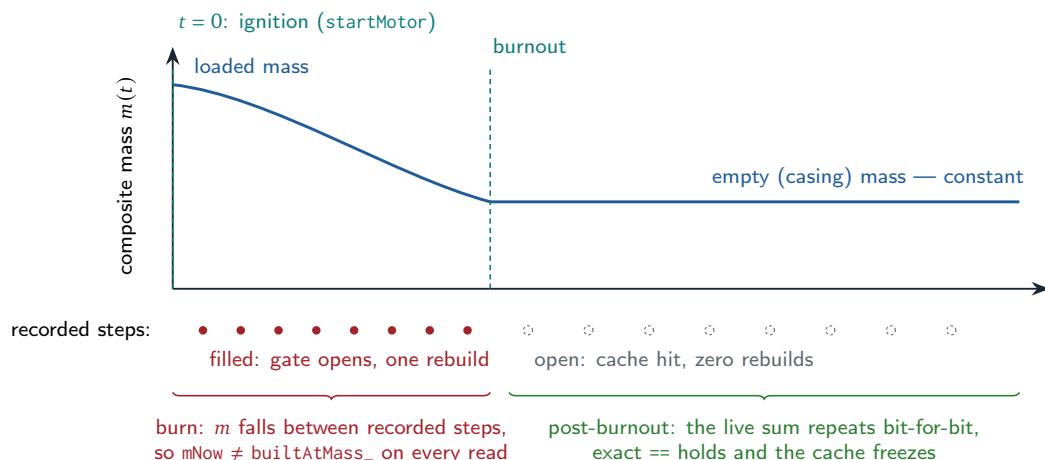


Figure 6. Gate cadence over one flight. During the burn the live sum differs from the stamp at every recorded step, so the gate opens once per step and the CM and tensor track the mass loss. From burnout on, the sum reproduces bit-for-bit, `exact ==` holds, and every read is a frozen-cache hit.

not a promise of physical immutability; it is a promise of logical immutability under the model’s single-writer discipline (Section 2). Concurrent access to a `PartsModel` — even `all-const`, even two readers racing the same `compositeI(t)` call — is a data race, because either read may be the one that rebuilds. The contract is stated at the class boundary (`core/model/PartsModel.h:36`) rather than paid for with per-node locks: QtRocket’s consumers (the GUI thread, the CLI loop, one propagator at a time) are serialized by construction, and a mutex around every composite read would tax the hot path of Table 2 to defend against a caller the architecture already forbids.

6 Placement integration

The node tree of Section 4 stores structure and intent: which part attaches under which, and one `StationLink` per edge describing *how* the child seats against its parent. Nothing in the stored model is an absolute coordinate. Everything spatial — each part’s pose in the rocket frame, the composite center of mass, the overlap verdict, the visualizer’s mesh positions — is derived on demand from that intent by the placement layer, a small vocabulary of value types and free functions in `core/model/parts/Placement.h`. The layer is pure geometry: no time, no mass, no tree access. The tree walk that drives it lives with the tree (Section 6.3), and its output is cached on the node behind the placement gate of Section 5.

This section reads bottom-up: the value types first, then the per-edge placement math, then the node-tree resolver that composes edges into absolute poses, and finally the diagnostics — the per-seam radial check and the whole-tree envelope sweep — whose verdict gates every composite read.

6.1 The placement vocabulary

Figure 7 shows the complete layer. Every type is a plain value; every function is free. The layer deliberately owns nothing and observes parts only through `const` references and non-owning pointers, so it composes with the single-ownership story of Section 4 without touching it.

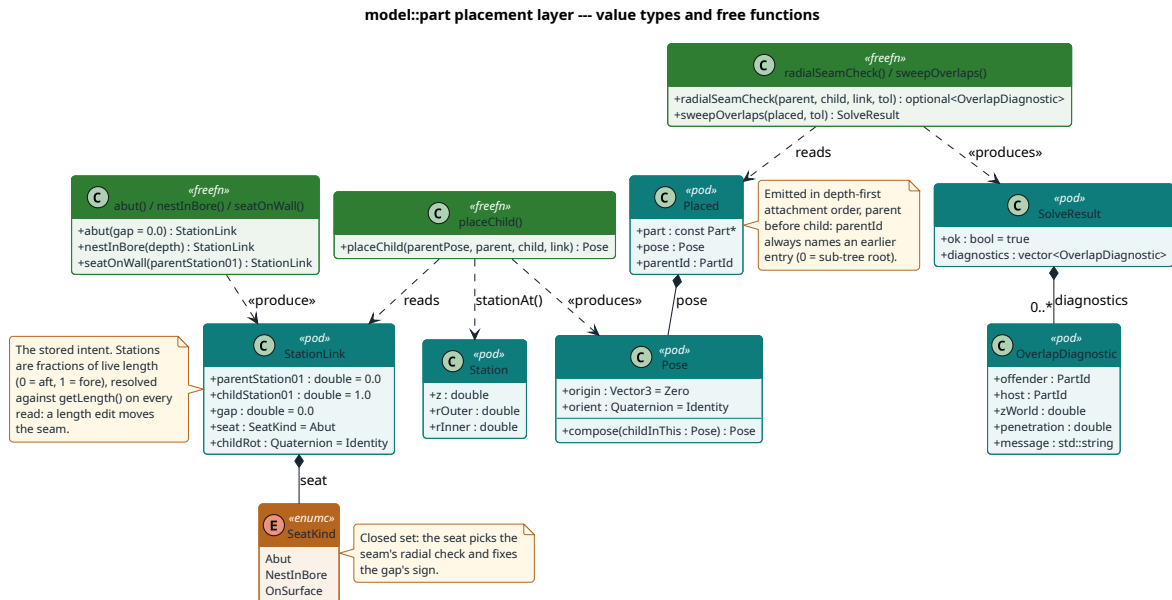


Figure 7. The placement layer of `core/model/parts/Placement.h`: stored intent (`StationLink`), derived geometry (`Station`, `Pose`, `Placed`), diagnostics values (`OverlapDiagnostic`, `SolveResult`), and the free functions that connect them.

StationLink: the stored intent. A `StationLink` (`core/model/parts/Placement.h:54`, [Listing 13](#)) is the only spatial relationship the model persists: a fractional station on the parent, a fractional station on the child, a gap in metres, a seat kind, and a per-seam orientation quaternion. Stations are fractions of *live* length — 0 names the aft plane, 1 the fore plane — and are resolved against `getLength()` on every read by `Part::stationAt` (`core/model/parts/Part.cpp:44`), which clamps the fraction and maps it to $z = (station01 - 1) \cdot L$ in the part's local $+z$ = forward frame.

```

struct StationLink
{
    double    parentStation01{0.0};           // 0 = aft plane, 1 = fore plane, on the PARENT
    double    childStation01{1.0};            // 0 = aft plane, 1 = fore plane, on the CHILD
    double    gap{0.0};                       // meters; meaning per SeatKind
    SeatKind  seat{SeatKind::Abut};           // selects the radial check and the gap sign
    Quaternion childRot{Quaternion::Identity()}; // (x,y,z,w) seam orientation; identity in 3-DOF
};

```

Listing 13. `StationLink` (`core/model/parts/Placement.h:54`). The default-constructed link means “child fore plane abuts parent aft plane” — the natural nose → body → tail build order, so a design assembled with default links stacks aft without any authored numbers.

Length edits move the seam

Because stations are stored as fractions and resolved against `getLength()` at read time, a link never goes stale. Stretch a body tube and every seam authored against its planes — and every part stacked behind it — moves with it on the next resolve, with no fix-up pass and no stored coordinate to forget to update.

Rejected — authored absolute offsets

The alternative is to store each child’s axial offset directly, either from the root or from its parent’s origin. That representation is simpler to resolve but wrong under edit: an authored coordinate is a snapshot of geometry that was true when it was written, and every length change afterwards silently invalidates it. Fractional stations keep the stored quantity in intent space (“my fore plane against your aft plane”), where it remains true under any resize.

Seat kinds. `SeatKind` (`core/model/parts/Placement.h:32`) is a closed three-value enum, because a rigid axisymmetric stack admits exactly three radial relationships between mated parts. The seat does double duty: it selects which radial compatibility check applies at the seam ([Section 6.4](#)), and it fixes the *sign* of the gap so the author always writes a non-negative number ([Table 3](#)). An arbitrary axial position needs no fourth kind — it is already expressible as a station fraction.

Seat	Seam geometry	Radial check	Gap meaning (sign in +z = forward)
Abut	rim to rim	outer radii equal	forward standoff (+z)
NestInBore	child inside parent bore	child OD $\leq$ parent bore	insertion depth (−z, child moves aft)
OnSurface	child on parent outer wall	outer radii equal	axial standoff (+z)

Table 3. The three seat kinds. Each fixes both the seam’s radial compatibility test and the direction a positive gap displaces the child, so no authored negative gaps exist.

Three snap verbs — `abut(gap)`, `nestInBore(depth)`, and `seatOnWall(parentStation01)` (`core/model/parts/Placement.h:134`) — produce a `StationLink` by value for the common cases. They are pure sugar over the brace-initializer form: no pose is stored and no part is mutated, they simply read better as verbs at call sites such as the CLI’s link parser (`cli/Repl.cpp:226`), which seeds each seat’s canonical station pair from its verb before applying the author’s explicit overrides ([Section 9](#)).

Reserved orientation. `childRot` is carried in every link but is the identity throughout the current 3-DOF regime. It exists so the placement schema needs no change when 6-DOF arrives: `Pose::compose` already multiplies orientations, and a stored 3-DOF link is bit-identical to a pure-translation tree. The cost of this reserve is one identity quaternion multiply per child per resolve.

Derived geometry. A `Station` (`core/model/parts/Placement.h:67`) is a resolved axial landmark — z , outer radius, inner radius — produced by `stationAt` and never stored. A `Pose` (`core/model/parts/Placement.h:79`) is a rigid transform into the sub-tree root frame: a fore-plane origin on the axis plus an orientation, with `compose` implementing the rotate-then-translate composition law (vector addition while orientations are identity). A `Placed` (`core/model/parts/Placement.h:98`) pairs a non-owning part pointer with its resolved pose and its seat parent’s id. The resolver emits `Placed` entries in deterministic depth-first attachment order, parent before child, which yields two guarantees consumers lean on: the envelope sweep is reproducible, and `parentId` always names an *earlier* entry in the vector (0 for the sub-tree root), so a consumer can reconstruct the tree relation by streaming the vector once, without access to the ownership tree itself.

Diagnostics values. An `OverlapDiagnostic` (`core/model/parts/Placement.h:109`) locates one violation: offender id, host id, the worst station, the penetration depth, and a human-readable message.

Parts are named by stable id (`Part::getId()`), never by name — names are not unique — and never by pointer, which would dangle if the tree were edited between resolve and report. A `SolveResult` (`core/model/parts/Placement.h:123`) bundles the verdict flag with the diagnostics list; [Section 6.5](#) covers how that verdict is enforced.

6.2 Per-edge geometry: placeChild

All per-edge placement math is one pure function, `placeChild` (`core/model/parts/Placement.cpp:39`, [Listing 14](#)). Given the parent’s already-resolved pose, the two parts, and the link binding them, it asks each part for the live landmark at its chosen fractional station, lines the two stations up, and displaces the child by the seat-signed gap.

```
Pose placeChild(const Pose& parentPose, const Part& parent, const Part& child,
               const StationLink& link)
{
    const Station p = parent.stationAt(link.parentStation01); // landmark on the parent
    const Station c = child.stationAt(link.childStation01);   // landmark on the child

    // signedGap encodes seat direction in +z = forward: NestInBore inserts aft (-z, gap is depth),
    // Abut/OnSurface stand off forward (+z). The author always writes a non-negative number.
    const double signedGap = (link.seat == SeatKind::NestInBore) ? -link.gap : link.gap;

    // Line the child's station up with the parent's, then displace by the gap. The pose locates the
    // child's fore plane, which sits c.z from its chosen station -- hence the '- c.z'.
    const double childOriginZ = p.z + signedGap - c.z;

    const Pose childInParent(Vector3(0.0, 0.0, childOriginZ), link.childRot); // coaxial in 3-DOF
    return parentPose.compose(childInParent);
}
```

Listing 14. The complete per-edge placement math (`core/model/parts/Placement.cpp:39`).

The central line reads as physical intent: `childOriginZ = p.z + signedGap - c.z`. The parent’s chosen station sits at `p.z` in the parent frame; the child must be positioned so its own chosen station lands there; and because a pose locates the child’s fore plane — its local origin — rather than the chosen station, the station’s own offset `c.z` is subtracted back out. The seat supplies the gap’s direction, so **Abut** and **OnSurface** standoffs move the child forward while a **NestInBore** depth drives it aft into the bore. In 3-DOF the child is coaxial with its parent ($x = y = 0$); radial position is a property the diagnostics *check*, not one the author places.

6.3 The node-tree resolver

Turning a tree of links into absolute poses is a depth-first walk, and the walk lives with the tree: `resolveNodes` (`core/model/PartsModel.cpp:29`, [Listing 15](#)) is a file-local function of `core/model/PartsModel.cpp` that plants the sub-tree root at the local origin (`Pose{}`) and composes one `placeChild` result per edge on the way down, emitting one `Placed` per node in attachment order.

The resolve is not run per consumer. `PartNode::ensurePlacementCache()` (`core/model/PartsModel.cpp:144`) runs `resolveNodes` and then `sweepOverlaps` once per structural change — node attached or detached, link edited, geometry mutated — and caches both the placement vector and the verdict on the node, `mutable` behind `const` accessors. `resolvedPlacements()` and

```

/// Depth-first resolve of a node tree from @p pose: one Placed per node in attachment order,
/// parent before child.
void resolveNodes(const PartNode& n, const part::Pose& pose, part::PartId parentId,
                  std::vector<part::Placed>& out)
{
    out.push_back(part::Placed{&n.part(), pose, parentId});
    for(const auto& child : n.children())
    {
        resolveNodes(*child, part::placeChild(pose, n.part(), child->part(), child->link()),
                     n.part().getId(), out);
    }
}

```

Listing 15. The node-tree resolver (`core/model/PartsModel.cpp:29`). The tree walk is ten lines because all geometry lives in `placeChild`; planting the root at the local origin makes *any* node a valid sub-assembly root for the composite queries.

`placementDiagnostics()` serve the cached values on every subsequent read; a mass change (a burning motor) never re-triggers them, because geometry is invariant under a burn ([Section 5](#)).

One resolve, many consumers

Every consumer of absolute placement reads the same cached resolve through `resolvedPlacements()`: the composite mass/CM/inertia pass (`core/model/PartsModel.cpp:171`), the aero datum shift (`core/model/PartsModel.cpp:224`), the envelope sweep itself, the CLI's `checkdesign` verdict (`cli/Repl.cpp:969`), and the visualizer's mesh walk (`visualizer/RocketMesh.cpp:433`). Two independent resolvers could drift — the simulator flying one rocket while the renderer draws another — so no consumer is permitted to compute its own placement; disagreement is unrepresentable rather than tested for.

6.4 Seam and envelope diagnostics

Diagnostics come in two granularities: a per-seam radial check for one attachment, and a whole-tree envelope sweep over a resolved placement vector.

The radial seam check. `radialSeamCheck` (`core/model/parts/Placement.cpp:57`) vets one parent-child edge, dispatched on the seat kind exactly as [Table 3](#) states: **Abut** and **OnSurface** demand equal outer radii at the mated stations, **NestInBore** demands the child's OD fit within the parent's bore. It returns a located `OverlapDiagnostic` on incompatibility and `nullopt` otherwise; its `zWorld` carries the parent-local seam station, since no pose is in scope at seam granularity. Being free and pose-free, it is directly exercisable by authoring flows and by the unit suite one seam at a time (`core/model/tests/ResolverSweepTests.cpp`).

The envelope sweep. `sweepOverlaps` (`core/model/parts/Placement.cpp:108`) checks the whole resolved tree for physical impossibility: one part's material occupying another's. It works entirely on the `Placed` vector — `parentId` gives it the tree relations it needs, so it never touches the ownership tree. For each part it builds the world axial interval $[z_{\text{fore}} - \text{axialLength}, z_{\text{fore}}]$, sorts intervals by aft station ($O(N \log N)$), and then tests each part (the *offender*) against the part whose interior it occupies (the *host*) at a finite set of stations.

Exact, not sampled

The test stations are *feature breakpoints*: the offender's own endpoints plus every other interval endpoint falling within its span. For the stack profiles — a tube constant, a cone linear, a fin set's body disc constant — the radius is monotone between breakpoints, so the extremum of offender radius against host capacity is attained at a breakpoint and the sweep is exact: no sampling density to tune, no thin intrusion slipping between samples. The one profile outside this argument is [HollowSphere](#), whose silhouette peaks at its equator between its own endpoints; an intrusion carried only by that bulge, with no breakpoint inside the sphere's span, is not sampled there.

At each breakpoint the host is chosen as the *smallest-id covering interval*, after three exclusions: the offender itself, its descendants (a child legitimately nests inside its parent's envelope), and its direct seat parent (that relationship belongs to the seam check). Taking the smallest id makes the report deterministic when several intervals cover the same station, so a violation is always attributed to the same host regardless of sort incidentals. The capacity the offender must fit within comes from `Part::innerCapacityAt` ([core/model/parts/Part.cpp:53](#)), which encodes the *solid-host rule*: a bored part is bounded by its inner wall, a solid part by its outer skin. Branching once inside the part keeps the sweep free of solidity special cases — the same comparison flags both a rod jammed through a motor mount and a fin can poking out of a solid boattail.

Two further host rules are guards in the sweep loop itself ([Listing 16](#)):

```
if(cand.zFore - cand.zAft <= tol) { continue; } // zero-span node (a Motor): no interior, hosts
↳ nothing
// ...
const double rOff = off->part->radiusOuterAt(zWorld - off->originZ);
const double cap = host->part->innerCapacityAt(zWorld - host->originZ);

// A hollow host's bore capacity gates only an offender nested within its outer envelope. Once
// the offender's outer radius reaches the host skin (rOff >= hostOuter) it sits on or outside
// that skin -- e.g. a fin set's body disc on a co-radial aft coupler -- so a bore "intrusion"
// there is a modelling artifact, not a collision. A solid host has cap == hostOuter, so this
// guard never trips and poke-through still flags.
const double hostOuter = host->part->radiusOuterAt(zWorld - host->originZ);
const bool boreHost = cap < hostOuter - tol; // hollow: capacity is the bore, not the skin
if(boreHost && rOff >= hostOuter - tol) { continue; } // offender lies on/outside the host skin
```

Listing 16. The zero-span-host rule and the bore-host skin guard inside the sweep ([core/model/parts/Placement.cpp:183](#), elided).

A zero-span node is mass, not envelope

A node with no axial extent has no interior and never hosts. The canonical case is [Motor](#) ([Section 7](#)), which keeps the base `getLength()` of 0: it contributes mass, CM shift, and inertia to the composite, but it is a point in the placement geometry, and a point cannot be intruded into. Without this rule a zero-length interval would cover exactly its own station and, as a low-id “host” of zero capacity, flag every co-located part as an offender.

The bore-host skin guard resolves the complementary edge case: a hollow host's bore capacity is only meaningful for an offender *inside* its outer envelope. An offender whose outer radius reaches the host's skin sits on or outside that skin — a fin set's body disc wrapped around a co-radial coupler

is the canonical case — so comparing it against the bore would manufacture an intrusion where none physically exists. Because a solid host’s capacity equals its skin, the guard never trips for solids and genuine poke-through still flags. Finally, the sweep reports only the worst (deepest) violation per offender/host pair, so one misplaced part yields one located diagnostic rather than a breakpoint-by-breakpoint flood.

6.5 The diagnostics gate

The sweep’s verdict is cached beside the resolve it judged — `verdict_` is written in the same `ensurePlacementCache()` pass that fills `resolved_` (`core/model/PartsModel.cpp:144`) — so the verdict and the geometry it describes can never be observed out of step, and no consumer can disagree with another about whether the design is sound. Figure 8 traces the full path from a structural edit to the gated consumers.

A self-intersecting design does not fly

When the cached verdict is not **OK**, `computeCompositeAt` throws `std::runtime_error` carrying the first diagnostic’s located message (`core/model/PartsModel.cpp:184`) — and because cache stamps are written only after a successful build (Section 5), the throw repeats on every composite read rather than being swallowed by the cache. A design whose geometry is physically impossible produces a hard, located failure, never a silently wrong mass, CM, or inertia tensor fed to the integrator.

Consumers differ only in how they surface the shared verdict. The flight path refuses to solve, as above (Section 10). The CLI’s `checkdesign` command (`cli/Repl.cpp:969`) prints each diagnostic’s message behind an **ERR** verdict, and layers one additional check the sweep deliberately does not make: an axial-coverage scan for air gaps, since a positive standoff is legal in a `StationLink` but leaves parts floating apart — a question of authored intent, not of physical impossibility (Section 9). The visualizer (`visualizer/RocketMesh.cpp:433`) keeps rendering — a picture of the broken design is exactly what a user fixing it needs — but draws each offender in an error color and logs the same messages, so a headless caller sees the same signal the simulator throws on. One resolve, one verdict, three presentations.

7 The motor subsystem

The motor is an ordinary node in the part tree with special provenance. Structurally it is a leaf like any other: it occupies one `PartNode`, seats against its parent through the same `StationLink` as every part (Section 6), contributes to the composite mass, CM, and inertia tensor through the same two-pass node walk (Section 5), and rides the same `aboutTo/did` event stream into the GUI (Section 11). What distinguishes it is where its data comes from and what varies at runtime. Every geometric part is authored — its dimensions are design parameters, built by the part factory and serialized as such. A motor is *catalog data*: a manufacturer’s measured thrust curve and mass breakdown, selected from `MotorModelDatabase` by common name, with a burn that makes it the only part whose mass is a genuine function of time.

The design concentrates that specialness into a deliberately small surface: one leaf type (`model::part::Motor`, `core/model/parts/Motor.h`) that wraps a `MotorModel` by value, and a compact verb set on `PartsModel` — one attach/swap verb, four `const` reads, and exactly one runtime mutation. Figure 9 shows the pieces and the two non-obvious edges between them: the friend seam and the

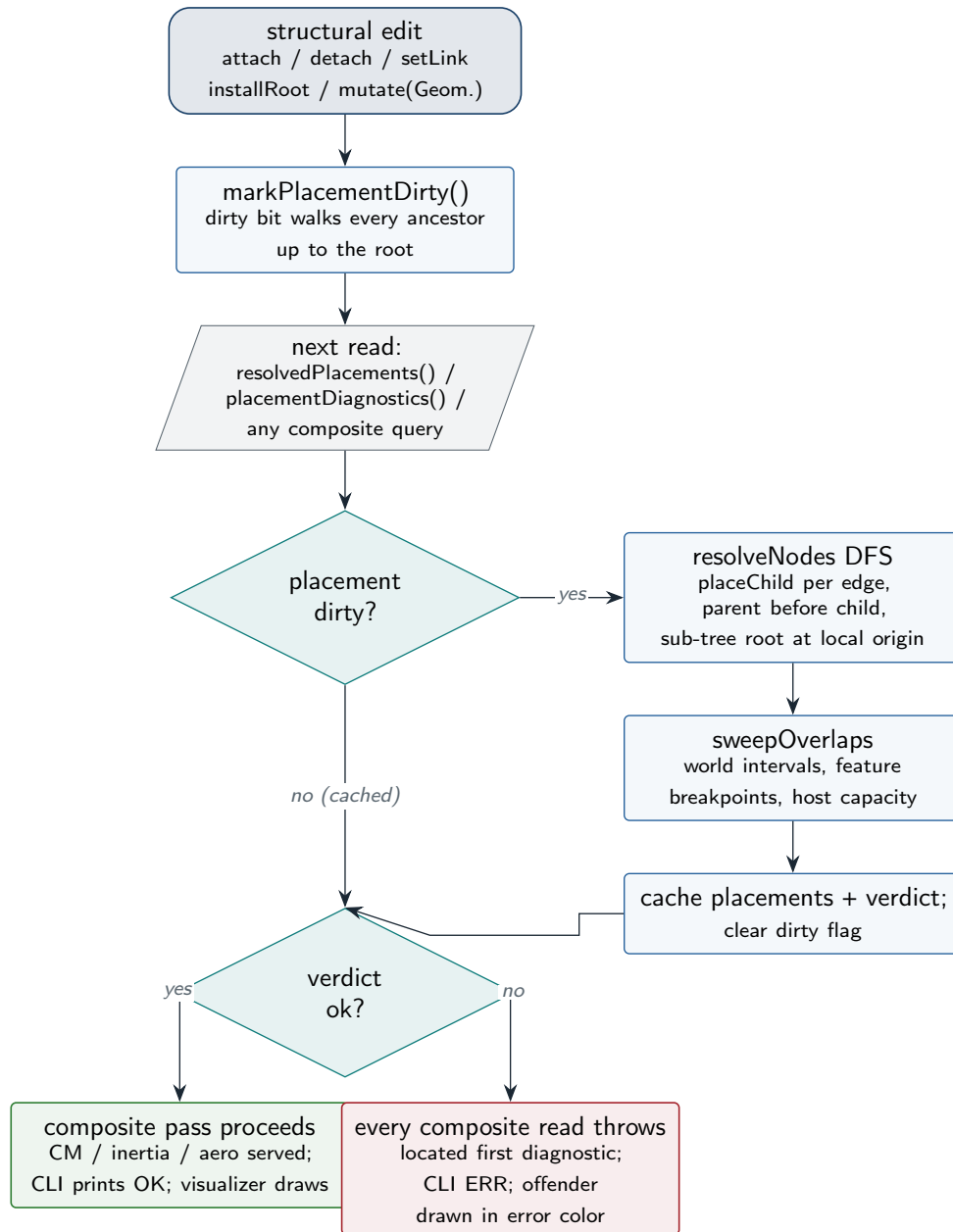


Figure 8. From structural edit to verdict: the edit dirties the placement chain up to the root; the next read resolves once, sweeps once, and caches both; every consumer then sees the same placements and the same verdict.

typed borrow.

7.1 The Motor leaf: everything derives live

Motor owns one `MotorModel` by value (the member `mm`, `core/model/parts/Motor.h:71`) and overrides exactly the two mass-property queries, both of which consult `mm` at call time (Listing 17):

```
class Motor : public Part
{
public:
    Motor(const std::string& name, const MotorModel& motor);
    std::string typeName() const override { return "Motor"; }

    /// This part's own mass at time @p t (kg): loaded weight pre-ignition, falling to the
    /// casing mass over the burn, constant after burnout. @see MotorModel::getMass
    double getMass(double t) const override { return mm.getMass(t); }

    /// Per-unit-mass tensor derived live from the wrapped MotorModel -- a solid cylinder
    /// from its catalog diameter/length.
    Matrix3 getI() const override { return motorTensor(mm); }

    const MotorModel& getMotorModel() const { return mm; }
    // ...

private:
    friend class model::PartsModel; // motor verbs: in-place swap, startMotor
    void setMotorModel(const MotorModel& motor);
    static Matrix3 motorTensor(const MotorModel& motor);
    MotorModel mm; ///< the wrapped motor, owned by value
};
```

Listing 17. The Motor leaf (`core/model/parts/Motor.h`, elided). The public surface is read-only; the mutation surface is private and reachable only through the `PartsModel` friend seam.

`getMass(t)` is the burn curve read directly. `MotorModel::getMass` (`core/model/MotorModel.cpp:28`) returns the catalog loaded weight (`data.totalWeight`, kg) before ignition, then during the burn interpolates a precomputed 128-sample propellant-mass table (built by `computeMassCurve` from the thrust curve, so mass leaves in proportion to delivered impulse), and after burnout returns the casing mass (`emptyMass = totalWeight - propWeight`). This single override is what makes the whole rocket's mass time-varying: `RocketModel::getMass(t)` is just the composite mass sum over the tree (`core/model/RocketModel.cpp:33`), and the motor node is the one term in that sum that moves.

`getI()` is derived live as well. `motorTensor` (`core/model/parts/Motor.cpp:23`) converts the catalog diameter and length (stored in `mm`) to metres and returns `InertiaTensors::Tube(0, d/2, L)` — the per-unit-mass solid-cylinder tensor (`core/model/InertiaTensors.h:63`), in the same geometric m^2 convention every leaf uses (Section 3). If the catalog lacks either dimension the tensor is `Matrix3::Zero()`: the motor degrades to a point mass that still contributes its full mass term. The tensor itself is constant over the burn; the composite becomes time-varying only through the $m(t)$ weighting and the CM shift that the node walk applies ($m \cdot I_{\text{unit}} + m \cdot f(d)$ per part, Section 5).

No seeded copy exists, so no swap can leave it stale

The constructor seeds the `Part` base fields for uniformity (`core/model/parts/Motor.cpp:12`), but no query path reads those copies: both overrides consult `mm` at every call. This closes a whole

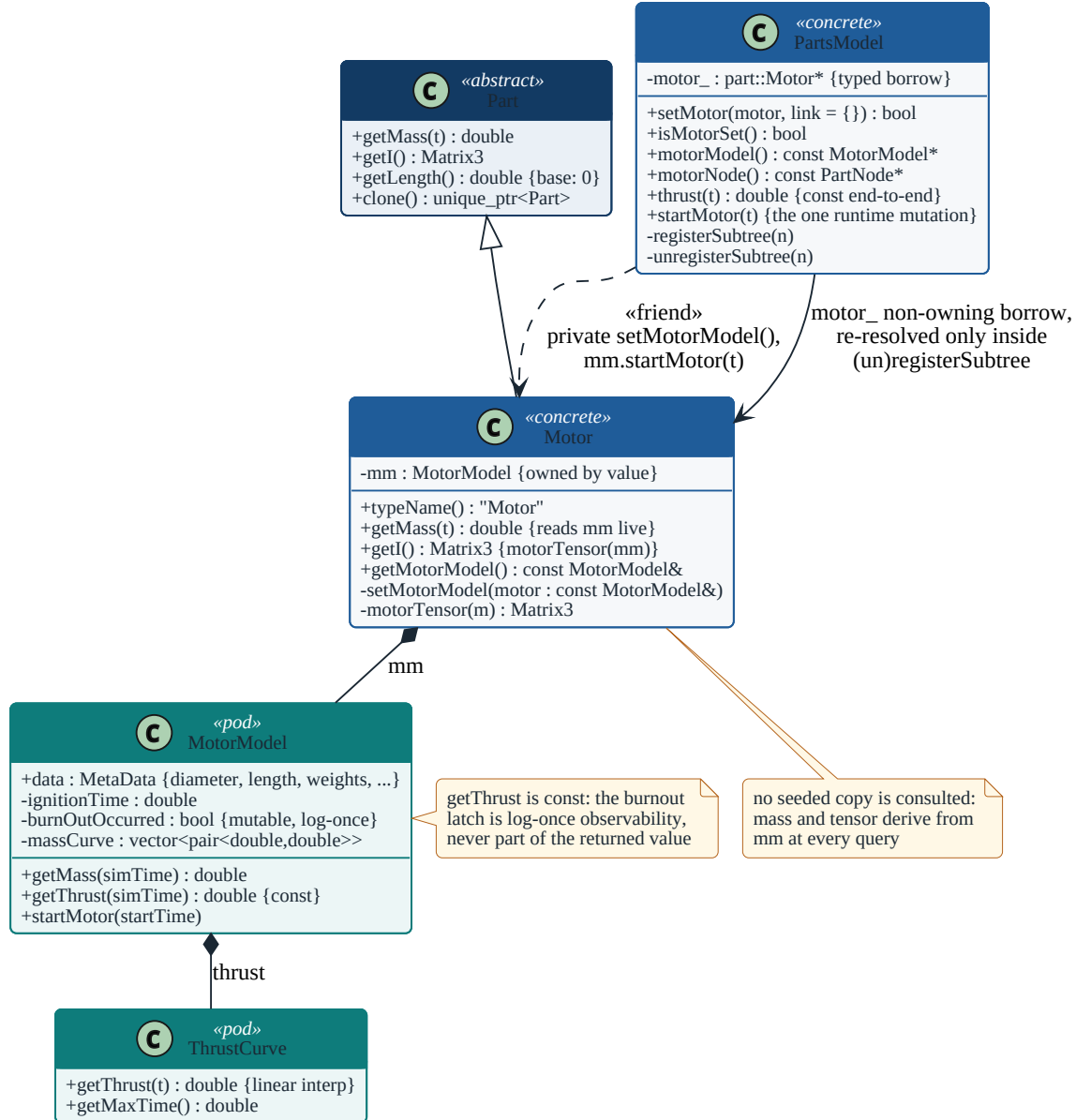


Figure 9. The motor subsystem. **Motor** owns its **MotorModel** by value and derives mass and tensor from it live; **PartsModel** holds a non-owning typed borrow into the tree it owns and reaches the private mutation surface through a friend seam. The **ThrustCurve** inside the **MotorModel** carries the manufacturer’s (time, thrust) samples, linearly interpolated.

divergence class by construction. The composite cache rebuilds behind a mass-delta gate (Section 5) — a gate that is *blind* to an equal-mass, different-diameter motor swap. If a seeded tensor copy existed and a swap path ever forgot to re-seed it, the rebuild would consult the stale copy and the flight would carry the wrong inertia with no error anywhere. With live derivation there is nothing to re-seed and nothing to forget: the swap verb replaces `mm` and dirties the mass chain, and the very next rebuild reads the new geometry because there is no other geometry to read.

Burn-dependent grain inertia is enabled, not gated out

Today `motorTensor` treats the grain as a full solid cylinder for the whole burn — a stated modeling approximation, adequate at 3-DOF where the tensor is recorded but not yet integrated. The structural point is that `getI()` is *already* a function of the live `MotorModel`. A future grain model — a core-burner’s tensor as a function of remaining propellant, read from the same mass curve — changes only `motorTensor`, growing `getI()` a time parameter mirroring `getMass(t)` (Section 12.4). The read path, the cache gates, the swap semantics, and every consumer are already shaped for a time-varying answer; nothing in the architecture has to move to admit better physics.

7.2 A point in the airframe geometry

`Motor` overrides none of the geometry surface. Its length is the base `Part::getLength()` of zero — the base-class contract explicitly names the motor as the intended zero-length node (`core/model/parts/Part.h:79`) — and its radius profiles are the base zeros. In the placement layer (Section 6) this makes it a *point node*: its resolved axial interval has zero span, so the envelope sweep never selects it as a host (`core/model/parts/Placement.cpp:183`), and its zero outer radius means it can never flag as an offender. It neither claims radial capacity nor grants it; it never extends the airframe’s envelope.

This is deliberate decoupling of inertial geometry from envelope geometry. The catalog knows the motor’s real casing dimensions, and `getI()` uses them — that is real physics the simulation wants. But `QtRocket` does not model motor-mount internals, and a motor given a real solid envelope would immediately collide with the body tube it sits inside, forcing every design to author a mount before the envelope sweep would pass. A point mass carrying a true cylinder tensor at the right station is the honest 3-DOF representation: correct mass properties, no fictitious solid to police.

Placement-wise the motor is unremarkable, by design. `setMotor` attaches it under the root with a default `StationLink` — the same zero-config “child fore plane abuts parent aft plane” every part gets — or with any explicit link the caller passes, and the seat can later be re-authored through the ordinary `setLink` verb (Section 4). Because $L = 0$ and the CM offset is zero, its composite CM contribution lands exactly at its resolved fore-plane origin.

7.3 The verbs and the typed borrow

`PartsModel` keeps one piece of motor-specific state: `part::Motor* motor_` (`core/model/PartsModel.h:261`), a non-owning typed borrow into the tree it already owns. The borrow exists so that `thrust(t)`, `isMotorSet()`, and `startMotor(t)` are $O(1)$ — the per-RK-stage hot path must not walk the tree hunting for a `Motor` by `dynamic_cast`.

A borrowed pointer into an owned structure is only safe if it is maintained wherever the structure changes, so the design pins it to the narrowest possible seam (Listing 18):

```

void PartsModel::registerSubtree(PartNode* n)
{
    n->model_ = this;
    index_[n->id()] = n;
    if(auto* m = dynamic_cast<part::Motor*>(n->part_.get()))
    {
        motor_ = m;
    }
    for(const auto& child : n->children_) { registerSubtree(child.get()); }
}

void PartsModel::unregisterSubtree(PartNode* n)
{
    if(motor_ != nullptr && static_cast<part::Part*>(motor_) == n->part_.get())
    {
        motor_ = nullptr;
    }
    index_.erase(n->id());
    n->model_ = nullptr;
    for(const auto& child : n->children_) { unregisterSubtree(child.get()); }
}

```

Listing 18. Borrow maintenance (`core/model/PartsModel.cpp:270`). The same walk that maintains the id index maintains the motor borrow; no structural verb touches the borrow directly.

The invariant lives with the only code that can break it

Every path by which a node enters or leaves the tree — `attach`, `attachSubtree`, `detach`, `installRoot` — funnels through `registerSubtree` / `unregisterSubtree`, and those two functions are the only writers of `motor_` (`core/model/PartsModel.h:261`). Clearing the design, loading a new one, or detaching a sub-tree that happens to contain the motor therefore nulls the borrow as a side effect of the same walk that unindexes the nodes; there is no separate “remember to fix the motor pointer” step to forget. The CLI leans on this directly: after `removepart` it simply asks `isMotorSet()` again, because the motor may have been in the removed sub-tree (`cli/Repl.cpp:1062`).

The tree admits at most one motor, and the invariant is enforced at the only gate through which parts enter: `attachSubtree` rejects with `AttachError::DuplicateMotor` when a motor is already set and the incoming sub-tree contains a *Motor anywhere* — a recursive scan, not a root check, because a pasted clone can carry a motor several levels down (`core/model/PartsModel.cpp:317`). The rationale is physical consistency: with the motor an ordinary node, a second one would burn mass in the composite while only the borrowed one produces thrust — a silently inconsistent flight. Rejecting at `attach` also keeps every reachable design serializable, since the file format carries exactly one motor reference (Section 8).

`setMotor` is the install-or-swap verb, and its two branches are intentionally asymmetric (Listing 19 and Fig. 10):

Rejected — swap by detach-and-reattach

Replacing the motor *node* on every selection would be simpler to write but worse everywhere it is observed. Part ids are per-instance and never reused, so a new node means a new id —

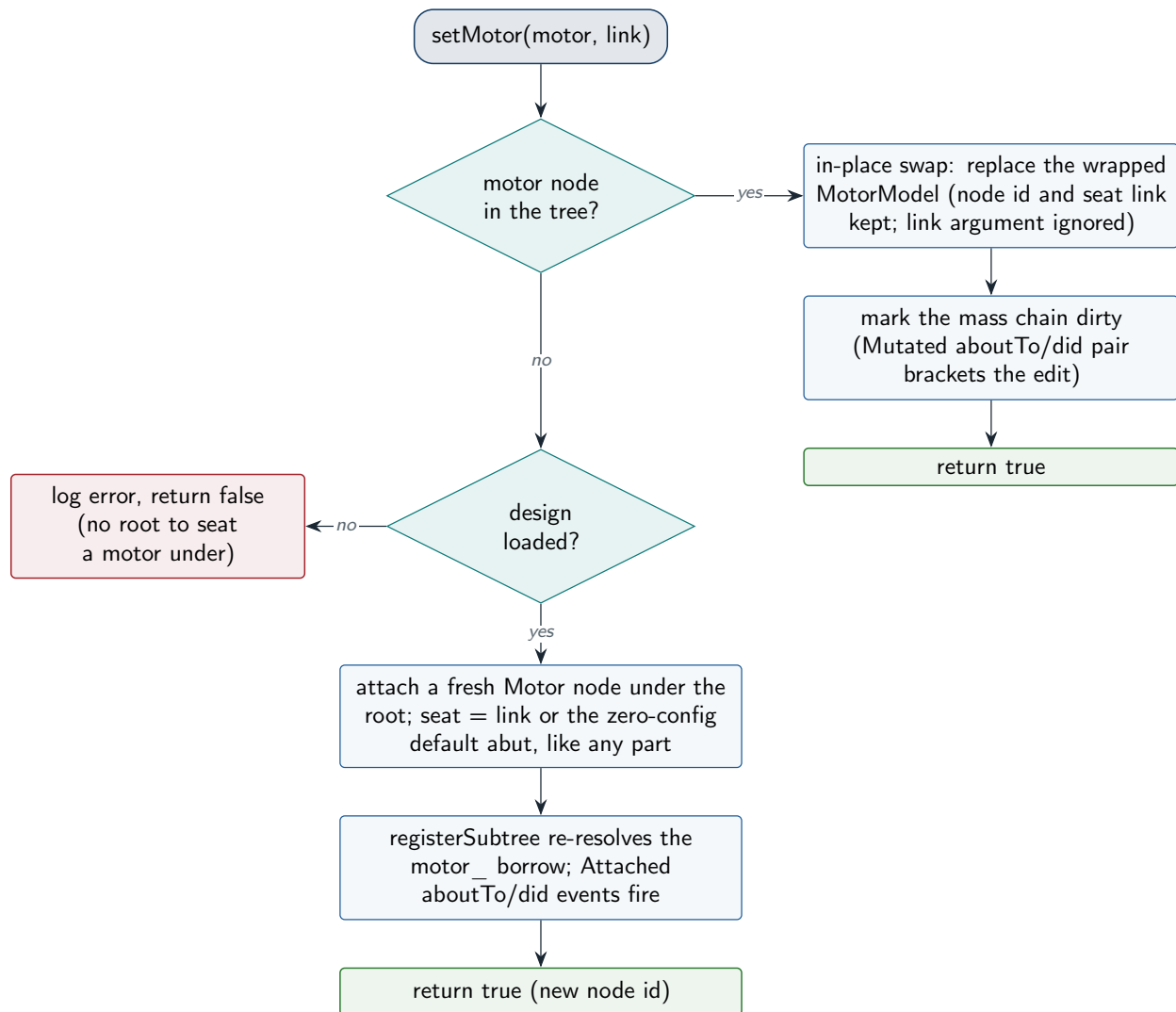


Figure 10. The `setMotor` flow. A swap is a leaf mutation — node id and seat link survive, the mass chain is dirtied, and the event stream sees `Mutated`. A first install is a structural attach with the zero-config default seat. With no design loaded the verb fails closed.

```

bool PartsModel::setMotor(const MotorModel& motor, std::optional<part::StationLink> link)
{
    if(motor_ != nullptr)
    {
        // In-place swap keeps the node, its id, and its link; getMass(t)/getI() read the new
        // MotorModel live, so only the cache chain needs dirtying. @p link is ignored on a swap.
        PartNode* n = find(static_cast<part::Part*>(motor_)->getId());
        const Event e{Event::Mutated, n->id(), /* ... */};
        emitEvent(e, true);
        motor_>setMotorModel(motor);
        n->markMassDirty();
        emitEvent(e, false);
        return true;
    }
    if(!root_)
    {
        utils::Logger::getInstance()->error("PartsModel::setMotor: no design to attach a motor to");
        return false;
    }
    return attach(root_->id(), std::make_unique<part::Motor>("Motor", motor),
                  link.value_or(part::StationLink{}))
        .has_value();
}

```

Listing 19. `PartsModel::setMotor` (`core/model/PartsModel.cpp:470`, elided). First call: an ordinary attach under the root. Every later call: an in-place swap of the wrapped `MotorModel`, presented to observers as a `Mutated` edit on an unchanged node.

invalidating any id the CLI or GUI holds; the event stream would carry a `Detached/Attached` pair, forcing Qt views through row-removal/insertion churn for what the user perceives as editing a property; and the authored seat link would have to be saved and restored around the splice. The in-place swap keeps identity stable and reduces the edit to exactly what changed: the wrapped catalog data, plus a mass-chain invalidation.

The read surface is three accessors: `isMotorSet()` (the null test on the borrow), `motorModel()` (a borrowed `const MotorModel*`, explicitly not for storing), and `motorNode()` (the motor's `PartNode`, existing chiefly so the serializer can persist the motor's seat link like any other edge, `core/model/PartsModel.h:241`).

7.4 Const thrust, one runtime mutation

The flight loop's entire motor read is `thrust(t)`: each RK stage evaluates `RocketModel::getForces`, whose thrust term is `parts_.thrust(t)` (`core/model/RocketModel.cpp:67`), which forwards to `MotorModel::getThrust` — zero with no motor set. The chain is `const` end-to-end, and that is a documented property, not an accident: `getThrust` returns a pure function of the thrust curve and the ignition epoch (`core/model/MotorModel.h:325`).

The one wrinkle is a `mutable` write inside `getThrust`: the first post-burnout call logs the burnout time and sets a latch so the line is logged once (`core/model/MotorModel.cpp:74`). The latch is observability, not physics — it never participates in the returned value, so the function is honestly `const` in the sense that matters: two calls with the same arguments return the same thrust regardless of

latch state. It joins the composite caches under the model layer’s single-writer `mutable` discipline (Section 5). Keeping the hot path `const` is itself a design tool here: the compiler now certifies that per-step force evaluation cannot mutate the motor, which makes the single genuine runtime mutation impossible to miss.

That mutation is `startMotor(t)` — the ignition epoch. `MotorModel` keeps thrust and mass as functions of $t - t_{\text{ignition}}$, and `startMotor` sets `ignitionOccurred` and `ignitionTime` (`core/model/MotorModel.h:330`). It is routed like every other post-attach edit: `PartsModel::startMotor(t)` reaches `motor_>mm.startMotor(t)` through the friend seam (`core/model/PartsModel.cpp:511`), and `RocketModel::launch` invokes it at $t = 0$ immediately before the propagator runs (`core/model/RocketModel.cpp:104`; Section 10).

Rejected — a public `Motor::setMotorModel`

Both mutators on `Motor` — the swap and, via `mm`, ignition — are private, with `PartsModel` as a friend (`core/model/parts/Motor.h:60`). A public mutator would be a standing invitation to swap the motor through any `Part` reference without dirtying the composite caches, and the mass-delta gate would mask the damage in exactly the worst case: an equal-mass swap whose diameter differs. Routing every mutation through `PartsModel` verbs means cache invalidation is not a caller obligation but a structural consequence — the same principle that makes every leaf setter private (Section 3), applied to the one part with runtime state.

7.5 Facade, persistence, and clients

`RocketModel` exposes the subsystem to clients as four thin delegations: `setMotorModel` → `parts_.setMotor`, `getMotorModel` (a value copy, or a default-constructed `MotorModel` when none is set), `isMotorSet`, and `getThrust` (`core/model/RocketModel.cpp:107`). `isMotorSet()` doubles as the launch gate on both front ends: the GUI disables trajectory calculation without it (`gui/CannonballTab.cpp:88`), and the CLI mirrors it in a session flag — re-synchronized against `isMotorSet()` after any removal — whose `launch` check refuses with a usage hint (`cli/Repl.cpp:744`). The CLI’s `setmotor` command is the whole selection flow in miniature: resolve a common name against `MotorModelDatabase`, then `setMotorModel` (`cli/Repl.cpp:559`; Section 9).

Persistence is by database reference, consistent with the motor’s provenance. The design-file writer never emits the motor as a tree part — `writePart` skips `Motor` children (`core/model/DesignSerializer.cpp:95`) — and instead records a single element carrying the catalog common name, plus the motor node’s seat link when it differs from the default. On load the name is re-resolved against the supplied database: a hit rebuilds the `Motor` node under the root with its persisted link, a miss warns and loads the design without a motor, and a hand-edited file that smuggles `type="Motor"` in as a tree part is rejected outright (`core/model/DesignSerializer.cpp:155`). A thrust curve is a measurement, not a design parameter; the design cites it rather than embedding it, and Section 8 covers the format and its failure modes in full.

8 Design persistence: the .qrd format

A rocket design persists as a .qrd file: versioned XML written and read by the two static entry points of `DesignSerializer` (`core/model/DesignSerializer.h`), on Boost property tree, the same XML machinery the motor database uses. The format records exactly what the in-memory model treats as authoritative — the part tree’s typed geometry parameters, each edge’s placement *intent*, the motor’s identity, and the `RocketModel`-owned aero options — and nothing the model derives. No

resolved pose, no composite mass, no absolute coordinate ever reaches disk; the resolver re-derives all of it on load (Section 6), so a hand-edit that lengthens a tube reloads with every seam moved to follow the new geometry. Both front-ends reach the format only through these two calls: the CLI's `savedesign/loaddesign` commands (`cli/Repl.cpp:1077`, `cli/Repl.cpp:1098`; Section 9) and the GUI's file menu (`gui/MainWindow.cpp:106`).

The reader is built around one structural commitment: the entire tree is constructed *detached* — outside any `RocketModel` — and handed over in a single `installDesign` call at the end. Every validation failure therefore lands before the current rocket is touched. This section walks the file anatomy, the parameter and link vocabularies it shares with the factory and the CLI, the motor-by-reference element, and the load pipeline that turns a file into a live design.

8.1 File anatomy and the version gate

The root element is `QtRocketDesign` with a `version` attribute; the writer emits 0.2 (`core/model/DesignSerializer.cpp:198`). The version string is *major.minor*, and the load gate keys on the major component alone: it splits at the first dot and rejects any major other than 0 (`core/model/DesignSerializer.cpp:233`), so the reader is forward-tolerant across minor revisions — a minor bump signals an element-shape change the reader can detect structurally, not a compatibility break. Under the root sit four elements: a `<design name=...>` attribute element carrying the design's human-facing name, one recursive `<part>` tree, an optional `<motor>` reference, and a `<sim>` options block. Listing 20 shows a complete file from the test corpus.

Each `<part>` element carries `type` (the part's `typeName()`, the exact factory key — `NoseCone`, `BodyTube`, `FinSet`, `HollowSphere`) and `name` attributes, a `<params>` attribute element, an optional `<link>`, and a `<children>` element holding the child `<part>`s in attachment order. The ownership tree in the XML is the tree structure; parts are never cross-referenced by name (names are not unique — identity is the per-instance id, which is process-local and deliberately not serialized).

The unpretty decimals are deliberate: the property-tree double translator emits the full 17 significant digits a `double` round-trip requires, so a save/load cycle reproduces geometry — and therefore mass, CM, and inertia — bit-exactly. The format optimizes for faithfulness over prettiness; a human authoring a file by hand writes 0.4 and loses nothing.

8.2 Parameters: the factory round-trip

The `<params>` attributes are not a per-type schema. They are the one shared vocabulary of `PartParams` (`core/model/parts/PartFactory.h`): every attribute name in the file is a field of that struct, the writer produces it by calling the reflector `params(part)` (`core/model/parts/PartFactory.h:69`), and the reader feeds it back to `makePart(type, params)` (`core/model/parts/PartFactory.h:60`), whose documented contract is that `params(makePart(p))` reproduces the geometry. Because the CLI's `addpart` key=value parser fills the same struct, the on-disk attribute names cannot drift from the interactive vocabulary — there is exactly one spelling of `outerRadius` in the system, and it is a struct field, not a string constant in two parsers.

Every field is `std::optional`, and `writeParams` emits only the fields that are present (`core/model/DesignSerializer.cpp:32`), so a nose cone's element carries cone attributes and nothing else. On read, a missing key stays `std::nullopt` — distinct from a supplied zero — and `makePart` applies its documented defaults or throws for a missing required field, the same fail-closed path an interactive command takes. The one non-numeric field, `solid`, serializes as the strings `true/false`, matching the motor database's XML idiom.

```

<?xml version="1.0" encoding="utf-8"?>
<QtRocketDesign version="0.2">
  <design name=""/>
  <part type="NoseCone" name="Nose">
    <params baseRadius="0.0155" length="0.14999999999999999" wallThickness="0"
      density="2500" solid="true"/>
  <children>
    <part type="BodyTube" name="Airframe">      <!-- no <link>: default abut -->
      <params innerRadius="0.0146" outerRadius="0.0155"
        length="0.40000000000000002" density="2500"/>
      <children>
        <part type="FinSet" name="Fins">
          <params density="2500" rootChord="0.059999999999999998"
            tipChord="0.029999999999999999" span="0.040000000000000001"
            sweep="0.02" thickness="0.0025000000000000001"
            bodyRadius="0.0155" finCount="6"/>
          <link seat="OnSurface" parentStation="0" childStation="0" gap="0"/>
        <children/>
      </part>
      <part type="BodyTube" name="Coupler">
        <params innerRadius="0.012" outerRadius="0.0146"
          length="0.050000000000000003" density="2500"/>
        <link seat="NestInBore" parentStation="1" childStation="0"
          gap="0.050000000000000003"/>
        <children/>
      </part>
    </children>
  </part>
</children>
</part>
<motor commonName="G80T"/>
<sim dragCoefficient="1" referenceArea="0.001134"
  referenceAreaOverridden="false"/>
</QtRocketDesign>

```

Listing 20. A complete .qrd file (tests/data/designs/mid29_multi.qrd; attribute lists re-wrapped to the page and the default-abut annotation added, content otherwise verbatim). All three seat kinds appear: the airframe abuts the nose by the elided default, the fins seat **OnSurface**, the coupler nests **NestInBore**. The motor is a database reference, not an inlined thrust curve.

8.3 Placement: the `<link>` element, elided by default

A child's spatial relationship to its parent is its `StationLink` (Section 6), serialized as an inline `<link>` on the child with exactly the four authored quantities: `seat`, `parentStation`, `childStation`, `gap`. The `seat` enumerator maps to its attribute string through the single shared table `seatKindToString/seatKindFromString` (`core/model/parts/Placement.h:41`); the writer (`core/model/DesignSerializer.cpp:57`), the reader (`core/model/DesignSerializer.cpp:140`), and the CLI's `seat=` token parser (`cli/Repl.cpp:232`) all consult it, so the three agree on the spellings (**Abut**, **NestInBore**, **OnSurface**) by construction rather than by three independently maintained switch statements. `seatKindFromString` returns `std::nullopt` for an unknown name, and the reader converts that into a thrown error on the same path as `makePart`'s unknown-type rejection — a hand-edited `seat` typo is a refused file with a directed diagnostic, never a silent coercion to **Abut**.

The link's quaternion (`childRot`) is not persisted: it is identity in 3-DOF, so a stored link is fully described by its scalars and `seat`, and the file format acquires the 6-DOF field only when a non-identity value exists to store.

The common case costs zero authored numbers

`writePart` compares each link against a default-constructed `StationLink` (`isDefaultLink`, `core/model/DesignSerializer.cpp:67`) and elides the element entirely when they are equal (`core/model/DesignSerializer.cpp:85`); the reader, seeing no `<link>`, attaches by the same default (`core/model/DesignSerializer.cpp:175`). The default is a meaningful physical relationship — child fore plane abutting parent aft plane, the natural nose→body→... stack — so eliding it loses nothing, and the two halves of the contract (elide on write, default on read) are pinned to the same default-constructed struct. The payoff is visible in Listing 20: the airframe, the largest part in the design, is attached by zero authored placement numbers, and the file's size tracks the design's *placement complexity*, not its part count.

Within major version 0, one element shape is refused by name: a 0.1-era file records placement as a resolved CM-to-CM `<offset>` coordinate triple, which the intent-only model cannot faithfully represent. The reader throws on that element with a message directing the user to re-create the design (`core/model/DesignSerializer.cpp:180`) — mis-seating every child by silently treating a resolved coordinate as the abut default would corrupt the design while appearing to load it.

8.4 The motor: a database reference

The `motor` element is one attribute — `<motor commonName="G80T"/>` (`core/model/DesignSerializer.cpp:205`) — plus an optional `<link>` for its seat, elided under the same default-equal rule as any other edge (`core/model/DesignSerializer.cpp:207`). The file stores a *reference* into the motor database, never an inlined thrust curve.

The database is the source of truth for motors

A motor is certified vendor data — a thrust curve, propellant and total masses, burn time — that the design merely selects (Section 7). Inlining a copy into every `.qrd` would create as many divergent copies of that data as there are saved designs, each frozen at its save date; resolving by common name on every load means a corrected or updated database benefits every existing file. It also keeps the file small and portable: a `.qrd` carries authored intent measured in tens of numbers, not a hundred-sample thrust curve.

The save walk enforces the reference form structurally: `writePart` skips any `Motor` child while recursing (`core/model/DesignSerializer.cpp:95`), so the motor — an ordinary node in the live tree (Section 4) — never appears as a `<part>`, only as the `<motor>` element. The reader enforces the inverse: `buildNode` rejects `type="Motor"` outright (`core/model/DesignSerializer.cpp:155`), because the geometry factory cannot rebuild a motor from `<params>` — a `Motor` tree part can only occur in a hand-edited or foreign file, and refusing it names the correct channel in the diagnostic.

On load, the common name is resolved against the caller-supplied `MotorModelDatabase` (`core/model/DesignSerializer.cpp:246`). A hit attaches a fresh `Motor` node under the root with the persisted (or defaulted) link (`core/model/DesignSerializer.cpp:253`); a miss logs a warning and the design loads without a motor (`core/model/DesignSerializer.cpp:257`) — degraded, flyable after the user selects a motor, and fully intact in geometry.

Rejected — fail-closed motor resolution

Symmetry with the seat and type gates argues for throwing on a motor miss; the reference form argues the other way, and wins. A seat typo is evidence the file is corrupt, but a missing motor is evidence about the *machine*: the same valid file loaded on a workstation whose database lacks one vendor file would be refused in its entirety — geometry, placements, and all — over the one datum the database can supply later. Fail-closed here would make every `.qrd` silently non-portable across database states. The warning path degrades exactly as far as the missing information requires, and Section 10's launch gate (`isMotorSet()`) already prevents flying the degraded design by accident.

8.5 Load: build detached, install once

Figure 11 traces the load pipeline; Listing 21 is its tail in code. After `read_xml` and the major-version gate, `buildNode` recursion constructs the entire tree as detached `PartNodes` — `PartNode::make` wrapping each factory-built part, `addChild` assembling bottom-up (`core/model/DesignSerializer.cpp:240`), the same detached-building path a clone uses (Section 4). The motor, when resolved, is attached to that detached root the same way. Only then does the one mutation of shared state happen: `rocket.installDesign(newRoot)` (`core/model/DesignSerializer.cpp:262`), which atomically replaces the tree through `PartsModel::installRoot` and re-resolves the motor borrow inside the install facade.

A malformed file leaves the current rocket untouched

Nothing before `installDesign` touches the `RocketModel`. Every validation failure — unreadable XML, unknown major version, unknown part type, non-physical geometry thrown by a concrete constructor, unknown seat, a `Motor` tree part, a `0.1 <offset>` — propagates out of the detached build and destroys only the half-built detached tree. A user who mistypes a filename or opens a corrupt file gets an error message and keeps the design they were editing; there is no partially-loaded state to recover from, by construction rather than by rollback code.

8.6 The `<sim>` block and the override tri-state

The `<sim>` element persists only the `RocketModel`-owned aero options: `dragCoefficient`, `referenceArea`, and `referenceAreaOverridden` (`core/model/DesignSerializer.cpp:216`). Launch and environment options — rail velocity, atmosphere, gravity, integrator — are session configuration

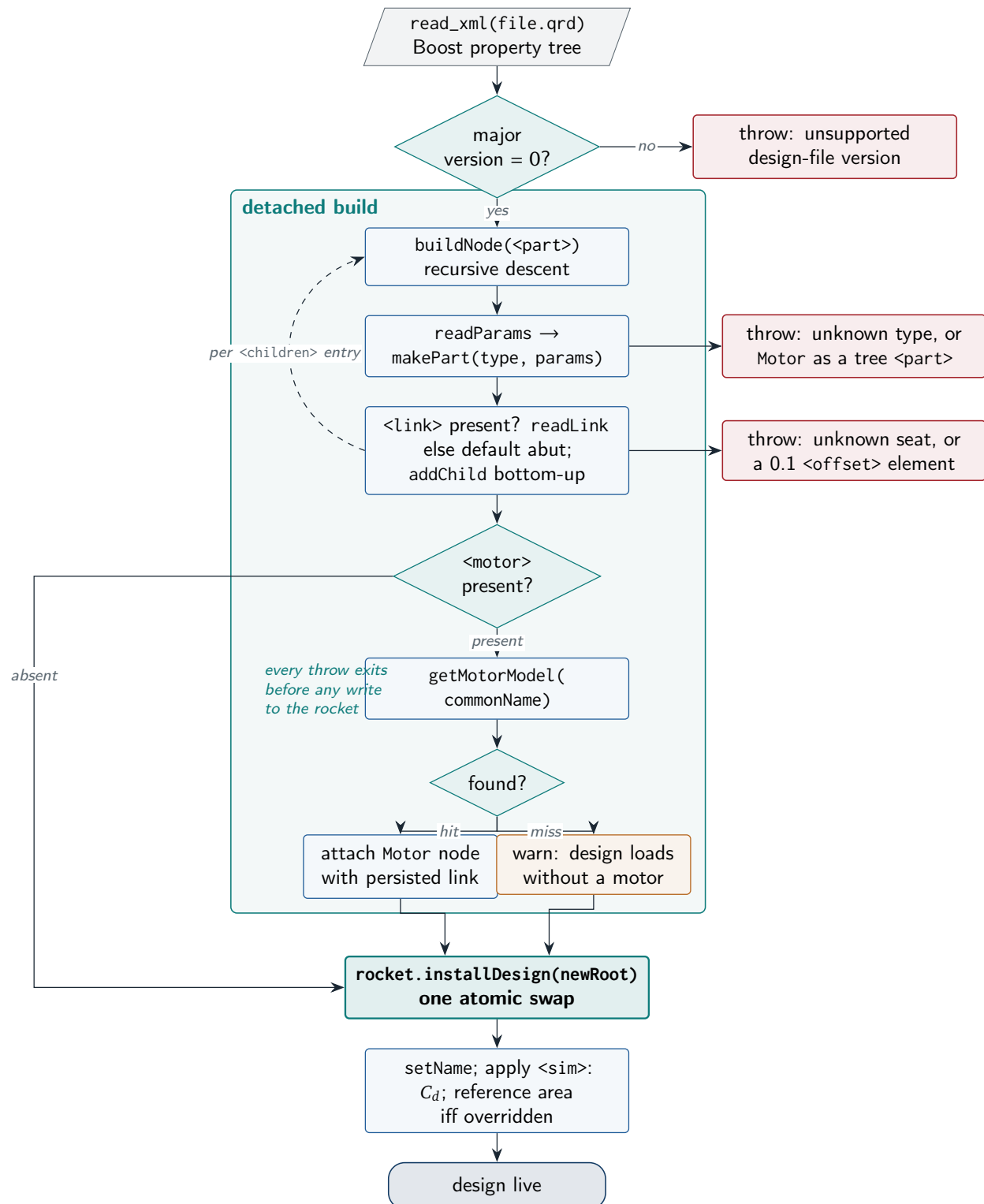


Figure 11. The load pipeline. Every fail-closed exit — version, part type, seat kind, placement element — throws from inside the detached region, before `installDesign`; the motor miss is the one soft path. The sim attributes apply after the install so a persisted manual reference area survives the install-time override reset.

```

// Build the whole tree (detached) before touching the rocket, then install in one
// shot so a malformed file leaves the existing rocket untouched.
std::unique_ptr<PartNode> newRoot = buildNode(root.get_child("part"));

// Motor by common name, re-resolved against the supplied database (a miss warns,
// never throws). An ordinary node: attached under the root with its persisted link.
if(const auto name = root.get_optional<std::string>("motor.<xmlattr>.commonName"))
{
    if(const auto m = motors.getMotorModel(*name))
    {
        part::StationLink motorLink{};
        if(root.get_child_optional("motor.link"))
        {
            motorLink = readLink(root.get_child("motor"));
        }
        newRoot->addChild(PartNode::make(std::make_unique<part::Motor>("Motor", *m)),
                           motorLink);
    }
    else
    {
        utils::Logger::getInstance()->warn(/* ... loaded without a motor */);
    }
}

rocket.installDesign(std::move(newRoot));
rocket.setName(root.get<std::string>("design.<xmlattr>.name", ""));

// Apply the sim/aero block after the install so a restored manual override wins
// over its reset.
rocket.setDragCoefficient(
    root.get<double>("sim.<xmlattr>.dragCoefficient", rocket.getDragCoefficient()));
const bool overridden =
    root.get<std::string>("sim.<xmlattr>.referenceAreaOverridden", "false") == "true";
if(overridden)
{
    rocket.setReferenceArea(
        root.get<double>("sim.<xmlattr>.referenceArea", rocket.getReferenceArea()));
}

```

Listing 21. The tail of `DesignSerializer::load` (`core/model/DesignSerializer.cpp:240`, lightly elided): detached motor attach, the single atomic install, then name and sim attributes.

owned by the front-ends (Section 9), not rocket state, and deliberately stay out of the design file: a design describes a vehicle, not a flight.

The reference area needs two attributes because it is one value with two provenances (Section 10): a geometry-derived default (the widest frontal disc in the tree) or a manual override set through `setReferenceArea`, which latches the `referenceAreaOverridden` flag (`core/model/RocketModel.h:69`). Persisting the flag alongside the value gives the reader three distinct behaviors: with `referenceAreaOverridden="true"` it calls `setReferenceArea`, restoring both the value and the manual-override provenance; with `"false"` it applies nothing — the stored `referenceArea` is a record of the value at save time, and the rocket's existing area continues to govern; with the attribute absent it likewise leaves the rocket's current state alone. Storing the value unconditionally but applying it conditionally keeps the file self-describing while never promoting a derived number into authored intent.

Ordering carries the correctness burden: `installDesign` clears the override flag as an install side effect — a freshly-installed airframe must not inherit a manual area sized for a different one (`core/model/RocketModel.cpp:121`) — so the loader applies the sim block *after* the install (`core/model/DesignSerializer.cpp:266`). A persisted override is thereby re-established on top of the reset rather than clobbered by it; applying the attributes first would make every load silently discard the user's saved area. The CLI then syncs its staged session copies from the freshly loaded rocket (`cli/Repl.cpp:1107`), so the next `fly` sees the restored values.

9 Execution walkthrough: a CLI design session

The preceding sections describe the part tree layer by layer; this one runs it. QtRocket ships a Qt-free command-line front end, `qtrocket-cli`, whose REPL (`cli/Repl.cpp`) drives exactly the API the GUI drives: the `QtRocket` controller, `RocketModel`, and the `PartsModel` verbs of Section 4. Because the CLI is a thin text skin over the model, a single scripted session exercises the factory (Section 3), the placement vocabulary (Section 6), the mutation verbs and their event stream (Section 4), the composite caches (Section 5), the motor slot (Section 7), the serializer (Section 8), and finally the flight loop (Section 10) — and every seam between them is visible in the terminal output. The transcript quoted throughout this section is genuine: it is the output of `build/cli/qtrocket-cli` running the script in Listing 22 against the bundled small-motor database, altered only where a caption notes an elision.

9.1 The line protocol

The REPL reads one command per line from any `std::istream` — interactively, from a pipe, or from a script file (`cli/Repl.h:31`) — and answers on a strict prefix protocol: a command's outcome arrives as `OK` or `ERR` status lines, with any detail on the lines that follow. The protocol is enforced at a single chokepoint. `Repl::execute` (`cli/Repl.cpp:297`) buffers each command's output, scans the buffer for `ERR`-prefixed lines to maintain an error count, and forwards the bytes verbatim; `run` then turns that count into the process exit code — 0 only if every command succeeded. A piped script is therefore CI-consumable with no output parsing at all: the exit status is the verdict. The same chokepoint wraps every command in one `try/catch`, because model reads can throw by design — a composite query on a design whose placement solve failed re-throws on every read (Section 5) — and a diagnostic that killed the session would punish exactly the user who most needs to keep editing. The catch turns the exception into an ordinary `ERR` line and the session continues.

9.2 The session

Listing 22 is the whole script. It builds a three-part rocket, deliberately provokes the design checker and the launch gate, removes the offending part, installs a motor, round-trips the design through disk, and flies it.

```
loaddb /home/travis/Development/qtrocket/tests/data/motors/estes_small.qmd
newdesign NoseCone name=Nose baseRadius=0.019 length=0.10 density=520
addpart Nose BodyTube name=Airframe outerRadius=0.019 innerRadius=0.018 length=0.45 density=680
addpart Airframe FinSet name=Fins finCount=4 rootChord=0.06 tipChord=0.03 span=0.05
    sweep=0.02 thickness=0.003 bodyRadius=0.019 density=680 seat=OnSurface
listparts
checkdesign
addpart Airframe BodyTube name=Boattail outerRadius=0.019 innerRadius=0.018 length=0.05
    density=680 gap=-0.02
checkdesign
removepart 4
checkdesign
launch
setmotor C6
listparts
savedesign demo.qrd
cleardesign
listparts
loaddesign demo.qrd
listparts
setvelocity 15
launch
quit
```

Listing 22. The session script (each `addpart` is one line; two are wrapped here to fit the page). Run as `qtrocket-cli <script>`; the same commands work interactively or via `-c "<command>"`.

9.3 Building the tree: newdesign and addpart

Listing 23 is the build phase of the output. Three code paths produce it, and each is worth tracing.

Token parsing. Every design command shares one `key=value` grammar, implemented by `parseDesignTokens` (`cli/Repl.cpp:180`). Geometry keys are routed into a `PartParams` through `doubleFieldFor` (`cli/Repl.cpp:151`), a one-page mapping from key string to `std::optional<double>` field; the placement keys (`seat`, `parentStation`, `childStation`, `gap`) accumulate separately in a `LinkTokens` staging struct. Numeric values go through strict full-string parsers that reject trailing garbage — `length=0.5abc` is an error, not a truncated 0.5. An *unknown* key, by contrast, is logged and skipped, so a script written against a newer CLI's vocabulary degrades gracefully on an older one instead of dying at the first novel key.

One parameter vocabulary, three consumers

`PartParams` (`core/model/parts/PartFactory.h`) is filled by this parser, consumed by `makePart`, and reproduced by `params()` for the serializer of [Section 8](#). Because all three speak through the same struct, the CLI's key names, the factory's required-field errors, and the on-disk attribute names cannot drift apart — there is no second vocabulary to fall out of sync.

```

OK loaddb: 13 new motors from ../tests/data/motors/estes_small.qmd (13 total)
OK newdesign: root NoseCone id=1
OK addpart: BodyTube id=2 under 1
OK addpart: FinSet id=3 under 2
OK listparts:
  [1] NoseCone "Nose"  m=0.019658 kg
  [2] BodyTube "Airframe" m=0.0355691 kg
  [3] FinSet "Fins"  m=0.01836 kg
  -- composite: mass=0.0735871 kg, cg_z=-0.306313 m (t=0)
OK checkdesign: 3 parts, contiguous, no overlaps
OK addpart: BodyTube id=4 under 2
ERR checkdesign: 1 air gap(s)
  gap: nothing spans z in [-0.57, -0.55] (0.02 m)
OK removepart: removed id=4 (BodyTube)
OK checkdesign: 3 parts, contiguous, no overlaps

```

Listing 23. Build-phase output (path elided to fit the line). Ids are handed out by the `Part` constructor in creation order; the boattail’s id 4 is visible again in `removepart 4`.

newdesign. The command (`cli/Repl.cpp:853`) parses tokens, calls `makePart(type, params)` (`core/model/parts/PartFactory.cpp:27`) — which throws with a named missing field or an unknown type, surfaced verbatim as the `ERR` line — then captures the part’s id *before* ownership moves and installs the tree root: `installDesign(model::PartNode::make(std::move(root)))` (`cli/Repl.cpp:880`). `RocketModel::installDesign` (`core/model/RocketModel.cpp:118`) delegates to `PartsModel::installRoot`, which fires the `Reset` event pair around the atomic root replacement (Section 4). The REPL also drops its staged `motorSet` flag — the old tree, motor node and all, is gone, so the staged launch configuration must not pretend otherwise. The `OK` line reports the root’s id.

addpart. The command (`cli/Repl.cpp:894`) resolves its parent token in three ways: the literal `root`, a numeric id, or a part name looked up via `parts().findByName` (`core/model/PartsModel.cpp:257`) — a pre-order first-match walk, because names need not be unique and ids are the real identity (`core/model/PartsModel.h:178`).

Names are the scriptable handle; ids are session currency

Part ids are assigned by a process-wide counter and are never persisted, so a design reloaded from disk carries *fresh* ids (Section 9.6 demonstrates this: the same four parts return as 6–9). A saved script that said `addpart 2 . . .` would silently target the wrong node in the next session. Scripts therefore address parents by name — stable because the author chose it and the serializer round-trips it — while ids remain the precise, collision-free handle for interactive surgery within one session, echoed in every `OK` line so the user always has both.

The placement tokens then become a `StationLink` in `linkFromTokens` (`cli/Repl.cpp:226`): the seat name is resolved through the shared `seatKindFromString` table (`core/model/parts/Placement.h:42`, fail-closed on unknown names), the link starts from that verb’s canonical station pair — `abut(0)`, `nestInBore(0)`, or `seatOnWall(0)` (`core/model/parts/Placement.h:134`) — and any explicit `parentStation/childStation/gap` tokens override individual fields. That is why the fin set command needs only `seat=OnSurface`: the verb’s canonical pair already reads “child aft plane on the parent wall at the parent’s aft station,” which is exactly where fins belong. The one semantic guard lives here too: a `NestInBore` gap is an insertion depth and must be non-negative, rejected before the model ever sees it.

With a parent id, a fresh leaf from the factory, and a link, the command becomes a single model verb: `rocket->parts().attach(parentId, std::move(child), link)`. The result is a `std::expected<Part::Id, AttachError>`, and the REPL maps each typed error to an exact `ERR` string (Listing 24).

```
const auto attached = rocket->parts().attach(parentId, std::move(child), link);
if(!attached)
{
    switch(attached.error())
    {
        case model::PartsModel::AttachError::NoSuchParent:
            out << "ERR addpart: no part with id " << parentId << "\n"; break;
        case model::PartsModel::AttachError::DuplicateMotor:
            out << "ERR addpart: the design already has a motor\n"; break;
        case model::PartsModel::AttachError::NullPart:
            out << "ERR addpart: invalid part\n"; break;
    }
    return true;
}
out << "OK addpart: " << type << " id=" << *attached << " under " << parentId << "\n";
```

Listing 24. Typed error mapping in `addpart` (`cli/Repl.cpp:952`). The switch covers `AttachError` exhaustively with no default, so a new failure mode added to the model cannot be silently dropped by the CLI — the build warns until this mapping learns the new arm.

The `DuplicateMotor` arm deserves a note on reachability. `attachSubtree` enforces the single-motor invariant by scanning any incoming sub-tree for a `Motor` (`core/model/PartsModel.cpp:319`), but the CLI's factory refuses to construct one (`core/model/parts/PartFactory.cpp:73`) — a motor enters a design only through `setmotor`. The arm is therefore defense in depth at this call site: it exists because the mapping is total over the enum, and totality, not present-day reachability, is what keeps the CLI honest as the model grows.

Figure 12 traces one `addpart` across the three layers, including the event pair that fires around the mutation. `PartsModel` emits `Event{Attached, id, parent, row}` twice — before and after the structural write — and `RocketModel`'s constructor-wired bridge (`core/model/RocketModel.cpp:20`) forwards the pair to any granular consumer and folds the completed edit into the coarse *structure changed* callback. In a GUI session those become Qt's `beginInsertRows/endInsertRows` bracket and a mass/CG readout refresh (Section 11); in a CLI session no consumer ever registers, both callbacks are null, and the emit sites cost two skipped function-object checks. The model does not know or care which kind of session it is in — that indifference is the point of routing every mutation through the verbs.

9.4 Inspecting the design: `listparts` and `checkdesign`

`listparts` (`cli/Repl.cpp:1021`) is the tree made visible: `forEachNode` (`core/model/PartsModel.h:186`) walks pre-order, parent before child in attachment order, and `printPartTree` (`cli/Repl.cpp:251`) indents by the depth the walk hands it — id, type, name, and own mass at $t = 0$ per line. The trailing composite line reads `compositeMass(0.0)` and `compositeCm(0.0).z()` from the root node: the same gated accessors the flight loop uses (Section 5), so what the designer reads on screen is by construction what the integrator will fly.

`checkdesign` (`cli/Repl.cpp:969`) layers two verdicts with deliberately different vantage points:

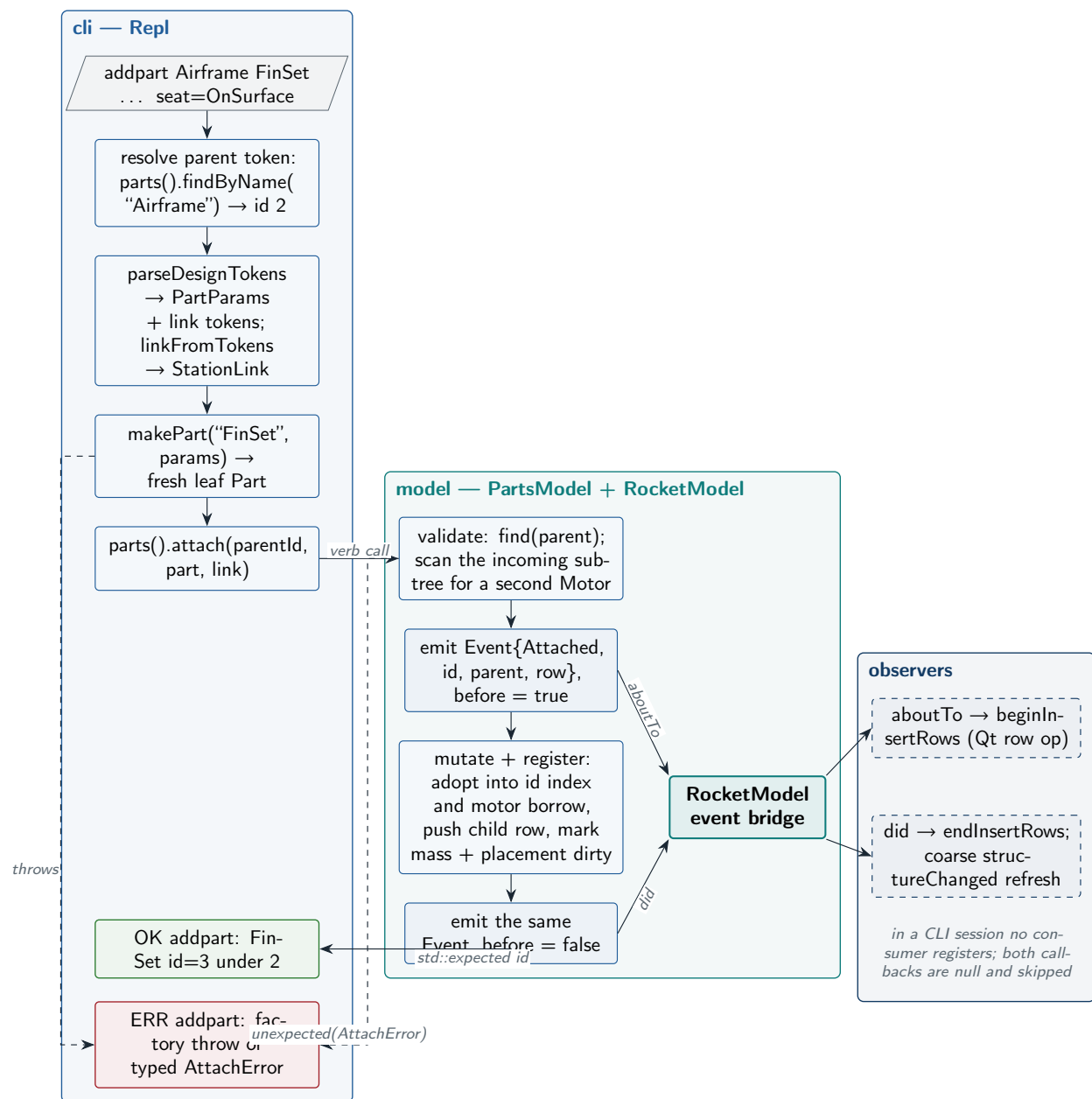


Figure 12. One `addpart`, end to end. The REPL resolves the parent, parses geometry and link tokens, builds the leaf, and calls the single model verb; `PartsModel` validates, brackets the mutation in an `aboutTo/did` event pair, and returns the new id as a `std::expected`; observers (dashed — present only in a GUI session) receive the pair via the `RocketModel` bridge.

- **The model’s overlap verdict.** `placementDiagnostics()` returns the cached envelope-sweep result (Section 6): radial interpenetration — a part poking through a wall that cannot contain it — located by id and station. It is resolved once per structural change and shared by every consumer.
- **The CLI’s axial-coverage scan.** On top of `resolvedPlacements()` the command builds one $[z_{\text{aft}}, z_{\text{fore}}]$ span per part (`cli/Repl.cpp:984`), sorts by aft edge, and walks a running coverage front; any span that starts more than 10^{-9} m beyond the front is reported as an air gap. Zero-length parts — a `Motor` node — are excluded, since a point cannot bound coverage.

Each layer sees what the other cannot

The sweep is intent-blind by design: a positive standoff is a perfectly legal `StationLink` — the model must allow a floating part mid-edit — so parts hanging in space are invisible to an overlap check. The coverage scan sees exactly that omission, but knows nothing of radii: a nested coupler and a tube-through-a-nose-cone look identical to it. Only their conjunction reads “physically sensible rocket,” which is why the gap scan lives in the CLI as a design-review lint rather than in the model as an invariant — a gap is suspicious in a *finished* design, not illegal in a design under construction.

The transcript shows the scan working. The boattail is attached with `gap=-0.02` — an **Abut** gap is a signed forward standoff, so `-0.02` retreats the part 2 cm aft of the airframe’s aft plane at $z = -0.55$ — and `checkdesign` answers with the located interval: `nothing spans z in [-0.57, -0.55]`. After `removepart 4` the design is contiguous again. `removepart` itself (`cli/Repl.cpp:1035`) guards the root by id before calling the model — `PartsModel::detach` would refuse with the typed `IsRoot` anyway, but the CLI pre-empts it to say *what to do instead* (use `newdesign` or `cleardesign`) — and afterwards re-reads `rocket->isMotorSet()`, because the detached sub-tree may have carried the motor node with it and the staged `motorSet` flag must follow the tree, not the user’s memory of it.

9.5 setmotor: the database meets the tree

`setmotor C6` (`cli/Repl.cpp:539`) looks the common name up in the `MotorModelDatabase`, and hands the found `MotorModel` to `RocketModel::setMotorModel`, which delegates to `PartsModel::setMotor` (`core/model/PartsModel.cpp:470`). On a design with no motor this *attaches* a `Motor` leaf under the root through the ordinary `attach` verb — default **Abut** link, ordinary events, ordinary cache dirtying; a later `setmotor` swaps the wrapped `MotorModel` in place, keeping the node, its id, and its link (Section 7). The effect is immediately visible in the tree (Listing 25):

```
OK setmotor: C6
OK listparts:
  [1] NoseCone "Nose"  m=0.019658 kg
  [2] BodyTube "Airframe" m=0.0355691 kg
  [3] FinSet "Fins"    m=0.01836 kg
  [5] Motor "Motor"    m=0.0241 kg
-- composite: mass=0.0976871 kg, cg_z=-0.255415 m (t=0)
```

Listing 25. The motor is a tree citizen: node 5 appears as a child of the root, its 24.1 g loaded weight joins the composite, and the CG moves aft accordingly — no separate motor mass bookkeeping anywhere.

9.6 Persistence and the no-design contract

```
OK savedesign: demo.qrd
OK cleardesign:
ERR listparts: no design
OK loaddesign: demo.qrd (motor C6)
OK listparts:
  [6] NoseCone "Nose"  m=0.019658 kg
  [7] BodyTube "Airframe" m=0.0355691 kg
  [8] FinSet "Fins"    m=0.01836 kg
  [9] Motor "Motor"    m=0.0241 kg
-- composite: mass=0.0976871 kg, cg_z=-0.255415 m (t=0)
```

Listing 26. The round trip. `cleardesign` leaves a genuinely empty model — not a placeholder — so the next `listparts` fails the `hasDesign` contract. On reload the identical physics returns under fresh ids 6–9, and the motor is re-resolved by common name.

`savedesign` and `loaddesign` (`cli/Repl.cpp:1067`) are one-call wrappers over the `DesignSerializer` of [Section 8](#); serializer exceptions surface as `ERR` lines with the underlying message. `cleardesign` calls `RocketModel::clearDesign` — `installDesign(nullptr)` — after which `hasDesign()` is false and every design command (`listparts`, `checkdesign`, `addpart`, ...) answers `ERR . . . : no design` rather than inventing an empty-tree interpretation. “No design” is a representable, first-class state (`core/model/PartsModel.h:160`), and the CLI’s contract makes it audible.

The reload output ([Listing 26](#)) is the evidence behind the name-vs-id rule of [Section 9.3](#): masses, CG, and structure are bit-identical, but the ids are new. `loaddesign` finishes by syncing the REPL’s staged configuration *from* the loaded rocket (`cli/Repl.cpp:1107`) — drag coefficient, reference area, and the motor flag — because the serializer re-attached the motor by common name from the session database (`core/model/DesignSerializer.cpp:244`) and the staging must reflect what actually loaded, including the degraded no-motor case when the database lacks the name.

9.7 The launch gate

```
ERR launch: no motor set (use loadmotors + setmotor first)
...
OK setvelocity: 15 m/s
OK launch: 1290 steps, t_final=12.890s
  apogee    = 170.855 m @ t=5.590s
  max_speed = 61.457 m/s @ t=1.820s
  downrange = 0.000 m
  landing   = t=12.890s, pos=(0.000, 0.000, -0.056)
CSV /tmp/qtrocket-demo/qtrocket_run.csv
OK bye
```

Listing 27. The gate and the flight. The motorless `launch` (issued mid-session, before `setmotor`) is refused with the remedy in the message; the flight summary reads the statistics tracked live during propagation and names the CSV written beside the session.

`launch` (`cli/Repl.cpp:740`) opens with the one precondition the CLI stages itself: `motorSet` ([Listing 27](#)). The flag is raised by `setmotor`, dropped by `newdesign` and `cleardesign`, and re-derived from the model after `removepart` and `loaddesign` — every path that can change whether a motor exists

converges on it, so the gate is a single boolean read. A motorless launch is refused before any state is touched.

The initial condition matters more than it looks. `setvelocity` and `setangle` stage a rail-exit state — $v_x = v \sin \theta$, $v_z = v \cos \theta$ from vertical (`cli/Repl.cpp:749`) — rather than defaulting every flight to a dead start, because a thrust curve begins with an implicit (0, 0) sample: over the first integration step a from-rest rocket has essentially no thrust. The propagator owns the pathological end of this — it holds the vehicle at the pad until a step genuinely lifts it (`core/sim/Propagator.cpp:111`) and aborts with a located `NoLiftoff` verdict if 3 s pass below 1 m (`core/sim/Propagator.h:26`) — but a launch rail is also just the true physics: real models leave the rod at speed, and a 15 m/s exit starts the integration past the thrust hole on the part of the curve that actually flies the vehicle. When a run does end abnormally, the CLI prints a `WARN` line built from the typed termination reason; the mapping switch has no default case (`cli/Repl.cpp:43`), so a new `TerminationReason` refuses to compile until the CLI can explain it.

One last protocol note closes the loop: this session’s process exit code is 1. Three `ERR` lines were provoked on purpose — the air gap, the empty-model `listparts`, the motorless `launch` — and the chokepoint of [Section 9.1](#) counts them all the same. The protocol offers no “expected failure” escape hatch by design: a script that treats an `ERR` as part of its happy path documents that fact in its own exit-status handling, and a CI pipeline that pipes a design script through `qtrocket-cli` gets a conservative, zero-parsing pass/fail signal for free.

10 Execution walkthrough: a simulated flight

Every preceding section examined the part tree at rest — how it is owned ([Section 4](#)), how its composites are cached ([Section 5](#)), how its geometry is resolved ([Section 6](#)). This section runs it, following a single flight end to end — from `QtRocket::launchRocket()` through the integrator’s innermost stage down to the part tree and back out to the CSV on disk — and counting every call the simulation makes into the model along the way. The point of the walkthrough is a claim the earlier sections could only assert: a tree built for interactive editing — ids, events, placement links, envelope sweeps — serves an adaptive numerical integrator’s hot loop while touching, per stage, nothing but one thrust-curve lookup and two recursive additions.

The trace below uses the Runge-Kutta-Fehlberg (RKF45) stepper as the illustrative case because it is the demanding one: six stages per attempted step, evaluated at fractional node times the step controller chooses and sometimes discards. The fixed-step RK4 backend is the configured default (`core/sim/Propagator.cpp:48`); either is selectable at runtime through the same `Integrator` facade (`core/sim/Integrator.h`), and everything said here holds for both — RK4 simply asks four times per step instead of six per attempt.

10.1 Ignition

A flight begins at the controller singleton. `QtRocket::launchRocket()` (`core/QtRocket.cpp:37`) does four things in order: it clears the previous run’s state history, rewinds the propagator clock to $t = 0$, tells the rocket to `launch()`, and hands control to `Propagator::runUntilTerminate()`. The model-side half of that handshake is two lines ([Listing 28](#)): copy the caller-supplied initial state into the current state, and ignite the motor.

Ignition is deliberately a *routed* mutation. The wrapped `MotorModel` lives inside the tree’s single `Motor` node ([Section 7](#)), and the only code allowed to flip its ignition latch is `PartsModel::startMotor` — the friend seam through which all motor runtime state flows. Even the simulation obeys the mutation-routing discipline of [Section 2](#): `parts_` is asked, the typed borrow `motor_` is dereferenced


```

void RocketModel::launch()
{
    setCurrentState(initialState);
    parts_.startMotor(0.0);
}
// ...
void PartsModel::startMotor(double t)
{
    if(motor_ != nullptr)
    {
        motor_>mm.startMotor(t); // friend seam: ignition is a routed runtime mutation
    }
}
// ...
void startMotor(double startTime) { ignitionOccurred = true; ignitionTime = startTime; }

```

Listing 28. The ignition chain: `core/model/RocketModel.cpp:101` → `core/model/PartsModel.cpp:511` → `core/model/MotorModel.h:330`. The simulation never holds a motor pointer; it asks the tree.

inside the model, and a design with no motor degrades to a no-op rather than a null dereference. From this point on the motor’s mass begins falling and its thrust curve is live; everything else about the rocket — every pose, every seam, every link — is already fixed and will not move again until the run returns.

10.2 The loop and its stages

`runUntilTerminate()` (`core/sim/Propagator.cpp:58`) is a synchronous loop on the caller’s thread: read the current state, take one integrator step, vet the result, record it, check for termination, advance t by the step the integrator actually took (`core/sim/Propagator.cpp:166`) — constant for RK4, adaptive for RKF45. The physics enters through a single closure, installed once at construction ([Listing 29](#)): the coupled linear ODEs $\dot{\mathbf{r}} = \mathbf{v}$, $\dot{\mathbf{v}} = \mathbf{F}/m$.

```

std::function<std::pair<Vector3, Vector3>(double, Vector3&, Vector3&)> linearODEs =
    [this](double t, Vector3& state, Vector3& rate) -> std::pair<Vector3, Vector3>
{
    Vector3 dPosition;
    Vector3 dVelocity;
    dPosition = rate;
    dVelocity = object->getForces(t, state, rate, *environment) / object->getMass(t);

    return std::make_pair(dPosition, dVelocity);
};

```

Listing 29. The ODE right-hand side (`core/sim/Propagator.cpp:36`). `object` is the rocket behind the `Propagatable` interface — the sim never sees a part, a node, or a motor.

The RKF45 stepper (`core/sim/RK45Solver.h:92`) evaluates this closure six times per attempted step, each stage at its Fehlberg node time $t + c_i h$ on a trial state assembled from the previous stages. Sampling at the true node times is what lets a thrust transient register in the embedded error estimate: across a sharp burn the fourth- and fifth-order solutions diverge, the error rises, and the controller shrinks h to resolve the transient. When the estimate exceeds tolerance the whole six-stage attempt

is discarded and re-run at a smaller h ; when it cannot shrink further the stepper throws, and the propagator’s enclosing `try` converts the exception into a reported termination rather than a crash (`core/sim/Propagator.cpp:81`). Figure 13 lays out the resulting per-stage and per-step data flow across the three layers.

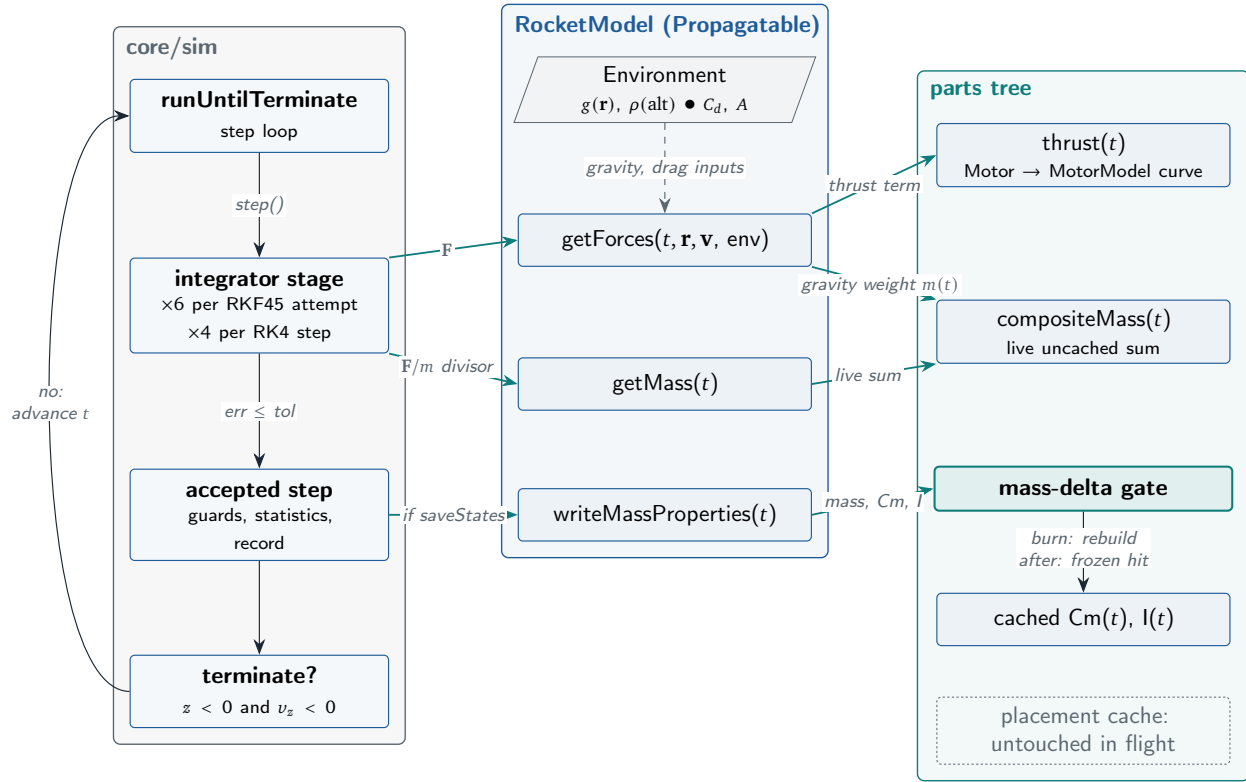


Figure 13. Data flow of one integration step. Stage traffic (upper path, six times per RKF45 attempt) reaches the parts tree only through the motor’s thrust curve and the live `compositeMass` sum. The accepted-step record (lower path, once per step) is the only path through the mass-delta gate; the placement cache is never consulted in flight.

10.3 One stage, itemized

Each stage evaluation costs the model exactly two calls: `getForces` and `getMass`. Listing 30 is the entire force model.

Three terms, three provenances:

- **Thrust** is `parts_.thrust(t)` (`core/model/PartsModel.cpp:505`): a null-check on the motor borrow, then `MotorModel::getThrust` (`core/model/MotorModel.cpp:69`) — linear interpolation over the catalog thrust samples, zero after burnout, `const` end to end. A motorless tree reads as zero thrust, not an error; the launch front ends refuse earlier (`cli/Repl.cpp:744`, Section 9).
- **Gravity** is the environment’s pluggable model evaluated at the *trial* position — not `currentState`, so every stage sees a self-consistent state — weighted by `getMass(t)`, which is nothing but `parts_.root()->compositeMass(t)` (`core/model/RocketModel.cpp:33`).
- **Drag** is $-\frac{1}{2}\rho(\text{alt})\|\mathbf{v}\|C_dA\mathbf{v}$, written with $\|\mathbf{v}\|\mathbf{v}$ rather than $v^2\hat{\mathbf{v}}$ so a zero-velocity stage costs no division. The altitude clamp exists because RKF45 trial states legitimately probe below $z = 0$ near the ground, outside the atmosphere models’ domain. A is the model’s stored reference

```

Vector3 RocketModel::getForces(double t, const Vector3& position,
                               const Vector3& velocity, sim::Environment& environment)
{
    // Thrust along the rocket's z-axis, assumed through the CM.
    Vector3 forces{0.0, 0.0, parts_.thrust(t)};

    auto gravityModel = environment.getGravityModel();
    Vector3 gravity = gravityModel->getAccel(position)*getMass(t);
    forces += gravity;

    auto atmosphere = environment.getAtmosphericModel();
    // Clamp altitude to >= 0: trial states crossing z=0 fall outside the atmosphere models' domain.
    const double altitude = position[2] > 0.0 ? position[2] : 0.0;
    const double rho = atmosphere->getDensity(altitude);
    const double speed = velocity.norm();
    const Vector3 drag = -0.5 * rho * speed * dragCoefficient * referenceArea * velocity;
    forces += drag;

    return forces;
}

```

Listing 30. The 3-DOF force model (`core/model/RocketModel.cpp:64`, comments elided). Thrust and mass come from the part tree; gravity and density from the injected environment; drag configuration from the model's own scalars.

area: an explicit `setReferenceArea()` marks it a manual override (`core/model/RocketModel.h:69`), `installDesign()` clears that flag so a later save does not record the old airframe's area as authored intent (`core/model/RocketModel.cpp:121`); the geometry-derived value — the largest single frontal disc, `maxFrontalReferenceArea()` over the same tree — is exposed to front ends through `deriveReferenceAreaFromGeometry()` (`core/model/RocketModel.cpp:44`, [Section 5](#)), but the force model always flies the stored scalar.

Per stage, then, the parts tree is touched exactly three times: one thrust-curve lookup and two live `compositeMass` sums — the gravity weight inside `getForces` and the F/m divisor in the ODE closure. Neither motor query does real work at stage cadence: thrust interpolates the sample table, and in-burn mass interpolates a mass curve precomputed at load time (`core/model/MotorModel.cpp:132`), so both are table walks, not quadrature. The live sum itself is N double additions over a tree whose N is tens — deliberately uncached, because it is simultaneously the cheapest quantity in the system and the key that keeps the composite cache honest (the full argument is [Section 5](#)). [Table 4](#) totals the traffic.

The absolute numbers make the “cheap” claim concrete. At the default 0.01 s step (`core/sim/Propagator.h:110`) a ten-second model-rocket flight is on the order of a thousand accepted steps: under RK4 that is 4,000 stage evaluations, hence 8,000 live sums and 4,000 curve lookups. For a twenty-part tree a live sum is twenty additions, so the tree's entire stage-cadence contribution over the *whole flight* is a few hundred thousand floating-point additions — noise against the Eigen vector algebra of the stages themselves. The expensive machinery runs elsewhere, and less often.

10.4 The accepted step: recording through the gate

When an attempt passes the error test, the propagator vets the advanced state (the non-finite and launch-pad guards of [Section 10.5](#)), commits it with `setCurrentState`, folds it into the running

Tree traffic	Per RKF45 attempt (6 stages)	Per RK4 step (4 stages)
Thrust-curve lookup	6	4
Live <code>compositeMass</code> sum (stages)	12 (2 per stage)	8
Live sums in the step record	3 (snapshot + one gate key per gated read)	3
Gated CM/tensor rebuild	≤ 1 while the motor burns; 0 after burnout	same
Placement resolve / envelope sweep	0	0

Table 4. What one integration step asks of the part tree. A rejected RKF45 attempt repeats its six-stage column at a smaller h ; the record column runs once per accepted, recorded step.

trajectory statistics, and — when state saving is on — records it (`core/sim/Propagator.cpp:132`):

```
object->writeMassProperties(currentTime, nextState);
object->appendState(currentTime, nextState);
```

`writeMassProperties` (`core/model/RocketModel.cpp:49`) snapshots `compositeMass`, `compositeCm`, and `compositeI` at the accepted time into the `StateData` sample, so every row of the state history carries a mutually consistent mass/CG/tensor triple. This is the flight’s only path through the mass-delta gate of [Section 5](#), and the gate’s two regimes are visible in the cadence. While the motor burns, the live sum differs from the stamp at every accepted step — mass genuinely moved — so the gate fires and the two-pass CM/tensor walk runs: once per recorded step, roughly 150 rebuilds across a 1.5-second burn. After burnout, `MotorModel::getMass` returns the casing mass verbatim for every t , the recursive sum reproduces bit for bit, `exact ==` holds, and every read for the rest of the flight — coast, apogee, descent, the majority of the samples — is a frozen-cache hit. Zero rebuilds, by construction rather than by tolerance tuning.

Just as telling is what *never* runs. The placement machinery of [Section 6](#) — seat resolution, pose accumulation, the $O(N \log N)$ envelope sweep — is keyed on structure, and a flight changes no structure: geometry is invariant under mass loss. Each gated rebuild does enter `ensurePlacementCache` (`core/model/PartsModel.cpp:144`), but finds `placementDirty_` false and returns after a single boolean test. The resolve count for an entire flight is zero; the integrator re-weights, thousands of times, a geometry that was solved once at edit time.

The flight owns the tree

`runUntilTerminate()` is synchronous on its caller’s thread, so the thread that could edit the tree is busy flying it, and the single-writer contract ([Section 2](#)) forbids any other thread from touching the model at all — even for `const` reads, since any composite read may be the one that rebuilds. This is not a grudging limitation; it is what excludes an otherwise ugly class of states. A structural edit landing between stages three and four of one attempt would blend two different rockets into a single Butcher tableau; it would raise both dirty chains, so the next recorded step re-resolves geometry mid-run and stamps the history with a rocket the first half of the flight never was; and it could flip the overlap verdict, turning a flying design into a throwing one between two samples. Rather than defending every read against a caller the architecture never creates, the contract makes the whole class unrepresentable: edits happen between flights, and a flight sees one rocket from ignition to touchdown.

A self-intersecting design refuses to fly

If the design's cached overlap verdict is a failure, the first composite read — `writeMassProperties` at the first recorded step, if no editor read composites earlier — throws `std::runtime_error` from `computeCompositeAt` carrying the first located diagnostic (`core/model/PartsModel.cpp:184`): which parts interpenetrate, and where. The propagator's enclosing `catch` (`core/sim/Propagator.cpp:169`) turns the throw into a terminated run with `TerminationReason::IntegratorError` and the message in the log — a located refusal, not a trajectory quietly integrated over a physically meaningless inertia tensor. Note the stage path cannot throw this: stages touch only the live mass sum, which never consults the verdict, so the refusal surfaces exactly at the first attempt to *record* the bad rocket. The full throw-and-rethrow mechanics are in [Section 5](#).

10.5 Termination and the flight record

The nominal end of flight is the model's own verdict (`core/model/RocketModel.cpp:58`): the rocket is below the launch site and descending, $z < 0$ and $v_z < 0$. Requiring both keeps an arcing trajectory that crosses $z = 0$ upward, or a pad-level start, from reading as a landing. Around that nominal check the loop carries four safety guards, each converting a would-be infinite loop or silent lie into a reportable `TerminationReason` (`core/sim/Propagator.h`):

- **Pad hold.** A thrust curve begins with an implicit (0, 0) sample, so over the first step thrust is nearly zero and a from-rest launch would sink below the pad and trip the nominal terminate after one step. Until the rocket first achieves $z > 0$ under power, the loop clamps it to the pad at rest (`core/sim/Propagator.cpp:111`) — the pad's normal force, modeled as a clamp.
- **NoLiftoff.** If after 3 s the flight has never climbed past 1 m, the run aborts with a thrust-versus-weight warning rather than holding the clamp forever.
- **NonFiniteState.** A NaN or Inf coordinate fails every comparison — $z < 0$ would never trip — so a non-finite step aborts immediately and is kept out of the recorded history (`core/sim/Propagator.cpp:97`).
- **Backstops.** A sim-time cap and an iteration cap catch a flight that never descends or collapses into vanishingly small steps; the `catch` that wraps the loop reports any integrator throw as `IntegratorError`.

What remains when the loop exits is the flight record, in two grains. The `TrajectoryStatistics` fold (`core/sim/TrajectoryStatistics.h`) — apogee and its time, maximum speed and its time — is updated on *every* accepted step, independent of state saving, so the summary and the no-liftoff guard work even for runs that retain no history. The state history itself is the vector of $(t, \text{StateData})$ samples, each carrying position, velocity, and the gate-consistent mass/CG/inertia triple recorded in [Section 10.4](#). Both surface through the controller (`core/QtRocket.h:47`): `getStates()`, `getTrajectoryStatistics()`, and `getTerminationReason()`. The CLI's `launch` command ([Section 9](#)) prints the summary from the statistics, warns on any non-nominal reason, and streams the full series to `qtrocket_run.csv` (`cli/Repl.cpp:785`) — one row per sample, `t,x,y,z,vx,vy,vz,mass,cg_x,cg_y,cg_z,Ixx,Iyy,Izz` — which is the artifact the test matrix regressions and any external analysis consume.

The walkthrough closes where the design intends: the part tree that a user edits by name and verb in [Section 9](#), and that the GUI mirrors row for row in [Section 11](#), is the same object the integrator interrogated several thousand times above — through two interface methods, three calls per stage, one gated rebuild per burning step, and zero geometry work.

11 GUI integration: the Qt item model over the event stream

The GUI is the one Qt-dependent layer in the program, and the part tree it displays lives in a layer that is deliberately Qt-free ([Section 2](#)). The bridge between the two is `RocketPartModel` (`gui/RocketTreeView.h:17`), a `QAbstractItemModel` that adapts the `PartsModel` of [Section 4](#) to Qt’s model/view protocol. The adapter is deliberately stateless: its only member is a borrowed `const model::PartsModel*`, so there is no shadow copy of the tree that could drift out of sync with the authority. Structure queries (`index`, `parent`, `rowCount`, `data`) derive every answer live from the owned tree, and change notification arrives over the same typed `aboutTo/did` event stream (`core/model/PartsModel.h:143`) that every mutation verb narrates. `RocketTreeView`, a `QTreeView` that owns its `RocketPartModel`, completes the picture: it binds the adapter to a rocket and registers the event forwarding.

The section covers the three design commitments in turn — id-based index identity, live derivation of structure, and surgical translation of events into Qt row operations — then the subscription chain that delivers the events, the routed-edit pattern that lets any widget mutate the tree without knowing the view exists, and the headless test binary that pins the `QAbstractItemModel` contract. [Figure 14](#) shows the participants.

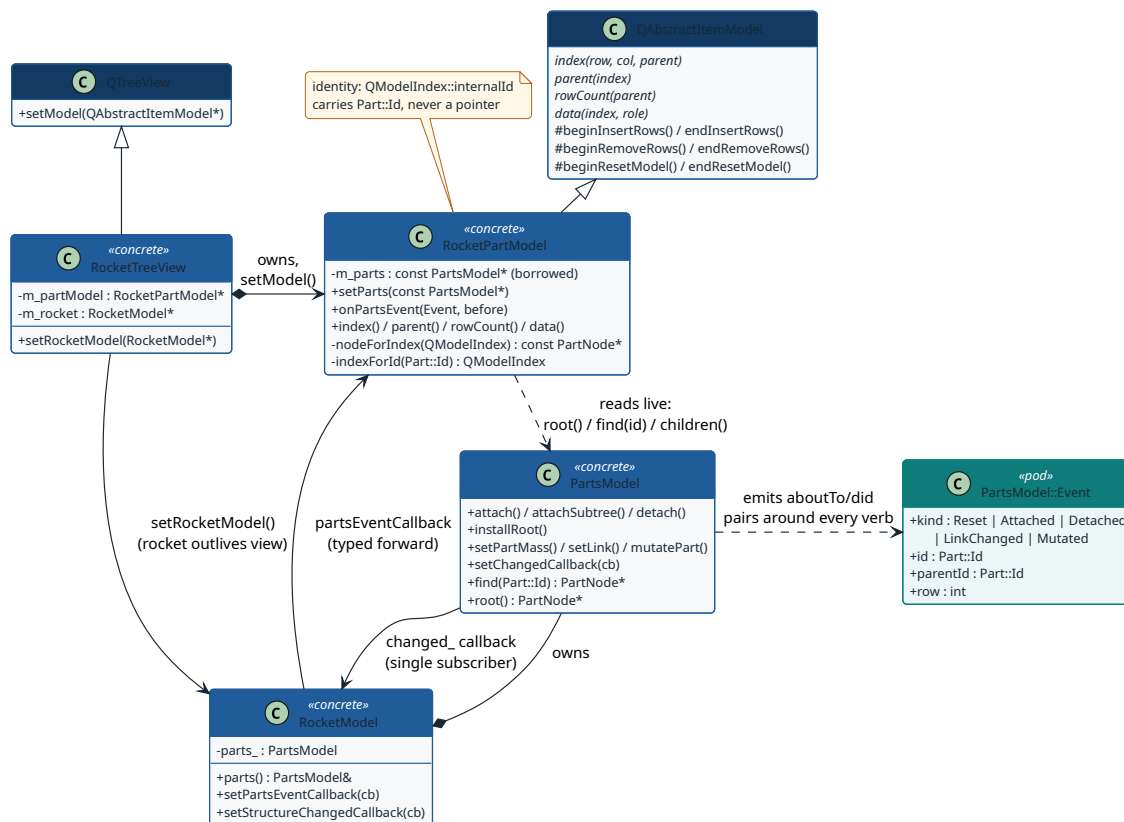


Figure 14. The GUI adapter stack. Plain boxes are Qt classes. `RocketPartModel` holds only a borrowed pointer to the `PartsModel` and derives all structure live; change notification flows `PartsModel` → `RocketModel` (the single subscriber) → `RocketPartModel` as typed `aboutTo/did` pairs.

11.1 Index identity: `internalId` carries `Part::Id`

Every `QModelIndex` needs a payload that identifies the row’s node across the index’s lifetime. `RocketPartModel` stores the node’s `Part::Id` in `QModelIndex::internalId` — a stable integer — and resolves it back to a node through the tree’s $O(1)$ id index on every access (Listing 31).

```
const PartNode* RocketPartModel::nodeForIndex(const QModelIndex& index) const
{
    if (!m_parts)
        return nullptr;
    if (!index.isValid())
        return m_parts->root();
    return m_parts->find(static_cast<model::part::Part::Id>(index.internalId()));
}

QModelIndex RocketPartModel::indexForId(model::part::Part::Id id) const
{
    const PartNode* node = m_parts ? m_parts->find(id) : nullptr;
    if (!node)
        return {};
    return createIndex(node->rowInParent(), 0, static_cast<quintptr>(id));
}
```

Listing 31. Both directions of the identity scheme (`gui/RocketTreeView.cpp:134`): an index resolves to a node by id lookup, and an event’s node id resolves to the column-0 index Qt’s row operations need.

A stale index fails a lookup, not a dereference

Views hold indexes across arbitrary user actions, so some of them *will* outlive the nodes they name. With an id payload, a stale index simply fails the `find()` (`gui/RocketTreeView.cpp:140`) and the accessor returns null data — `data()` yields an empty `QVariant`, `rowCount()` yields zero. And because ids are minted from a process-wide atomic counter and never reused — every `Part` construction, including the copy constructor behind `clone()`, draws a fresh id (`core/model/parts/Part.cpp:16`) — a stale id can never alias a *different* live row. Failure degrades to blankness, never to misdirection.

Rejected — a node pointer in `internalPointer`

Qt’s conventional payload is a `void*` to the backing item. Here that pointer would be a `PartNode*` into a tree whose nodes are destroyed by `detach` and `installRoot` (Section 4); any index surviving such an edit would dereference freed memory on its next paint. The id costs one hash lookup per access — negligible at tens of parts — and buys memory-safety by construction rather than by careful invalidation.

11.2 Structure derived live

The four structure overrides never consult stored rows; they walk the authoritative tree at call time. `rowCount` on the invisible root returns 1 when a design is loaded and 0 otherwise — “no design” renders as an empty view, the same first-class state the rest of the program represents (Section 4) — and on any other index returns the node’s `children().size()`. Children hang only off column 0, per Qt convention (`gui/RocketTreeView.cpp:89`). `index(row, col, parent)` reads the child span of the

parent node; `parent(index)` follows the node’s parent pointer and derives the row with `rowInParent()`, a linear scan over the sibling span (`core/model/PartsModel.cpp:55`) that is trivially cheap at realistic part counts. Because rows are always computed, there is no bookkeeping to fall out of step when the tree changes.

`data()` exposes three display columns: **Name** (the part’s human-facing name), **Type** (its `typeName()`), and **Mass** — the part’s own mass at $t = 0$ in kg (`gui/RocketTreeView.cpp:115`), not the sub-tree composite. The tree view answers “what is this component”; the gated composites of [Section 5](#) answer “what does the rocket weigh” and belong to the simulation path, not a per-paint widget query.

11.3 Consuming the aboutTo/did stream

Qt’s row-op protocol is a bracketing protocol

`QAbstractItemModel` requires each structural change to be announced *before* it happens — `beginInsertRows`/`beginRemoveRows` with the exact parent index and row range, while the model still answers queries about the pre-edit tree — and confirmed with the matching `end` call after the change lands. The event stream fires as `aboutTo`/`did` pairs around every mutation verb for precisely this reason: the `aboutTo` edge arrives while the tree is still un-mutated (so `indexForId(parentId)` and the carried row resolve against the old structure), and the `did` edge arrives once the write is complete. The event shape is Qt’s contract, expressed without Qt.

Event kind	aboutTo edge (before)	did edge
Reset	<code>beginResetModel()</code>	<code>endResetModel()</code> ; view re-expands
Attached	<code>beginInsertRows(parentIdx, row, row)</code>	<code>endInsertRows()</code>
Detached	<code>beginRemoveRows(parentIdx, row, row)</code>	<code>endRemoveRows()</code>
Mutated	— (no rows move)	<code>dataChanged</code> across the node’s row
LinkChanged	— (no rows move)	<code>dataChanged</code> across the node’s row

Table 5. Event kind $\times$ edge $\rightarrow$ Qt call. `parentIdx` is `indexForId(e.parentId)`, resolved on the `aboutTo` edge against the not-yet-mutated tree.

Three details of the mapping ([Listing 32](#), [Table 5](#)) are load-bearing. First, the event payload is a *location*, not a data snapshot: `{kind, id, parentId, row}` is exactly what `beginInsertRows`/`beginRemoveRows` require, and the emitting verbs compute the row from the live tree at emission time — an attach appends, so its event carries the parent’s current child count as the insertion row (`core/model/PartsModel.cpp:339`); a detach carries the victim’s `rowInParent()` before removal. Second, a sub-tree attach is a *single* insert of its root row: the event names only the sub-tree root, and Qt discovers descendants lazily through `rowCount/index`, so the payload never needs to enumerate them. Third, `Mutated` and `LinkChanged` act only on the `did` edge — an in-place edit moves no rows, so there is nothing to bracket, and `dataChanged` is by design an after-the-fact notification. The emitted range spans all columns of the row because a routed edit may change any displayed cell, and the indexes are built directly from the payload (`createIndex(e.row, ., e.id)`) with no lookup at all.

11.4 What granularity buys

Because ordinary edits become surgical row operations, everything the user has invested in the view survives them: `QPersistentModelIndexes` (and therefore selection) survive inserts and removes by

```

void RocketPartModel::onPartsEvent(const model::PartsModel::Event& e, bool before)
{
    using Event = model::PartsModel::Event;
    switch (e.kind)
    {
        case Event::Reset:
            if (before) { beginResetModel(); } else { endResetModel(); }
            break;
        case Event::Attached:
            if (before) { beginInsertRows(indexForId(e.parentId), e.row, e.row); }
            else { endInsertRows(); }
            break;
        case Event::Detached:
            if (before) { beginRemoveRows(indexForId(e.parentId), e.row, e.row); }
            else { endRemoveRows(); }
            break;
        case Event::Mutated:
        case Event::LinkChanged:
            // In-place edit: no rows move; refresh the affected row's cells
            // once the write landed.
            if (!before)
            {
                const QModelIndex left = createIndex(e.row, 0, static_cast<quintptr>(e.id));
                const QModelIndex right = createIndex(e.row, ColumnCount - 1,
                                                       static_cast<quintptr>(e.id));
                emit dataChanged(left, right);
            }
            break;
    }
}

```

Listing 32. The complete event consumer (`gui/RocketTreeView.cpp:21`). Each event kind maps onto exactly one Qt notification pattern; [Table 5](#) tabulates the mapping.

construction, expansion state is untouched, and the scroll position does not jump. The only event that resets the view is `Event::Reset` — a design install or clear (Section 8) — after which the view deliberately calls `expandAll()` (`gui/RocketTreeView.cpp:183`): a freshly-installed design starts fully expanded, while every smaller edit preserves whatever the user had. That re-expansion is view policy, so it lives in `RocketTreeView`’s forwarding lambda, not in the reusable adapter.

Rejected — reset on every change

A subscriber that answers every event with `begin/endResetModel` is trivially correct and a fifth the code. It loses because a reset invalidates every index: each edit would collapse the tree, drop the selection, and yank the scroll position — hostile exactly in the tight edit loops where the view matters most. The granular consumer is also the enabling layer for what comes next: drag-reparent maps onto `beginMoveRows` over the same event vocabulary, and command-pattern undo needs stable ids and surviving selection to restore context after an undone edit. Neither can be built over a reset-only subscriber.

11.5 The subscription chain and its lifetime

`PartsModel` keeps exactly one change callback (latest registration wins, `core/model/PartsModel.h:248`), and `RocketModel` claims it in its constructor (`core/model/RocketModel.cpp:20`), making itself the single subscriber on the tree it owns. The bridge then fans out over two channels: the typed stream is forwarded verbatim to whatever granular consumer registered via `setPartsEventCallback` (`core/model/RocketModel.h:101`) — in the GUI, the tree view — and every completed mutation (did edge) is folded into the coarse `setStructureChangedCallback` (`core/model/RocketModel.h:96`), a payload-less “something changed” signal for consumers that only refresh derived read-outs. Both are `std::function` seams: the model layer notifies Qt-shaped consumers without linking Qt, which is what keeps Section 9’s headless CLI and the test binaries free of any GUI dependency.

The view side of the chain is established once at startup, when `MainWindow` binds the `QtRocket` singleton’s rocket into the tree view (`gui/MainWindow.cpp:33`). `RocketTreeView::setRocketModel` (`gui/RocketTreeView.cpp:167`) seeds the adapter with the current tree (a bind is itself a reset), then registers the forwarding lambda; re-pointing to a different rocket detaches the previous registration first, and binding `nullptr` empties the view. The teardown ordering is the one lifetime subtlety: the rocket, owned by the `QtRocket` singleton, outlives the view, and the registered lambda captures `this` — so the view’s destructor clears the callback before dying (`gui/RocketTreeView.cpp:159`). Leaving it registered would leave the model holding a dangling widget pointer for the rest of the process.

11.6 Edit widgets route through the verbs

No widget edits a `Part` directly; widgets call the routed verbs of Section 4, and the event stream does the rest. `CannonballTab` is the canonical example — its trajectory calculation pushes the mass field into the design through `setPartMass` (`gui/CannonballTab.cpp:130`):

```
if(rocket->parts().hasDesign())
{
    // routed edit: invalidates the composite caches and refreshes the tree view's mass column
    rocket->parts().setPartMass(rocket->parts().root()->id(), mass);
}
```

One call buys three guarantees: the write lands through the single mutation authority, the mass cache chain is dirtied so the next simulation sees the new value (Section 5), and the `Mutated` event

refreshes the tree view’s mass column — with no coupling between the two widgets, neither of which knows the other exists. Any future property editor follows the same pattern: call a verb, let the stream propagate.

11.7 Verification: the model contract under test

The adapter has its own headless test binary, `gui_model_tests` (registered with `ctest` as `qtrocket_gui_model_tests`), which compiles `gui/RocketTreeView.cpp` against `qtrocket_core` and runs under a `QCoreApplication` — the model under test is widget-free, so the suite needs no platform plugin and runs on a displayless CI runner (`gui/tests/CMakeLists.txt:5`). The fixture wires `RocketModel` to `RocketPartModel` exactly as `RocketTreeView` does, minus the widget, and then attaches a `QAbstractItemModelTester` in `Fatal` reporting mode for the duration of every test (`gui/tests/RocketPartModelTests.cpp:41`): Qt’s own protocol checker rides the full edit sequence — install, attaches, mass edit, link edit, detach, clear, reinstall — and aborts the test on any begin/end violation at the instant it fires.

Beyond the protocol, the surgical guarantees are pinned directly: a `QPersistentModelIndex` survives unrelated attaches and leaf edits unchanged, dies exactly when its node is detached, and its held id can never alias a live row (`gui/tests/RocketPartModelTests.cpp:87`); structural edits emit one insert and one remove with zero resets, and only a design install resets (`gui/tests/RocketPartModelTests.cpp:130`). [Listing 33](#) shows the pin that a leaf edit is a `dataChanged`, never a reset.

```
TEST_F(RocketPartModelTests, LeafEditEmitsDataChangedNotReset)
{
    rocket.installDesign(model::PartNode::make(bodyTube("Root")));
    const auto rootId = rocket.parts().root()->id();

    QSignalSpy dataChanged(&partModel, &QAbstractItemModel::dataChanged);
    QSignalSpy resets(&partModel, &QAbstractItemModel::modelReset);

    ASSERT_TRUE(rocket.parts().setPartMass(rootId, 0.321));

    ASSERT_EQ(dataChanged.count(), 1);
    const auto args = dataChanged.takeFirst();
    const auto left = args.at(0).toModelIndex();
    EXPECT_EQ(static_cast<model::part::Part::Id>(left.internalId()), rootId);
    // ...
    EXPECT_EQ(resets.count(), 0);
}
```

Listing 33. A routed mass edit reaches the view as exactly one `dataChanged` on the right node — and zero resets (`gui/tests/RocketPartModelTests.cpp:111`).

Together with the suite-wide gates of [Section 13](#), this closes the loop the section opened with: a Qt-free tree, one subscriber, a stateless adapter, and a protocol checker confirming that the translation from typed events to Qt row operations is exact. The same layer is the substrate for the interactive editing that follows — drag-reparent expressed as `beginMoveRows` over the same event vocabulary, and undo built on `detach`’s owned sub-tree return value ([Section 4](#)) with selection restored through the stable ids this section already carries.

12 Design decisions and rejected alternatives

The preceding sections describe the machinery; this one defends it. Every load-bearing choice in the part-tree design is collected here as a decision record: the decision, the rationale that wins, and the alternatives that lose — with the specific failure that disqualifies each. The entries are grouped by theme (ownership, caching, mutation, the motor, the error and event surface, concurrency), but each stands alone and is grounded in the code it defends.

A pattern recurs throughout and is worth naming up front: wherever a correctness property can be made *unrepresentable* — by ownership type, by access control, by a typed verb — the design prefers that to a runtime check, and prefers a runtime check to a convention. The alternatives below mostly lose because they defend the same property at a weaker rung of that ladder.

12.1 Ownership and structure

Linear ownership. Every `PartNode` owns its `Part` and its children through `std::unique_ptr` (`core/model/PartsModel.h`); `PartFactory` produces `std::unique_ptr<Part>` (`core/model/parts/PartFactory.h:60`); and `PartsModel::attach` consumes one by value, so ownership transfer is visible in the signature and enforced by the move. No `std::shared_ptr<Part>` exists anywhere in the production tree. The consequence is that a part has exactly one owner at all times: double-attach, membership in two models, and post-attach aliasing are compile errors, not audited conventions.

Rejected — shared-pointer interior

Holding parts by `std::shared_ptr` and handing copies to consumers loses on two counts, both silent. First, a retained alias is an unrouted write path: the composite mass-delta gate ([Section 5](#)) backstops a mass edit — the live sum moves on the next read — but the gate keys on composite mass *alone* (`core/model/PartsModel.cpp:122`), so an equal-mass tensor edit through an alias (a same-mass, different-diameter motor is the canonical case) flies a stale composite inertia tensor with no error, ever. Widening the gate to notice tensor edits would mean recomputing the tensor in order to validate the tensor — the cache doing the work it exists to skip. The staleness class therefore cannot be closed by checking; it must be closed structurally, by making the alias uncompileable. Second, one shared part becomes attachable under two models: each composite counts the same mass, and an edit routed through one model leaves the other's caches stale with no event. Linear ownership deletes both failure classes at the type level.

A recursive owned node tree, not a flat arena. The tree is stored as it is shaped: each node holds a vector of owned children and a parent back-pointer, and `Part::Id` — process-unique, never reused, fresh on every clone — doubles as the O(1) index key (`core/model/PartsModel.h`).

Rejected — flat arena with generational ids

Contiguous node records indexed by integer slot, parent indexes, generation counters, and a cached preorder were evaluated as an alternative storage scheme. The arena has the cleanest invalidation story on paper — structural edits bump a generation exhaustively, by construction — but it loses three ways. (1) Slot reuse makes a bare index silently wrong after freelist recycling; every retained handle must carry a generation stamp to dodge the ABA hazard, and the handles that matter most — GUI selection and undo — are held across arbitrary edits. `Part::Id` as a never-reused counter is immune outright: a stale id fails the O(1) lookup safely instead of resolving to a stranger. (2) A payload-free generation bump can never drive Qt's granular

row operations, which need $\{parent, row\}$ before the edit (Section 11); the event stream the GUI consumes carries exactly that payload (`core/model/PartsModel.h:143`). (3) Contiguity pays only when cache misses dominate traversal, i.e. at node counts orders of magnitude beyond a rocket's. A design is tens of parts; the hot-path walk touches each node once per integrator stage regardless of layout. The arena's genuinely good ideas — typed mutation verbs, exhaustive invalidation on structural edits, stamp-after-success — appear in the tree design without the storage cost.

12.2 Cache placement and the gate

Per-node caches. Every `PartNode` carries its own composite-mass-properties cache and its own resolved-placement cache (`core/model/PartsModel.h`), not just the root.

Rejected — root-only caching

Caching only at the root halves the field count and nothing else. Every composite query is defined on `PartNode` precisely so that *any* node is a valid sub-assembly root — the API surface that stage composites (multi-stage designs are planned) and detached-subtree preview consume. Root-only caching turns each such query into a full recompute with nowhere to memoize. Meanwhile the invalidation logic is identical either way: the dirty walk already marks every ancestor to the root (`core/model/PartsModel.h`), so per-node caching adds no invalidation complexity — only a few hundred bytes of `mutable` state across the whole tree.

The mass-delta gate compares exactly. The composite CM/tensor cache rebuilds iff the node is structurally dirty or the live mass sum differs from the stamp by exact `==`, with a NaN sentinel forcing the first build (`core/model/PartsModel.cpp:122`). The key is the composite mass itself — already computed every stage as the ODE divisor (`core/sim/Propagator.cpp:41`), so the gate costs nothing beyond a comparison. The division of labor is deliberate: the gate covers *time* variation (a burning motor), routing covers *edits* — because an equal-mass edit is invisible to any mass-keyed gate, as Section 12.3 develops.

Rejected — an epsilon tolerance on the gate

An approximate comparison trades the bit-exact post-burnout freeze for a tunable with no principled value. After burnout the live sum repeats bit-for-bit, so exact `==` freezes the cache with zero false rebuilds; during the slow tail of a burn, per-stage mass deltas shrink below any fixed epsilon, so an epsilon gate silently freezes the CM and tensor *while mass is still draining* — a stale-cache class with no error and no way to pick the threshold that avoids it.

Rejected — time-keyed caching

Caching keyed on the query time t assumes t is monotone and unique. The adaptive RK45 integrator violates both: stages evaluate at repeated intermediate times, and a rejected step re-queries times already seen. A t -keyed cache therefore either thrashes (rebuild per stage — exactly the hot-path cost the cache exists to avoid) or serves a value built *before an edit* at the same t . Keying on the live mass sum is immune to both, and doubles as a physical statement: for a fixed structure, the composite CM and tensor change only when the mass distribution does.

12.3 Routed mutation

Verbs on the model, private setters on the leaf. Every post-attach write path is a `PartsModel` verb — `setPartMass`, `setPartInertia`, `setLink`, `mutatePart`, and the motor verbs — each of which writes and then dirties the correct cache chain before returning. `Part`’s setters are `private` with `PartsModel` as the sole friend (`core/model/parts/Part.h:133`), so no other code can reach them. `MutateHint` declares whether an arbitrary edit is mass-only or geometric, so a mass edit never re-runs the placement resolve and envelope sweep. Listing 34 shows the pattern at its most instructive point: a seat edit moves resolved poses, and with them the composite CM and tensor, at possibly unchanged mass — so the verb dirties *both* chains, because no gate downstream could detect the change.

```
bool PartsModel::setLink(part::Part::Id child, const part::StationLink& link)
{
    PartNode* n = find(child);
    if(n == nullptr || n == root_.get())
    {
        return false; // a root has no incoming edge
    }
    const Event e{Event::LinkChanged, child, n->parent_->id(), n->rowInParent()};
    emitEvent(e, true);
    n->link_ = link;
    // A seat edit is geometry: resolved poses move and the composite CM/tensor move with them --
    // the mass gate alone cannot see an equal-mass geometry change, so both chains dirty.
    n->parent_->markMassDirty();
    n->parent_->markPlacementDirty();
    emitEvent(e, false);
    return true;
}
```

Listing 34. A routed verb writes, then dirties the chains the write actually invalidates (`core/model/PartsModel.cpp:431`).

The gate is a backstop, not a router

The mass-delta gate catches mass drift from any source, but it is structurally blind to equal-mass edits — a tensor write, a seat move, a same-mass motor swap. Routing is therefore load-bearing correctness, not style: it is the only invalidation-complete option, and linear ownership (Section 12.1) is what makes the routed surface exhaustive by construction.

Rejected — an observer or back-pointer on `Part`

Giving the leaf a pointer to its node so its setters self-invalidate reintroduces tree knowledge into the class whose contract is precisely that it has none (Section 3): every detached part would need null-check discipline on every setter. Worse, a public self-invalidating setter bypasses the model’s event stream — caches would refresh but no `Mutated` event would fire, so the GUI’s mass column goes stale over a perfectly fresh cache. Invalidation and notification must travel together, and only a model-level verb sees both.

Rejected — immutable parts, replace-on-edit

Modeling every edit as clone-with-new-value plus node swap makes staleness unrepresentable, but at the cost of identity: `Part::Id` is fresh per instance, so every mass edit churns the id. GUI selection (keyed on `QModelIndex::internalId`, `gui/RocketTreeView.cpp:140`), the model's typed motor borrow, and any held undo handle would all invalidate on a slider drag, and a scalar edit would masquerade as a structural detach/attach with event noise to match. Stable identity across in-place edits is exactly what mutable-behind-verbs buys.

Rejected — polled version counters

A per-part version integer bumped by public setters, aggregated and compared on read, turns every composite query into an $O(N)$ version sweep on the per-stage hot path — paid on every read, forever, to detect edits that arrive through a handful of call sites. And a version bump says *that* something changed, not *what*: placement work would re-run on mass edits, or a second per-aspect counter would reinvent `MutateHint` with strictly more state.

Rejected — protected leaf setters

Protected access lets any subclass re-export a public mutator that skips invalidation and events; a motor type wrapping its own state is the obvious candidate. Private plus a single named friend (`core/model/parts/Part.h:133`) makes the compiler enforce the seam against subclasses too.

12.4 The motor

Live-derived state. `part::Motor` owns its `MotorModel` by value and derives everything at query time: `getMass(t)` and `getI()` read the wrapped model live (`core/model/parts/Motor.h:46`). No seeded copy of the motor's mass or tensor exists anywhere, so `PartsModel::setMotor`'s swap path reduces to *replace the wrapped model, dirty the mass chain* (`core/model/PartsModel.cpp`) — and the stale-seed divergence class has no representation to occupy.

Rejected — seeded base-field copies

Seeding the leaf's mass/tensor fields from the motor at construction and re-seeding on swap creates a write path that must *remember* to invalidate — and the one swap most likely to be forgotten is the one the mass gate can never catch: equal mass, different diameter, stale tensor, no error. Live derivation deletes the state that could go stale instead of testing for its staleness.

Burn-dependent grain inertia is enabled, not gated out

The live-derivation contract deliberately declines to pin the motor's tensor constant. Propellant-grain inertia that tracks the burn is desirable future modeling, and when it arrives `getI()` grows a time parameter mirroring `getMass(t)` — the composite walk already has t in hand at the call site. A design that froze the tensor by contract would buy nothing today and cost that feature later.

Persisted as a database reference. A design file records its motor as a common-name reference plus the seat link on its edge (`core/model/DesignSerializer.cpp:205`); load re-resolves the name against the motor database, and a miss degrades to design-without-motor with a logged warning (`core/model/DesignSerializer.cpp:257`) rather than a failed load. A `Motor` appearing as an ordinary tree

`<part>` element is rejected outright (`core/model/DesignSerializer.cpp:155`), so the reference is the one channel.

Rejected — inlining the thrust curve

Embedding the full curve in the design file creates two sources of truth that skew as the database updates, bloats every file with data the database already owns, and leaves the motor's identity ambiguous (no key to re-resolve corrections against). Motors are per-flight database entities; the airframe file names one, it does not contain one — and a missing name must not brick the airframe, hence degrade-with-warning rather than throw.

Zero-span nodes host nothing. The envelope sweep excludes any candidate host whose axial span is zero (`core/model/parts/Placement.cpp:183`); the `Motor` node, with the base zero-length geometry (`core/model/parts/Part.h`), is the standing example. The solid-host rule samples radii across the host's interior span; a point mass has no interior stations to sample, and admitting it as a host would let it shadow the real enclosing part — an offender “fitting” against a point.

Rejected — a synthetic motor envelope

Inventing a nonzero casing envelope for the `Motor` node so it can participate in the sweep would collide with the mount geometry the airframe already models — the same volume claimed twice — and would oblige every future point-mass-like part (ballast, electronics) to carry fake geometry just to stay out of the sweep's way. Exclusion by span is one comparison and states the physics plainly.

12.5 Errors, events, and the empty design

Typed `std::expected` errors. Structural verbs return their cause: `attach` yields `std::expected<Id, AttachError>` with `NullPart` / `NoSuchParent` / `DuplicateMotor`, and `detach` yields the detached subtree or `NoSuchId` / `IsRoot` (`core/model/PartsModel.h:136`). The CLI switches on the enumerator and prints a precise message per cause (`cli/Repl.cpp:955`); tests pin each enumerator directly. On success, `detach`'s value is the subtree — the natural undo/clipboard token — so the error channel and the payload share one signature.

Rejected — `bool` returns or logged no-ops

A verb that fails silently (or with only a log line) forces callers to *probe* for failure — counting children before and after the call — and to guess at the cause in their own error text, which then diverges per call site. A boolean is barely better: it answers “did it work” but not “why not,” so the CLI cannot distinguish a bad parent id from a second motor, and a test cannot pin the rejection it intends to provoke.

Dual `aboutTo/did` events. Every mutation fires its `Event` twice — once with `before == true`, once after (`core/model/PartsModel.h:143`) — carrying `{kind, id, parentId, row}`. This is the exact shape Qt's model contract needs: `beginInsertRows`/`beginRemoveRows` must run *before* the structure changes, while the pre-edit rows are still queryable, and the matching `end*` call runs after (`gui/RocketTreeView.cpp:32`). The core stays Qt-free; it simply emits an event shape a Qt adapter can consume losslessly ([Section 11](#)).

Rejected — a single post-hoc notification

A lone “something changed” callback after the fact can never satisfy the `begin*/end*` protocol, so a Qt view built on it must fall back to `beginResetModel/endResetModel` on every edit — destroying selection and expansion state on each keystroke of a mass field. The design keeps a coarse post-hoc callback *as well* (`core/model/RocketModel.h`) for consumers that only repaint; the dual stream is a superset, not a compromise.

A nullable root. “No design” is a real state and is represented directly: `hasDesign()` is a null test on the root (`core/model/PartsModel.h:161`). Every consumer is forced to confront the empty state explicitly, which is what makes the CLI’s contract crisp: `ERR addpart: no design (use newdesign first)` (`cli/Repl.cpp:906`), and likewise for `checkdesign` and `listparts`.

Rejected — a placeholder root part

A mandatory synthetic root makes the empty state unrepresentable rather than handled: `hasDesign()` degrades to recognizing a magic part, the serializer must persist or special-case it, the composite walk and the sweep process a part that is not part of the rocket, and the CLI’s verified `no design` error contract dissolves into heuristics. A null pointer is one comparison and cannot be mistaken for geometry.

12.6 Concurrency: single-writer now, snapshots later

The model is single-writer by contract, and the contract is stated where it binds: in the `PartsModel` class documentation and at the `PartNode` cache fields (`core/model/PartsModel.h`). The `const` composite accessors are *reads that write* — they rebuild and re-stamp `mutable` caches — and even `thrust(t)` writes a `mutable` log-once burnout latch inside a `const` call chain (`core/model/MotorModel.h:342`). Concurrent access, all-`const` included, is therefore a data race by design, and the signatures must not be read as a thread-safety promise.

Rejected — a lock or mutex

The reads-that-write pattern defeats reader-writer locking outright: any read may need to become the writer, so a shared/exclusive lock degenerates to exclusive on every composite read — a mutex acquisition per RK45 stage, on the hottest path in the program (the propagator divides by `getMass(t)` every stage, `core/sim/Propagator.cpp:41`). Locking also fails the consumers it claims to serve: a CM and a tensor are physically meaningful only as a pair from the same rebuild, and a per-call mutex gives no such multi-value consistency unless held across grouped reads — inviting GUI-holds-lock stutter and callback re-entrancy deadlocks. Its one genuine advantage, zero-staleness reads, is worthless to a GUI repainting from millisecond-old frames.

If threading arrives: snapshot, do not share

The direction of record for a future worker-thread simulation is snapshotting: fly a `PartNode::clone()` of the design on the worker — edits during a flight then apply to the *next* launch, the sane semantics regardless of threading — and publish telemetry as immutable `StateData` frames. Batch work (motor-ladder sweeps, Monte Carlo) takes one clone per flight and is embarrassingly parallel. Each thread owns its data; only the handoff synchronizes. The one known design obligation for that day: `Part::Id` is fresh on clone, so mapping results back

onto the edited tree needs a clone-time id correspondence. Until a real concurrency requirement exists, the single-writer contract stays documented at the cache sites so threading work reopens it deliberately instead of trusting `const`.

Taken together, the record reads as one decision made twelve ways: put each invariant inside the smallest boundary that can actually defend it — the type system where possible, a friend seam or a typed verb where not — and spend runtime checks only on what structure cannot express. [Section 13](#) closes the loop by showing how the test suite pins each of these choices in place.

13 Verification strategy

The preceding sections make strong claims: edits route through one authority and dirty exactly the right caches ([Section 4](#)), composite mass properties rebuild only when their inputs move and freeze bit-for-bit after burnout ([Section 5](#)), a self-intersecting design is a hard located failure on every read ([Section 6](#)), and the Qt adapter consumes the event stream without ever resetting the view for an ordinary edit ([Section 11](#)). Claims of that kind are cheap to state and easy to lose — a one-line change to a routed verb can silently stop invalidating a cache, and nothing in a typical flight would notice for weeks. The test architecture exists to make each such claim *fail loudly* the day it stops being true, and this section describes how the suites are organized to do that.

Verification runs at three altitudes. Unit suites pin each behavioral seam of the model layer in isolation — including the seams that are deliberately adversarial to observe, such as a cache gate that is blind to equal-mass edits by construction. Integration suites pin the assembled system numerically, against committed baselines and a fixture corpus built by the real CLI. Toolchain gates — sanitizers, static analysis, and coverage — pin the implementation itself. [Table 6](#) maps the test binaries to what each one holds still; all are registered with ctest under names matching `qtrocket_*`, which is exactly what CI runs (`.github/actions/build-and-test/action.yml`).

13.1 Behavioral seams, not white-box hooks

The parts tree exposes no test-only surface. The single `friend` declaration in the leaf is the production mutation seam itself — `PartsModel` (`core/model/parts/Part.h:133`) — and nothing in `core/model/PartsModel.h` grants a test access to the caches, the id index, or the dirty flags. Every test in `core/model/tests/PartsModelTests.cpp` observes the tree exactly the way the CLI, the GUI, and the integrator do: through routed verbs, `const` composite readers, and the event callback.

Tests pin behavior, not layout

A test may construct adversarial *parts* — subclasses of the public leaf that override its virtuals — but it may not open the tree. If a property matters, there is a public observation that changes when it breaks; if no such observation exists, the property is not yet a contract and does not get a test.

This rule has teeth because the leaf is designed to be subclassed ([Section 3](#)). The suites instrument by *overriding virtuals and counting calls*: `CountingRampPart` counts `getMass()` queries so cache rebuilds become arithmetic (`core/model/tests/PartsModelTests.cpp:67`); `CountingTube` counts `radiusOuterAt()` queries, a virtual touched only by the resolver and the envelope sweep, so “did the geometry cache rebuild?” has a faithful witness (`core/model/tests/DiagnosticsGateTests.cpp:64`); `RampMassTube` fakes a burning motor with a strictly increasing mass; `FixedAeroPart` returns a fixed

Binary (ctest entry)	What it pins
<code>model_tests</code> <code>(qtrocket_model_tests)</code>	The model layer seam by seam: PartsModel verbs, typed errors, cache gates, events, and the motor lifecycle (PartsModelTests); composite physics against closed forms (PartCompositionTest, HollowSphereTest); seat semantics, deterministic traversal, and the envelope sweep (ResolverTests, SweepTests, DiagnosticsGateTests); Barrowman composition and datum algebra (AeroTest); one suite per concrete leaf (BodyTubeTests, NoseConeTests, FinSetTests, MotorTests); the factory, geometry profiles, inertia tensors, placement types, and the thrust-curve loaders.
<code>sim_tests</code> <code>(qtrocket_sim_tests)</code>	The numerics below the model: RK45 adaptive stepping, the US Standard Atmosphere, gravity and geoid models, integrator selection.
<code>propagator_tests</code> <code>(qtrocket_propagator_tests)</code>	Propagator termination guards and trajectory statistics, driven by a mock Propagatable — no motor database, no Qt.
<code>integration_tests</code> <code>(qtrocket_integration_tests)</code>	The assembled system: <code>.qrd</code> round trips including motor-by-reference (DesignRoundTrip), the placement-invariance corpus baseline (PlacementInvariance), the frozen poke-through offender (PokeThroughRegressionTests), end-to-end flight physics, and motor-database persistence.
<code>cli_tests</code> <code>(qtrocket_cli_tests)</code>	Black-box REPL command contracts: build → save → reload → fly through the real <code>cli::Repl</code>, input validation, seat-vocabulary authoring, checkdesign reporting, and the process exit-code contract.
<code>design_matrix_tests</code> <code>(qtrocket_design_matrix_tests</code> and the heavy -labeled full-ladder entry)	The fixture matrix: every committed design loads, round-trips its exact composite footer and tree structure, and flies its motor ladder to a nominal, impulse-monotonic apogee; concrete-part scaling laws; script-replay anti-drift (the DesignRebuild suite); the gram→kg motor-mass regression guard.
<code>gui_model_tests</code> <code>(qtrocket_gui_model_tests)</code>	RocketPartModel's QAbstractItemModel contract under the granular event stream, plus the surgical guarantees: persistent indexes survive unrelated edits, leaf edits emit dataChanged, only a design install resets (Section 11).

Table 6. The test binaries and the claims each one keeps honest. Sources live next to what they test: `core/model/tests/`, `core/sim/tests/`, `tests/`, and `gui/tests/`.

aerodynamic contribution so the composition walk is tested independently of any real part’s coefficients (`core/model/tests/AeroTests.cpp:28`).

Rejected — a test-access friend class

A `friend`ed accessor struct could read the private caches directly and assert on their contents. It is rejected because it verifies the wrong thing: a cache-layout assertion passes while observable behavior regresses (a stale value served with a perfectly healthy-looking cache), and it fails while behavior is correct (any refactor of the private layout). Worse, it freezes the interior of `PartNode` into the test suite, so the layout can never change without rewriting tests that were never entitled to see it. Call-counting through a public virtual costs a few lines per suite and measures precisely the contract: how many times the system *asked*.

13.2 Unit pins on the tree and its gates

`PartsModelTests` covers the authority’s whole surface, but its center of gravity is the handful of cases the architecture is most exposed to. The structural verbs are pinned with their typed errors — `NullPart`, `NoSuchParent`, `IsRoot`, `NoSuchId` — and `AttachReportsTypedErrors` drives the single-motor invariant from both directions: a second `Motor` is rejected whether it arrives as the sub-tree root or buried inside an attached sub-tree (`DuplicateMotor`), and a failed attach leaves the tree untouched (`core/model/tests/PartsModelTests.cpp:175`). Stale ids fail safe: after a detach, every id in the removed sub-tree leaves the index and `find` returns `nullptr` rather than a dangling node. Events are pinned as strict aboutTo/did pairs — one mutation, two records, identical payload, `before` true then false — across a full edit sequence of fourteen records (`core/model/tests/PartsModelTests.cpp:496`). Deep clone yields fresh ids with verbatim per-edge links; traversal is pre-order in attachment order; and a direct `addChild` on an owned node is a rejected no-op, pinning the rule that owned trees mutate only through verbs (`core/model/tests/PartsModelTests.cpp:562`).

Attacking the gate’s blind spot

The mass-delta gate keys on one number: the live composite mass sum ([Section 5](#)). Any edit that changes geometry without changing mass is therefore *invisible to the gate by construction* — correctness rests entirely on each routed verb dirtying the cache chain. Three tests aim at exactly that blind spot: `SetPartInertiaChangesCompositeIAtEqualMass` swaps a part’s per-unit-mass tensor at constant mass (`core/model/tests/PartsModelTests.cpp:282`); `MotorSwapAtEqualMassChangesCompositeI` installs a same-mass, different-diameter motor and requires the composite I_{zz} to grow (`core/model/tests/PartsModelTests.cpp:297`); and `SetPartMassAndSetLinkInvalidateComposite` moves a link at constant mass and requires the composite CM to follow. If any verb ever stops dirtying, one of these fails on the next read — the silent-staleness bug the design works hardest to exclude is also the one the suite works hardest to detect.

The gate itself is pinned by counting. Each `compositeI(t)` costs the counting part one `getMass` call for the gate sum, plus one more inside the rebuild — so a rebuild reads as 2 and a cache hit as 1, and the gate’s entire lifecycle becomes a sequence of small integers ([Listing 35](#)). The same technique pins the placement gate one level up: `DiagnosticsGateTests.GateDoesNotReFirePerStepDuringBurn` runs a ramp-mass “burn” that forces a genuine composite recompute every step, while the counted `radiusOuterAt()` shows the resolve-and-sweep pass runs exactly once (`core/model/tests/DiagnosticsGateTests.cpp:109`) — per-step re-weighting, per-edit geometry, never the reverse.


```

const auto delta = [&](double t)
{ const int before = raw->calls; (void)root->compositeI(t); return raw->calls - before; };

EXPECT_EQ(delta(0.0), 2); // first query always builds (NaN sentinel)
EXPECT_EQ(delta(0.0), 1); // same t: gate sum only, no rebuild
EXPECT_EQ(delta(5.0), 1); // different t, same mass: still no rebuild

// ... and with mass ramping over a two-second burn:
delta(0.5); // first build
EXPECT_EQ(delta(1.0), 2); // mass moved during the burn -> rebuild
EXPECT_EQ(delta(3.0), 2); // crossing into post-burnout: final transition rebuild
EXPECT_EQ(delta(3.5), 1); // frozen: mass constant -> gate sum only
EXPECT_EQ(delta(3.2), 1); // out-of-order post-burnout probe: still frozen

```

Listing 35. Cache rebuilds as arithmetic: the call-count delta distinguishes a rebuild (2) from a gate-only hit (1), so the gate’s whole lifecycle — first build, constant-mass hits, burn tracking, the burnout transition, and the out-of-order probe an adaptive integrator can issue — is pinned in a dozen lines. From `core/model/tests/PartsModelTests.cpp:316` and `core/model/tests/PartsModelTests.cpp:336`.

`BurningMotorTracksAndFreezesBitwiseAfterBurnout` pins the freeze *bitwise*, not approximately: it drives a real motor through its burn — the composite CG walks forward as the aft-seated propellant depletes — and then requires $\|I(t) - I_{\text{frozen}}\| = 0$ exactly for every post-burnout read (`core/model/tests/PartsModelTests.cpp:356`). The failure side is pinned with the same severity: `SelfIntersectingDesignThrowsOnEveryCompositeRead` builds an over-nested coupler, checks the located diagnostic names the right offender and host, and then reads the composite twice — because the cache stamp is written only after a successful solve, a failed solve can never be swallowed by the cache, and *every* read throws, not just the first (`core/model/tests/PartsModelTests.cpp:385`). The one deliberate exception is asserted too: `compositeMass` is the hot ODE divisor, does no placement work, and stays readable.

Beneath the authority, `PartCompositionTest` pins the composite physics against closed forms a human can re-derive: a point-mass pair’s transverse inertia is μL^2 about the mass-weighted CM, two tubes end to end equal one longer tube, a three-mass chain at depth two matches its flat reference, and `HollowSphereTest` checks the thick-shell tensor against the textbook expression (`core/model/tests/PartTests.cpp`). `ResolverTests` and `SweepTests` pin seat semantics and the envelope sweep — abut stacks aft, nest-in-bore signs its gap aft, deterministic DFS order, the poke-through diagnostic located to offender and host, ties broken by smallest id — and `AeroTest` pins the Barrowman composition seam: per-part x_{cp} re-expressed on the sub-tree-root datum shared with `compositeCm` (so CP minus CG is the static margin), reference-area invariance, CP undefined for a body-only rocket, and the motor contributing mass but never aerodynamics (`core/model/tests/AeroTests.cpp`).

13.3 Assembled-system pins: round trips and the corpus baseline

`DesignRoundTrip` (`tests/DesignPersistenceTests.cpp`) pins the serialization contract of [Section 8](#) through real files: geometry round-trips mass, CG, and structure; the reference-area override round-trips in both states (overridden and untouched — a load must not manufacture an override that was never set); per-edge links round-trip field by field, the default link is elided on save and reloads as abut, and an unknown seat kind or unsupported major version is rejected leaving the in-memory rocket untouched. The motor round-trips *by reference*: `MotorByNameRoundTripsAndPreservesMassCurve` proves the reloaded rocket’s $m(t)$ matches at ignition,

mid-burn, and post-burnout — equality that can only hold if the database lookup reproduced the full burn curve — while `MotorAbsentInDatabaseLoadsGeometryWithoutTheMotor` pins the degradation path: geometry loads, a warning logs, no motor is set.

The corpus pins are numeric. `PlacementInvariance` loads all 24 committed fixtures airframe-only (an intentionally empty motor database, so the snapshot is pure geometry) and compares every integrator-facing quantity — composite mass, CG, inertia, aero CP, and each part’s resolved CM station in DFS order — against the committed baseline `tests/data/placement-invariance-baseline.txt`. Doubles are stored at 17 significant digits so they round-trip to the identical IEEE-754 value ([Listing 36](#)), and the comparison tolerance is 10^{-9} relative — wide enough to admit ULP-level drift from re-associated arithmetic, orders of magnitude tighter than any real placement bug (`tests/PlacementInvarianceTests.cpp:130`). The baseline has exactly one writer: a regeneration test gated behind `QTROCKET_REGEN_PLACEMENT_BASELINE=1` that is skipped in every normal run, so a diff can only be silenced deliberately (`tests/PlacementInvarianceTests.cpp:251`). A companion test reports the worst observed relative drift across the whole corpus on every run, so the margin between reality and the tolerance stays visible in the log.

```
format 2
design large38_basic
mass 0.16378115690911332
cm 0 0 -0.25124370615147357
inertia 0.0046368498312709906 0 0 0 0.0046368498312709906 0 0 0 9.3042721313888361e-05
cp 1 -0.41986417012945754
parts 3
part 0 NoseCone 0 0 -0.08999999999999997 Nose
```

Listing 36. One fixture’s block of the committed placement baseline (`tests/data/placement-invariance-baseline.txt`): mass, CG, row-major inertia, CP, and each part’s resolved CM station in the tip datum, printed at 17 significant digits for exact double round-trip.

Two hand-computed pins guard the one place a baseline cannot: a sign error frozen *into* the baseline. The solid cone’s local CM is pinned to $-3L/4 = -0.225$ m in the tip datum and the fin set’s to its hand-derived trapezoid centroid, both at 10^{-12} , with the expected values computed in the test from raw geometry rather than read back through the part (`tests/PlacementInvarianceTests.cpp:480`). A reversed-frame defect in either part would survive the aggregate corpus pins — consistently wrong is still consistent — but not these. Finally, `PokeThroughRegressionTests` keeps the frozen self-intersecting fixture `tests/data/designs/xl75_multi_pokethrough.qrd` firing end-to-end through the file reader: without a committed offender, nothing outside the unit suites would ever drive the diagnostic path ([Section 6](#)).

13.4 Black-box command contracts

The CLI suites test the program a user actually runs, through the same `cli::Repl` the `qtrocket-cli` binary wraps ([Section 9](#)). `CliDesignCommands` drives a full build → save → reload → fly session and asserts on the printed output only: the composite footer matches after reload, `addpart` authors every seat vocabulary token and rejects malformed link tokens with an `ERR` line, `checkdesign` reports overlaps and air gaps, RASP `.eng` files import, and the process exit code reflects command failures — the property that makes piped CLI scripts CI-consumable (`tests/CliDesignCommandsTests.cpp`).

`design_matrix_tests` scales that contract across the whole fixture matrix: the 24-design corpus is present and every file loads to a sane rocket; every fixture, saved and reloaded, reproduces its

exact composite footer and tree structure; the concrete part types obey their scaling laws (tube mass linear in length and density, fin-set mass linear in fin count); and each reference airframe flies its motor ladder — impulse classes $1/4A \rightarrow M$ across seven diameter classes — terminating nominally with a positive apogee that increases strictly with total impulse. Because high-impulse motors coast to tens of kilometers, the full ladder dominates suite runtime, so the binary is registered with `ctest` twice: the default entry flies one motor per class (a fast smoke that still covers every diameter), and a second entry, labeled `heavy`, re-runs only the flight sweeps with `QTROCKET_FULL_LADDER=1` flying every motor (`tests/CMakeLists.txt:97`). `ctest -LE heavy` is the development loop; CI runs a plain `ctest -R "qtrocket_*"`, covering both.

The `DesignRebuild` suite (`tests/DesignRebuildTests.cpp`) closes the loop on the corpus itself: every committed fixture is the exact product of replaying its build script (`tests/data/designs/scripts/`) through the real REPL, so fixture and script cannot drift — and `tests/data/designs/regenerate.sh` rebuilds the corpus through `qtrocket-cli` after a deliberate design change, with each script ending in `checkdesign` so a regeneration that produces overlaps fails at the source.

13.5 The Qt adapter contract

`gui_model_tests` wires `RocketPartModel` to a `PartsModel` exactly as the tree view does and attaches a `QAbstractItemModelTester` in `Fatal` mode for the lifetime of every test, so each mutation streams through Qt’s own protocol checker and any contract violation aborts the run (`gui/tests/RocketPartModelTests.cpp`). On top of that continuous check, the surgical guarantees of [Section 11](#) are pinned directly: a `QPersistentModelIndex` survives unrelated edits and dies with exactly its node, a leaf edit emits one `dataChanged` and no reset, and the only `modelReset` in an entire edit sequence is a design install. The binary runs headless under `QCoreApplication` — the model under test is widget-free, so no platform plugin is required and the suite runs in the same CI lane as everything else.

13.6 The corpus transcript diff

Flying the corpus as a physics-preservation gate

The pieces above compose into a verification technique that is more than the sum of the suites: because the simulation core is deterministic, the composite readers freeze bit-for-bit ([Section 5](#)), and `qtrocket-cli` exits nonzero on any `ERR`, a scripted flight over the `tests/data/designs` fixtures produces a byte-stable transcript — `loaddesign`, `setmotor`, `launch`, apogee statistics. Any change that is not supposed to alter physics — a cache restructuring, an ownership change, a performance pass — can therefore be gated by flying the full corpus before and after and diffing the transcripts *byte-for-byte*. A clean diff is a far stronger statement than “the tests still pass”: it says every load, every resolve, and every integration step produced the identical bits.

The committed placement baseline ([Section 13.3](#)) is the always-on, $t=0$ subset of this idea; the transcript diff is its on-demand, full-flight form. The two are kept honest by the same discipline: baselines and fixtures are regenerated only for a deliberate, explained change — never to silence a diff — and the regeneration paths are gated (an environment variable for the baseline, an explicit script for the corpus) so they cannot run by accident.

13.7 Toolchain gates

Three gates guard the implementation independently of any assertion. The `asan-clang` preset builds the entire tree — FetchContent dependencies included — under AddressSanitizer and UndefinedBehaviorSanitizer in a separate `build-asan/` directory, with `-fno-sanitize-recover=all` making every UBSan finding fatal (`CMakeLists.txt:20`); a passing `ctest --preset asan-clang -R "qtrocket_*` is therefore a genuinely clean run, not a run whose warnings scrolled by. This matters specifically for the ownership design of [Section 4](#): linear `unique_ptr` ownership and `detach-returns-the-subtree` are exactly the shapes where a lifetime bug would hide, and ASan turns any such bug into a hard failure in the same suites that pin behavior.

Static analysis is layered. The root `.clang-tidy` enables a curated, signal-dense set for a numerics codebase — the `bugprone`, `clang-analyzer`, `concurrency`, `performance`, and `portability` families plus selected `misc`, `modernize`, and `readability` checks — with `WarningsAsErrors` set to everything, so a finding fails the run. Test directories carry a one-line override (`tests/.clang-tidy` and siblings) inheriting the parent config and disabling only `bugprone-unchecked-optional-access`: gtest's `ASSERT_*` guards abort the test on failure, but the check's flow analysis cannot see that abort path, so every guarded `std::optional` dereference in a test is a false positive — while the check stays armed for production code, where the same pattern would be a real defect. `scripts/run-tidy.sh` runs the identical configuration CI runs, over every hand-written translation unit.

Coverage closes the loop on the suites themselves. The `coverage-clang` preset builds LLVM source-based instrumented binaries in a separate `build-coverage/` tree, and the `coverage` target runs the full `qtrocket_*` suite, merges the raw profiles, and writes a text summary and an HTML report (`cmake/RunLlvmCoverage.cmake`). Its role is honesty about the pins: a behavioral seam that the suites never actually exercise shows up as an uncovered region in the very file that claims to be pinned — the one failure mode a green test run cannot report about itself.

A Glossary

This appendix defines the terms of art as they are used throughout this whitepaper, in alphabetical order. Every entry names a construct in the working tree and cites where it lives; the definitions are deliberately terse — the load-bearing discussion is in the body sections cross-referenced from each entry.

aboutTo/did event pair One mutation’s change notification, fired twice: once with `before == true` immediately before the write lands, once with `before == false` immediately after (`PartsModel::Event`, `core/model/PartsModel.h:144`). The pairing exists because Qt’s `QAbstractItemModel` row-op contract demands `begin*` before the model mutates and `end*` after; the stream is shaped so a Qt adapter is a pure translation ([Section 11](#)).

Composite (mass / CM / inertia) The aggregate of a node together with all its descendants: total mass, mass-weighted center of mass, and the full kg m^2 inertia tensor about that CM, computed by the two-pass walk `PartNode::computeCompositeAt` (`core/model/PartsModel.cpp:171`) and served by `compositeMass/compositeCm/compositeI`. Any node is a valid sub-assembly root for these queries ([Section 4](#)).

Detached sub-tree A `std::unique_ptr<PartNode>` owned by no `PartsModel`: the return value of `detach`, the product of `PartNode::make/addChild/clone`, and the currency of serializer load and clipboard-style workflows. It is a first-class value — it re-enters a model only through `attachSubtree` or `installRoot` (`core/model/PartsModel.h`).

Gap (seat-signed) The `gap` field of a `StationLink`: metres, with meaning selected by the seat — a forward standoff along $+z$ = forward for **Abut** and **OnSurface**, a non-negative insertion depth driving the child aft for **NestInBore**. `placeChild` applies it as the signed term in $z_{\text{child}} = z_p + \text{signedGap} - z_c$ (`core/model/parts/Placement.h:162`).

Install facade The single path by which a whole design enters or leaves the model: `RocketModel::installDesign` (`core/model/RocketModel.h:87`) delegating to `PartsModel::installRoot` (`core/model/PartsModel.h:220`). It atomically replaces the root (null clears), re-resolves the motor borrow, and applies the one install side effect owned by `RocketModel`: a fresh airframe does not inherit a manual reference-area override (`core/model/RocketModel.cpp:121`).

Leaf What a `Part` is: one component’s mass, geometry-derived inertia, CM offset, silhouette profile, and aero contribution — and nothing else. Tree structure, placement, and every composite quantity live outside the leaf, in `PartNode/PartsModel` ([Section 3](#)).

Live sum The uncached recursive mass total `PartNode::compositeMass(t)` (`core/model/PartsModel.cpp:102`): this node’s `getMass(t)` plus every descendant’s, recomputed on each call. It is both the ODE mass divisor and the key the mass-delta gate compares against its stamp; it does no tensor work, which is why it is cheap enough to evaluate every integrator stage.

Mass-delta gate The cache gate for composite CM and inertia (`PartNode::ensureCompositeCache`, `core/model/PartsModel.cpp:114`): rebuild when the structure was edited (`massDirty_`) or when the live sum differs from the stamped `builtAtMass_` under exact `==` comparison. The NaN sentinel forces the first build; after burnout the live sum repeats bit-for-bit and the cache freezes ([Section 5](#)).

Motor borrow The typed non-owning pointer `part::Motor* motor_` (`core/model/PartsModel.h:261`) aiming into the owned tree, giving the motor verbs $O(1)$ access to the one motor node. It is re-resolved only inside `registerSubtree/unregisterSubtree`, the adopt/release seam every

structural verb funnels through, so no structural edit can leave it dangling (Section 7).

MutateHint The declared invalidation scope of an arbitrary leaf edit through `mutatePart` (`core/model/PartsModel.h:139`): `MassOnly` dirties the mass chain alone, `Geometry` dirties both chains. The hint exists so a mass-only edit never pays for a geometry re-resolve and envelope sweep.

No-design state The representable empty model: `root_` is null, `hasDesign()` is false, `size()` is 0, and every verb fails cleanly rather than operating on a placeholder rocket (`core/model/PartsModel.h:161`). “No design” is a real state of the system, not an error (Section 4).

Part The pure-leaf base class of all rocket components (`core/model/parts/Part.h`): mass, per-unit-mass inertia, CM offset, length/radius profile, and Barrowman aero contribution, duplicated only via `clone()` (type-preserving, fresh id). Its mass/inertia setters are private to the `PartsModel` friend seam (`core/model/parts/Part.h:133`), so post-attach mutation cannot bypass cache invalidation.

Part::Id The stable per-instance identity: a `std::uint64_t` assigned at construction (fresh even for a copy), never changed, never serialized — ids are meaningful only within one process run (`core/model/parts/Part.h:114`). Names are human-facing labels and need not be unique; identity is id-only. `part::PartId` (`core/model/parts/Placement.h:25`) is its dependency-free mirror in the placement layer.

PartNode One node of the owned tree (`core/model/PartsModel.h:42`): it uniquely owns a `Part` leaf, stores the incoming `StationLink`, owns its child nodes, and carries the two gated caches (composite mass properties; resolved placements plus verdict). Reads hand out `const` references only.

PartsModel The single owner of the node tree and the only mutation authority (`core/model/PartsModel.h:133`), owned by `RocketModel`. It holds the nullable root, the $O(1)$ id index, the motor borrow, and the change callback; every post-attach edit is one of its verbs.

Per-unit-mass tensor The geometric inertia tensor a leaf stores and `getI()` returns: units m^2 , taken about the part’s own CM, independent of mass (`core/model/parts/Part.h:59`). The composite pass multiplies it by `getMass(t)` and parallel-axis-shifts it, producing the full kg m^2 tensor. Storing geometry, not mass-weighted values, keeps a time-varying mass (a burning motor) from invalidating the stored tensor.

Placement gate The structural cache gate (`PartNode::ensurePlacementCache`, `core/model/PartsModel.cpp:144`): geometry is re-resolved and re-swept only when `placementDirty_` is set — a node attached or detached, a link or geometry edit — and never on a mass change, because geometry is invariant under a burn (Section 5).

Pose A rigid placement in the root frame: a fore-plane origin on the axis plus an orientation quaternion, identity in 3-DOF (`core/model/parts/Placement.h:79`). `Pose::compose` is the rigid-transform composition law, degenerating to vector addition today and carrying 6-DOF at zero schema cost.

Resolved placement (Placed) A part paired with its resolved `Pose` and its seat parent’s id (`core/model/parts/Placement.h:98`), produced by the depth-first resolve `resolveNodes` (`core/model/PartsModel.cpp:29`) and cached per node. The one resolve every consumer shares: composite pass, sweep, CLI reporting, visualizer (Section 6).

Routed verb A post-attach mutation path on `PartsModel` — the structural verbs `attach`, `attachSubtree`, `detach`, `installRoot`; the leaf edits `setPartMass`, `setPartInertia`, `setLink`, `mutatePart`; and the motor verbs: each performs the write, dirties exactly the cache chains the edit invalidated, and fires the paired events. Routing is what makes skipped invalidation unrepresentable rather than merely discouraged (Section 2).

Seat kind The closed three-value enum `SeatKind` (`core/model/parts/Placement.h:32`) naming the ra-

dial relationship of one attachment — **Abut**, **NestInBore**, **OnSurface**. The seat selects both the radial seam check and the sign convention of the gap. It is closed by design: a rigid axisymmetric stack has only these three radial relationships.

Single-writer contract The concurrency rule for every mutable-behind-const cache in the model layer: one thread reads and writes; concurrent access — even all-const — is a data race, because `const` accessors write cache fields (`core/model/PartsModel.h:39`). The motor’s log-once burnout latch is under the same contract (`core/model/MotorModel.h:342`).

Solid-host rule The capacity predicate `Part::innerCapacityAt` (`core/model/parts/Part.h:106`): the radius a host grants an intruder is its bore for a bored part and its outer skin for a solid one; an offender of outer radius r fits iff $r \leq \text{capacity} + \text{tol}$. One predicate, correct for both host kinds, consumed by the envelope sweep.

Stamp-after-success The failure discipline of the mass-delta gate: the stamp (`builtAtMass_`, `cleared massDirty_`) is written only after `computeCompositeAt` returns. A failed placement solve therefore throws out of the build and again on every subsequent read — the cache never launders a failure into stale-but-plausible numbers (`core/model/PartsModel.cpp:127`).

Static margin / datum `compositeAero` re-expresses each part’s `x_cp` (reported from that part’s own CM) onto the sub-tree-root fore-plane datum — the same datum `compositeCm` uses — so `cp - cg` is directly the static margin (`core/model/PartsModel.cpp:224`). One shared datum is the point; two datums would make the margin a bookkeeping exercise.

Station (fractional) A position along a part’s axis as a fraction `station01` $\in [0, 1]$: 0 the aft plane, 1 the fore plane, mapped to $z = (\text{station01} - 1) L$ in the part’s $+z = \text{forward}$ frame and resolved against the live `getLength()` on every read — editing a length moves the seam (`core/model/parts/Part.h:102`). `stationAt` returns the resolved `Station` landmark $\{z, r_{\text{outer}}, r_{\text{inner}}\}$, a derived value never stored.

StationLink The stored placement intent of one parent→child edge (`core/model/parts/Placement.h:54`): parent fractional station, child fractional station, seat-signed gap, `SeatKind`, and a per-seam rotation (identity in 3-DOF). A default link means “child fore plane abuts parent aft plane,” so the natural nose→body stack costs no authored numbers. It is the only spatial relationship persisted; poses are always derived.

Sweep (envelope sweep) The whole-tree interference diagnostic `sweepOverlaps` (`core/model/parts/Placement.h:178`): build a world axial interval per part, sort ($O(N \log N)$), sample each offender at feature breakpoints, and test its outer radius against the smallest-id covering host’s capacity under the solid-host rule, excluding the offender’s descendants and direct seat parent. It runs once per structural resolve, never per integration step ([Section 6](#)).

Verdict The cached `SolveResult` of one structural resolve (`core/model/parts/Placement.h:123`): an `ok` flag plus located `OverlapDiagnostics` naming offender and host by id. `ok == false` makes every composite read throw (`core/model/PartsModel.cpp:184`) — a self-intersecting design is a hard, located failure, never a silently wrong mass or tensor.

B Invariants and guarantees

This appendix is the checklist form of the design: each guarantee the implementation upholds, stated in one sentence, with the class or function that enforces it. “Enforced at” names the code that makes the property hold — by construction where the type system carries it, by a guard where a runtime check does. The verification suite in [Section 13](#) exercises each of these; this list is what those tests are tests of. The I-numbers are this appendix’s own, finer-grained list; a bare “invariant n ” in the

body always refers to the eight commitments of [Section 2](#).

- I1 — Linear ownership** Every leaf is owned by exactly one node and every node by exactly one parent (or the model root), so double-attachment, ownership cycles, and retained mutable aliases are unrepresentable.
Enforced at: `std::unique_ptr` end-to-end — `PartNode::part_` and `children_` (`core/model/PartsModel.h:108`), attach verbs taking `std::unique_ptr` by value (`core/model/PartsModel.h:205`), and `Part`’s deleted assignment / protected copy constructor (`core/model/parts/Part.h:56`).
- I2 — Routed mutation** The only post-attach writes to a leaf or to tree structure are `PartsModel` verbs; nothing else can touch a leaf’s setters or an owned node’s children.
Enforced at: `Part::setMass/setI` private behind `friend class model::PartsModel` (`core/model/parts/Part.h:133`), and `PartNode::addChild` rejecting any node already owned by a model (`core/model/PartsModel.cpp:79`).
- I3 — Gate semantics** A mass edit dirties only the mass chain; a structural or link edit dirties both chains; the placement sweep therefore never re-runs for a mass-only change, and a same-mass geometry change still invalidates the composites.
Enforced at: `setPartMass/setPartInertia` (mass chain only, `core/model/PartsModel.cpp:399`), `setLink` dirtying both chains because the mass gate cannot see an equal-mass geometry change (`core/model/PartsModel.cpp:443`), and `mutatePart` honouring `MutateHint` (`core/model/PartsModel.cpp:462`).
- I4 — Stamp-after-success** Cache stamps are written only after a successful composite build, so a failed placement solve throws on every read instead of being swallowed into a stale cache.
Enforced at: `PartNode::ensureCompositeCache` — `builtAtMass_` and `massDirty_` land after `computeCompositeAt` returns (`core/model/PartsModel.cpp:127`).
- I5 — Motor borrow maintenance** The borrowed `part::Motor*` is re-resolved at every point where tree membership changes and nowhere else, so no structural edit — including a full clear — can leave it dangling.
Enforced at: `registerSubtree/unregisterSubtree`, the only code that assigns `motor_` (`core/model/PartsModel.cpp:270`), funnelling `attach`, `attachSubtree`, `detach`, and `installRoot`.
- I6 — Single motor at attach** The tree holds at most one `Motor` node, rejected where the invariant could be broken: an incoming sub-tree containing a motor fails typed when a motor is already installed.
Enforced at: the `attachSubtree` scan returning `AttachError::DuplicateMotor` (`core/model/PartsModel.cpp:319`).
- I7 — Install resets the area override** Installing a design clears any manual reference-area override, so a fresh airframe always starts from its geometry-derived reference area.
Enforced at: `RocketModel::installDesign` setting `referenceAreaOverridden` false after the root swap (`core/model/RocketModel.cpp:121`).
- I8 — Id-only identity** Parts are identified by `Part::Id` — unique per instance, fresh on every construction including copies, never serialized — and never by name or raw pointer; a stale id fails safe as a null lookup.
Enforced at: `Part::getId` (`core/model/parts/Part.h:114`), the id-keyed `index_` behind `PartsModel::find` (`core/model/PartsModel.h:167`), and `findByName` documented as a convenience over non-unique labels (`core/model/PartsModel.h:178`).
- I9 — Resolver determinism** A resolve emits one `Placed` per node in depth-first attachment order, parent before child, so `parentId` always names an earlier entry and the sweep is reproducible run to run.

Enforced at: `resolveNodes` (`core/model/PartsModel.cpp:29`) and the ordering contract on `Placed` (`core/model/parts/Placement.h:94`).

- I10 — Zero-span parts never host** A part with no axial interior (a `Motor` node, `span` $\leq$ tolerance) is skipped as a candidate host, so a geometrically inert node can never generate spurious containment violations.

Enforced at: the covering-host filter in `sweepOverlaps` (`core/model/parts/Placement.cpp:183`).

- I11 — No-design is representable** An empty model is a first-class state — null root, `hasDesign()` false, size 0 — and every verb degrades cleanly against it rather than assuming a placeholder rocket.

Enforced at: the nullable `root_` (`core/model/PartsModel.h:259`), `hasDesign` (`core/model/PartsModel.h:161`), and `setMotor` refusing to attach into no design (`core/model/PartsModel.cpp:485`).

- I12 — Single writer** All mutable-behind-const caches (composite stamps, resolved placements, verdict, the motor's burnout latch) are single-writer by contract; concurrent access, even all-const, is a data race.

Enforced at: contract, stated where the caches live — `PartNode`'s class doc (`core/model/PartsModel.h:39`) and the burnout latch (`core/model/MotorModel.h:342`); nothing in the model layer takes a lock.

- I13 — Events fire in pairs** Every tree or leaf mutation verb fires its `Event` exactly twice — `before == true` ahead of the write, `before == false` after it — which is precisely the `begin*/end*` shape Qt's row-op contract requires.

Enforced at: the paired `emitEvent` calls bracketing every verb body in `core/model/PartsModel.cpp` (e.g. `attachSubtree`, `core/model/PartsModel.cpp:342`), consumed one-to-one by the tree view (`gui/RocketTreeView.cpp:32`).

- I14 — The core stays Qt-free** The model layer notifies through `std::function` callbacks only; the Qt adaptation lives entirely in `gui/`, so the CLI, tests, and sim core link no Qt.

Enforced at: the callback types `PartsModel::ChangeCallback` (`core/model/PartsModel.h:151`) and `RocketModel`'s registration hooks (`core/model/RocketModel.h:96`), with the sole Qt subscriber in `gui/RocketTreeView.cpp`.

Reading this list

I1, I2, and I12–I14 are architectural: the type system and the layering carry them, and a violation is a compile error or an API that does not exist. I3–I11 are behavioral: each has a specific guard in the code cited above and a corresponding regression test ([Section 13](#)). When extending the model, the discipline is to add to this list, not around it — a new mutation is a new routed verb (I2) that fires paired events (I13) and dirties its declared scope (I3).